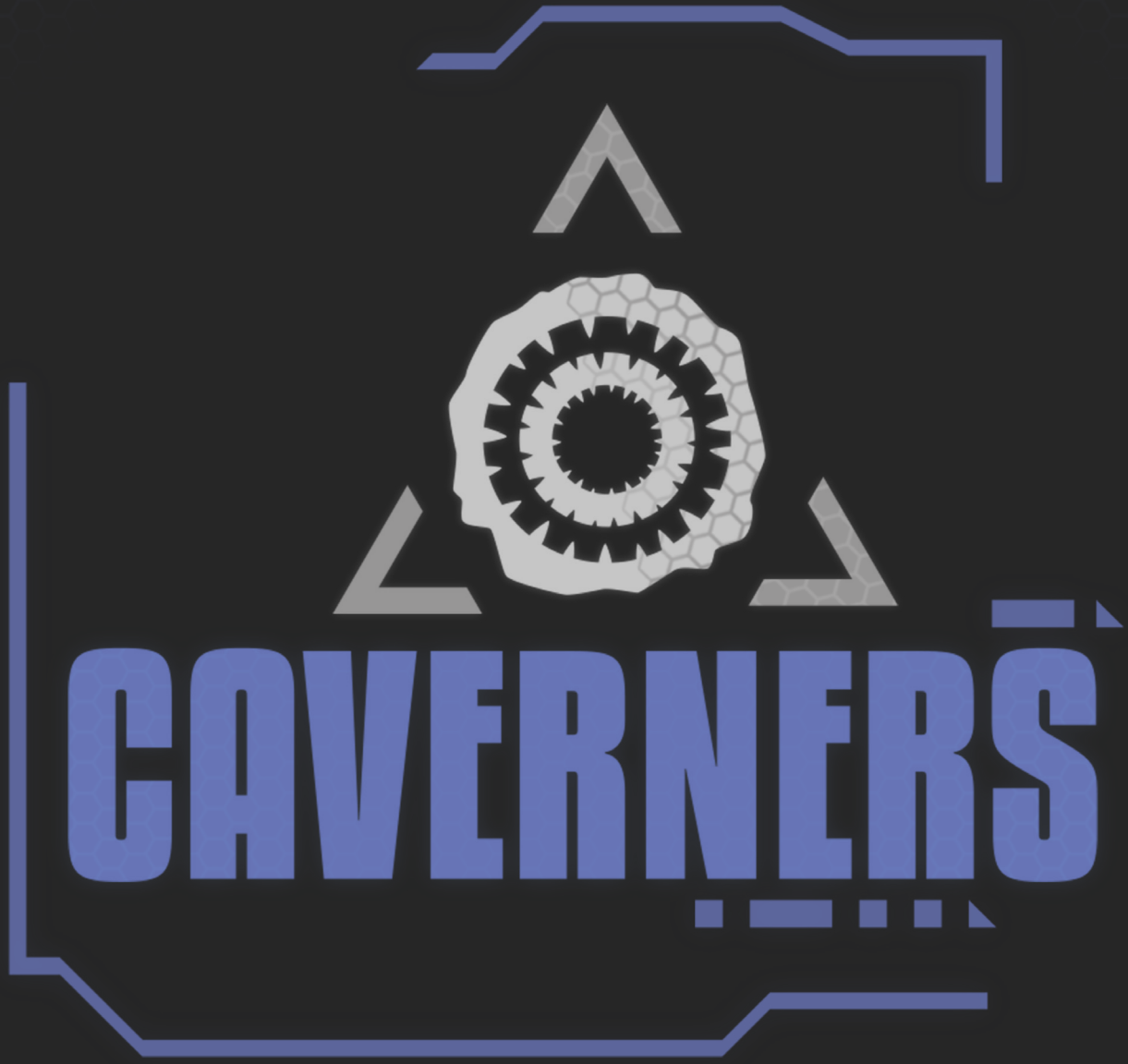


Fev - Mai 2024

Sommaire

- Présentation 5
- Game Système
- 3CS
- Fiche d'identité 6
- Simulation d'une partie 7
- Game Design 10
  - Intentions
  - Itérations
  - Noyau Jouet
  - Mécaniques
  - Shooter-Core
  - Ingrédients
  - Signes & Feedbacks
  - Tension ludique
  - Cohérence & émergence
  - Gamification
- Level Design 33
  - Metrics
  - Gestion de la difficulté
  - Apprentissage & OnBoarding
- Game Programming 37
  - Philosophie d'implémentation
  - Processus d'implémentation
  - Controller Design
  - Weapon Design
  - Mob Design
  - Système de jeu et rejouabilité
- Direction artistique Visuelle 64
  - Pitch
  - Intentions Générales
  - Inspirations
  - Mood Boards
  - Concept Arts
  - Itérations
  - Visuels
  - Visuels 2D
- Direction Artistique Sonore 109
  - Sound Effects (SFX)
  - Processus de production
  - Asset Sonores
  - Musique
- INTENTIONS FUTURES 124

Equipe :

Adrian

DA Visuelle

Modélisation 3D

Rigging

Maxime

Programming

Concept Art

Charadesign

Composer

Gabriel

DA Sonore

Programming

Documentation

Baolan

User Interface

Documentation

Samuel

Lead Project

Game Design

Level Design

Animation 3D

G.O.D

Game Overview Document

Présentation

Caverners est un jeu de tir à la première personne, aux traits de roguelite. Aux commandes d'un drone dans une base de minage attaquée, le joueur devra tenir le plus longtemps possible, permettant vague après vague l'extraction de minerai rare. A chaque mission seront mis à l'épreuve sa dextérité, sa vitesse, sa précision, mais aussi son sens tactique et son esprit d'adaptation, dans une lutte explosive contre les pestes locales.

Game Système

La structure de Caverners hérite directement de celle du roguelite. Constitué de parties courtes, variant entre 15 minutes et une heure, le jeu présente des objectifs successifs par vagues à accomplir. Chaque objectif accompli marque un léger temps de pause, et donne le choix au joueur à un panel d'améliorations, tirées au hasard pour la partie en cours. Chaque partie se présente donc comme une nouvelle tentative, pour démontrer les meilleures performances possibles et vaincre le plus de vagues possibles.

L'objectif principal du joueur est de vaincre 20 vagues successives, après quoi il passe en mode endless, où chaque vague supplémentaire gagnée lui rapporte des points, et une récompense honorifique.

Fiche d'identité

Pitch:

Le joueur est aux commandes d'un drone humanoïde, activé près d'une base de minage attaquée par des aliens. Son but est de la défendre, vague après vague, en redoublant de vitesse et de précision.

Support:

Windows, Mac IOS, linux

Cible:

Joueurs expérimentés dans les FPS, découvrant ou non le système des roguelites, et à la recherche d'une expérience nerveuse et technique, rajoutant déplacements à haute vitesse au genre du tir.

Système:

Chaque partie est une tentative différente pour aller le plus loin possible dans les vagues, et de défendre la base le plus longtemps possible. L'objectif est de défendre 20 vagues, après quoi toute vague défendue rapporte des points supplémentaires.

Chaque partie présente au joueur un build différent, par des améliorations aléatoires à choisir à la fin de chaque vague, lui permettant de devenir de plus en plus puissant, d'une façon qui sera propre à chaque partie.

Genres:

Fast FPS, RogueLite

Jeu Solo

PEGI 12 (violence, effusions de sang et armes à feu)

Camera

Une caméra à la première personne, montrant la vue depuis la tête du drone. Le joueur peut l'orienter dans toutes les directions pour diriger le drone et regarder autour de lui.

La caméra à un FOV variable, évoluant en fonction de la vitesse, et une interface rajoutée affichant de nombreuses informations : points de vie, valeur de turbo, vitesse actuelle, améliorations possibles... Pendant les vagues, apparaissent finalement des marqueurs, qui suivent les objectifs et indiquent leur position, ainsi que d'autres informations les concernant.

Character

Le joueur contrôle un drone, activé à distance dans une base de minage attaquée. Son but est de défendre la base contre le plus de vagues d'attaque successives possible. Il est équipé de nombreuses mécaniques de déplacement, lui permettant de se mouvoir dans l'espace, de gagner et de conserver de la vitesse. Il peut en particulier courir, glisser, sauter, et activer un boost temporaire au prix d'une ressource collectée sur place. Il possède également un set d'armes dont il peut se servir, pouvant viser, tirer, recharger, et changer d'armes en main. Il aura finalement l'option de s'améliorer entre chaque vague, en choisissant parmi un panel d'options proposées aléatoirement.

Controller

Les contrôles sont définis par défaut mais éditables depuis les menus du jeu, et permettent la prise en main du drone au cours de la mission.

Caméra première personne

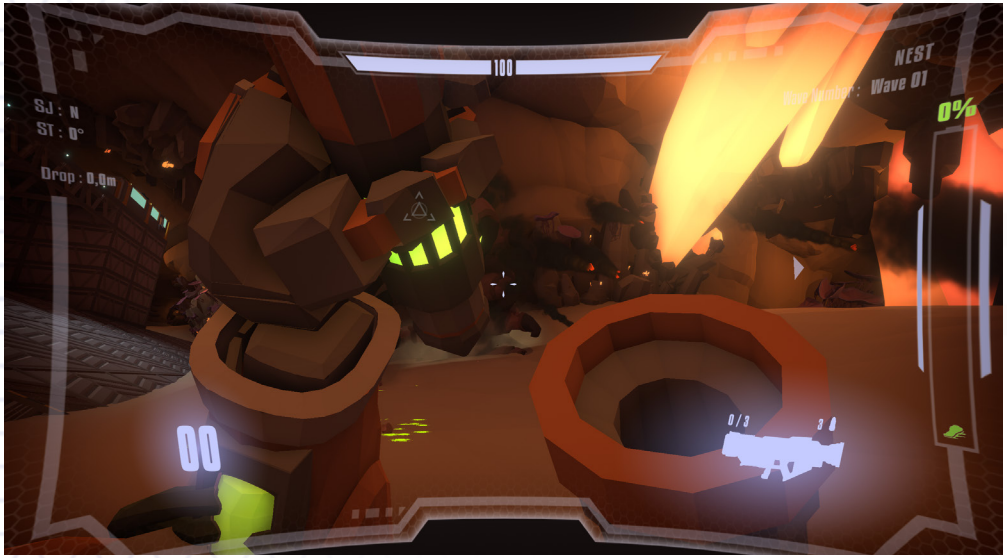
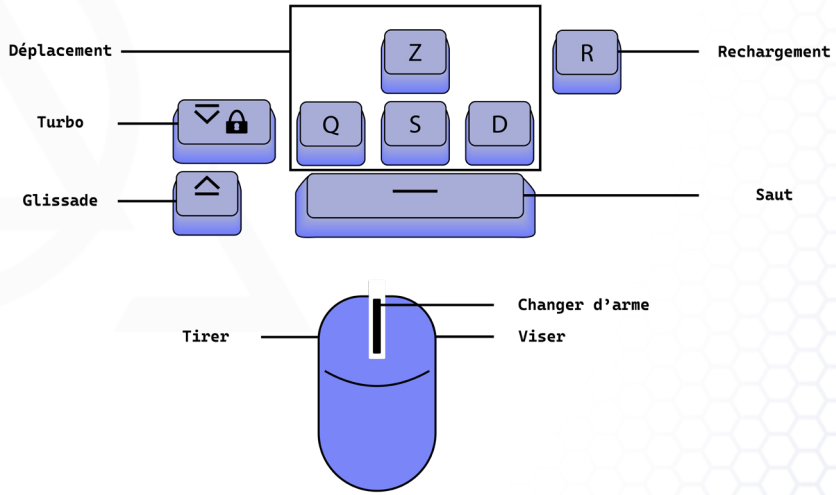
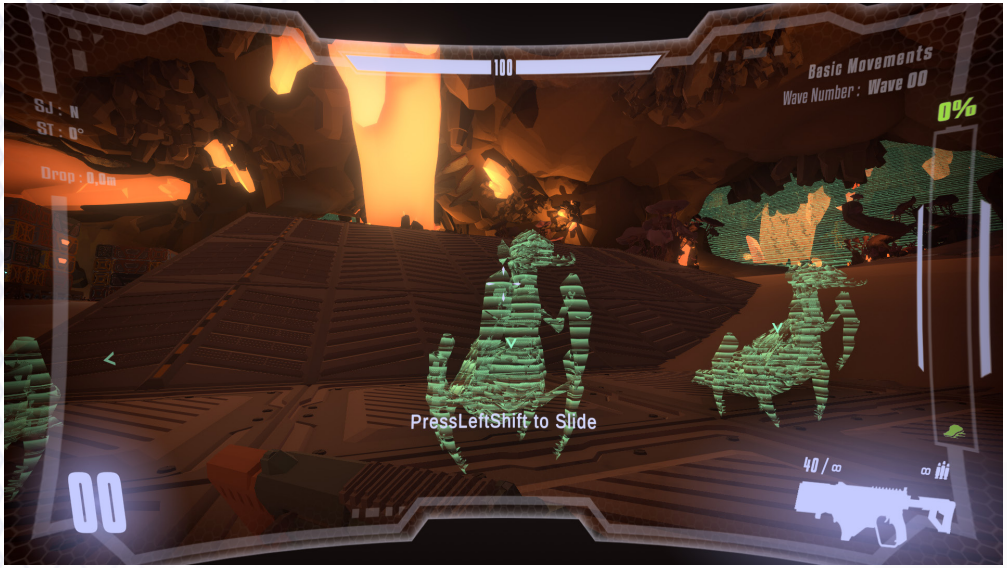


Schéma Input

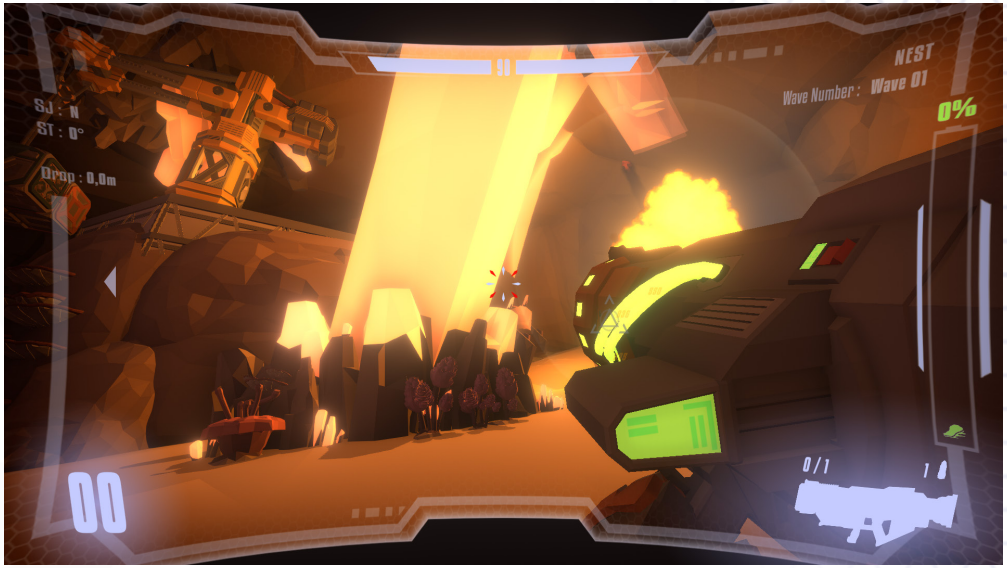


Simulation d'une partie

Tutoriel



1er vague de Vers après le tutoriel



Amélioration fin de vague

Seconde vague de Nexus

G.D.D

Game Design Document

G.D
Game Design

Game Design

Intentions

Reboot

Nous avons effectué un *Reboot* (départ à zéro) lors du début de ce second semestre de troisième année, après avoir constaté que notre premier projet ne nous convenait pas. Nous souhaitions créer un *Puzzle Game,* mais ce genre était trop méconnu de notre équipe, et nous n'arrivions pas à le faire marcher avec la méthode *Bottom-Up*.

De plus, la tension ludique des Puzzle Games s'est avérée particulière et difficile à analyser. Nous avons donc décidé de suivre le conseil des professeurs et de repartir à zéro.

Volonté

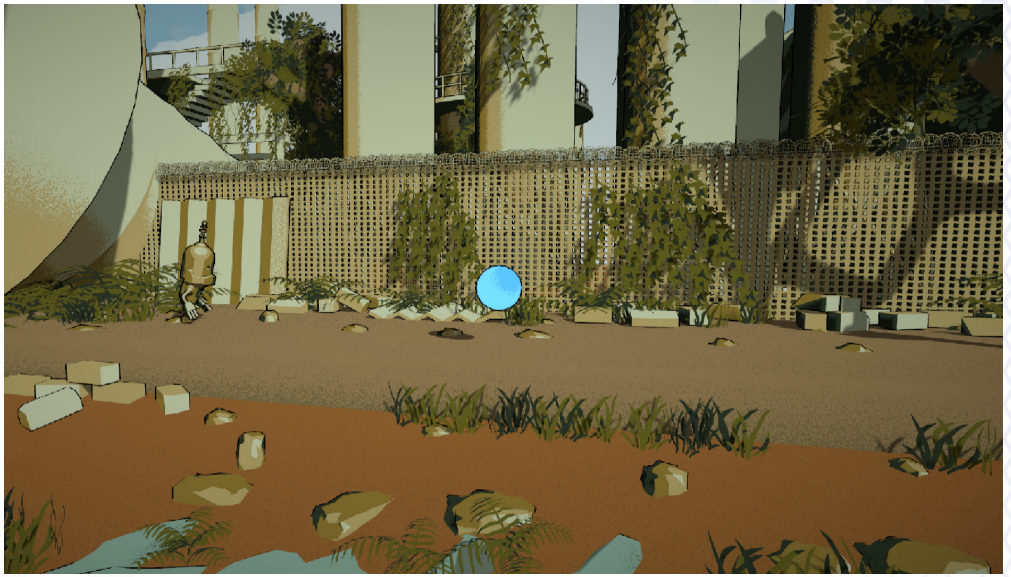
Nous avons donc décidé de partir sur un genre de jeu que nous avons tous pu expérimenter plusieurs fois et en profondeur en tant que joueurs : le *First Person Shooter (FPS)* ou *Jeu de Tir à la première personne*.

Nous avons alors développé comme ambition de créer un jeu du sous-genre *Fast-Shooter,* c'est-à-dire proposer un mélange entre des challenges de visée et de timing, ainsi que des challenges de déplacement à haute vitesse.

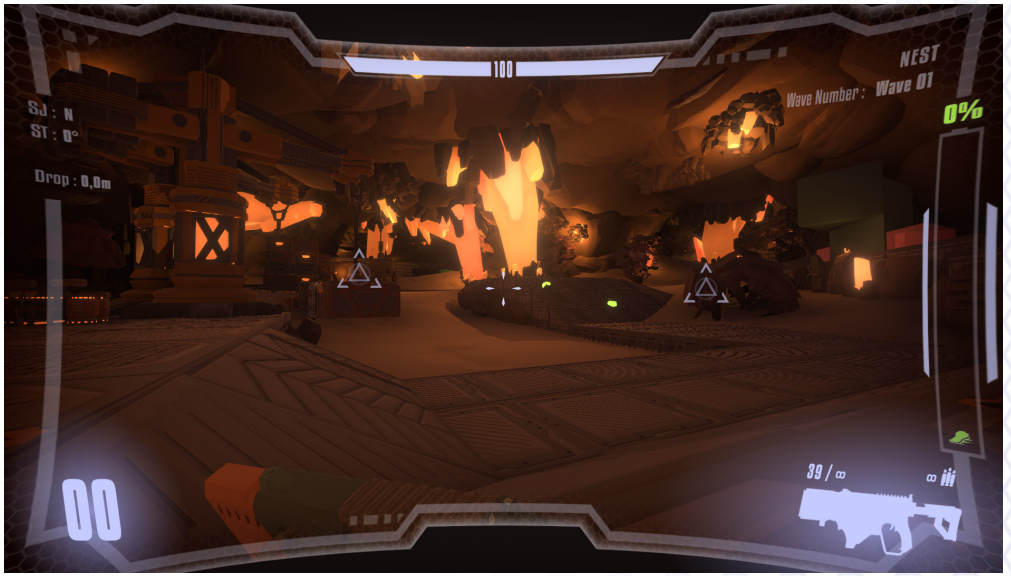
Cette ambition de *Fast-Shooter,* c'est également une promesse d'un *Game Feel* très développé, tourné autour de la vitesse et de la frénésie du combat.

Afin de suivre la méthode *Bottom-Up*, nous nous sommes dirigés vers des itérations successives de Controllers et de différents types d'armes, en les corrigeant / modifiant à répétition.

Ancien projet



Nouveau Projet



Références Gameplay

Nous nous sommes tout d'abord inspirés de classiques du genre comme références de Gameplay, comme Quake ou Doom.

Quake Saga

Doom Saga

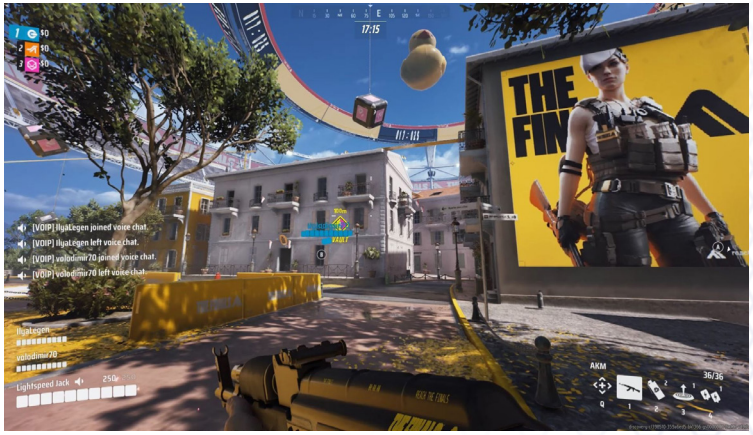
Pour ce qui est du Game Feel, Warstride Challenge a été notre référence principale. Ses mécaniques de déplacement de prise de vitesse nous ont inspiré, ainsi que la manière dont le jeu propose ses challenges :

- Des challenges de déplacement plutôt que de visée
- Des challenges de Tracking sur une très courte période de temps

Warstride Challenge

Notre dernière référence a été The Finals, chez qui nous avons pioché le Level Design en arène, la diversité des armes, ainsi que les sensations de vitesse et de frénésie, proche des jeux de type Arcade.

The Finals



Itérations

Itération 1

Dans cette itération, le personnage peut se déplacer, courir, sauter et glisser.

Les mécaniques ne sont pas reliées entre elles, elles sont juste des manières différentes de parcourir l'espace proposé au joueur.

La partie FPS est apportée par un Crosshair réactif, ainsi que des cibles destructibles et immobiles disposées dans l'espace de jeu. Le joueur a accès à un fusil d'assaut et un arc.

Fusil d'assaut :

Le fusil d'assaut est une arme automatique (tir en continu), ne possédant pas de recul, et dont les dégâts varient avec la distance.

Arc :

L'arc est une arme au coup par coup, qui possède un temps de chargement, et dont le projectile est physicalisé (il subit la gravité). Les dégâts sont fixes, mais la portée de l'arc est proportionnelle au temps de chargement du tir, avec un maximum prédéterminé.

Il n'y a ni recul, ni Bloom ou imprécision lors du déplacement : notre idée dans cette itération est de permettre au joueur de se déplacer rapidement dans l'espace, et de se concentrer au milieu de ses déplacements afin de détruire les cibles.

Nous avions en tête la volonté de créer des séquences de Gameplay très cinématiques : un tir au milieu d'un saut, ou bien d'une glissage, etc.

Itération 2

Dans cette itération, nous avons ajouté des archétypes d'armes disponibles dans les jeux de tir. Nous avons donc rendu disponibles au joueur des armes comme :

- *Une SMG* (haute cadence de tir, dégâts légers)
- *Un Fusil à Pompe* (tir au coup par coup, dégâts de zone)
- etc.

Nous avons également décidé d'ajouter des armes au comportement « exotique », qui possèdent un fonctionnement unique et se distinguent des armes plus classiques. Nous avons donc rendu disponibles au joueur des armes comme :

- Un Lance-Roquettes (tir de projectiles explosifs)
- Des Couteaux de Lancer (tir instantané de projectiles ayant du *Travel Time* et du *Bullet Drop*)
- etc.

Cette itération avait pour but de tester de multiples armes, aussi bien classiques que singulières, afin d'expérimenter avec les différentes manières d'aborder l'aspect « Shooter » de notre jouet.

Nous avons tiré de cette itération énormément d'armes différentes, et une envie de créer une composante Roguelike lors de la Gamification, grâce à laquelle le joueur pourrait personnaliser le comportement de son arme.

Itération 3

Dans cette itération, nous avons ajouté les mécaniques de prise de vitesse.

Cette itération avait pour but de se concentrer sur les déplacements du Controller. Notre démarche a été d'observer les comportements joueur vis-à-vis des trois mécaniques de base (Courir, Sauter, Glisser) et d'observer quelles tendances nous souhaitions récompenser afin d'inciter le joueur à les effectuer.

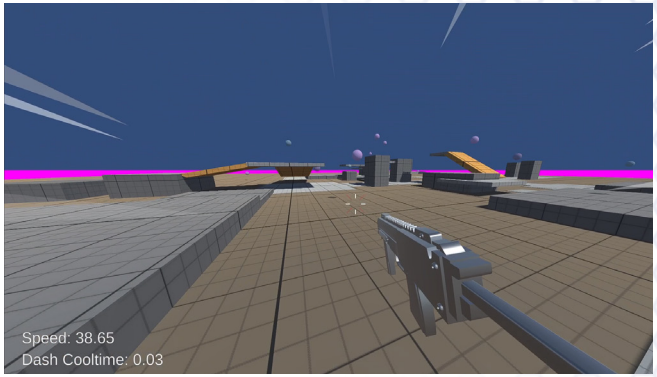
Nous avons donc récompensé ces comportements par des gains de vitesse lors de certains enchaînements d'actions.

Itération 4

Dans cette itération, nous avons ajouté les différents objectifs.

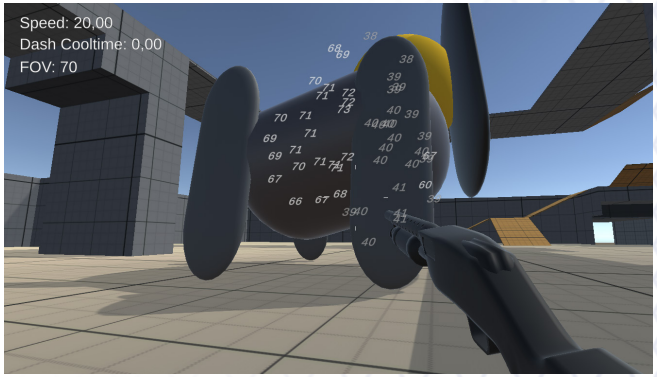
Cette itération avait pour but de tester différents types d'objectifs, qui cherchaient à pousser le joueur à se déplacer et à vouloir aller vite. Nous avons également supprimé l'Input de course à partir de cette itération : le joueur n'a aucun intérêt à se déplacer sans courir, il était donc inutile de lui rajouter un Input afin de le faire. Le personnage court donc de base.

Itération 1



Itération 1

Itération 1



Controller

Controls

Les contrôles sont définis par défaut mais éditables depuis les menus du jeu, et permettent la prise en main du drone au cours de la mission.

Buffer

Il existe un « *Buffer* » dans le jeu, permettant de réduire légèrement le challenge de *Timing* des *Inputs*.

Il s'agit d'un système enregistrant les *Inputs* effectués quelques millisecondes trop tôt, et les restituant comme s'ils avaient été faits au bon *Timing*.

(ex : le joueur appuie sur *SpaceBar* pour faire sauter son personnage, mais son personnage n'était pas encore tout à fait au sol. Le Buffer enregistre l'Input, et si le personnage touche le sol suffisamment vite après cet enregistrement, le *Buffer* ré-effectue l'*Input* et le personnage saute de nouveau dès qu'il touche le sol).

Schéma Input

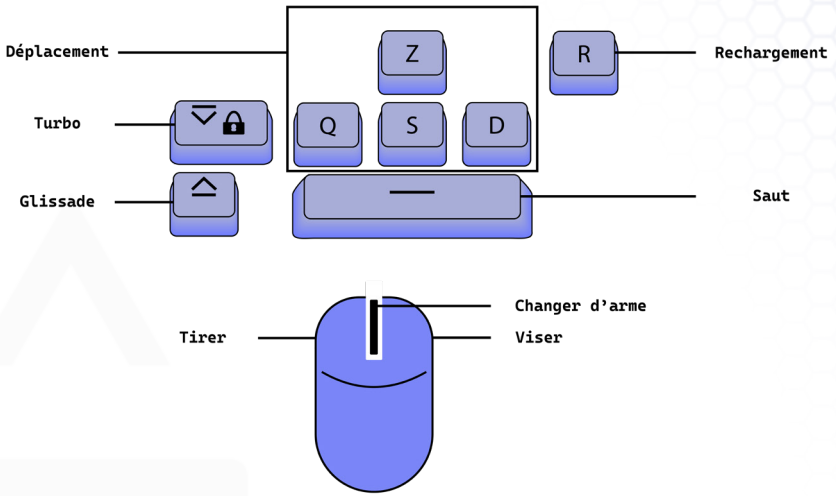
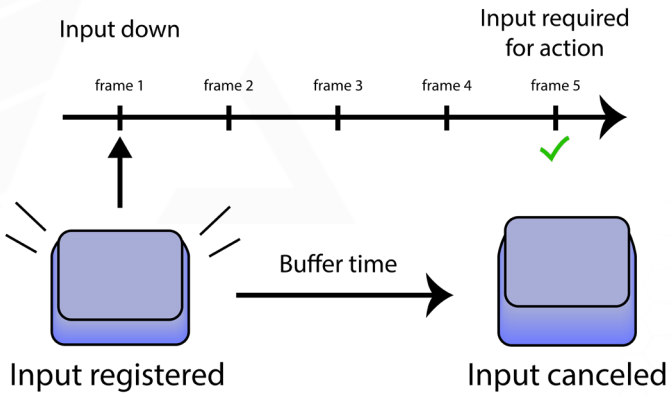


Schéma Buffer

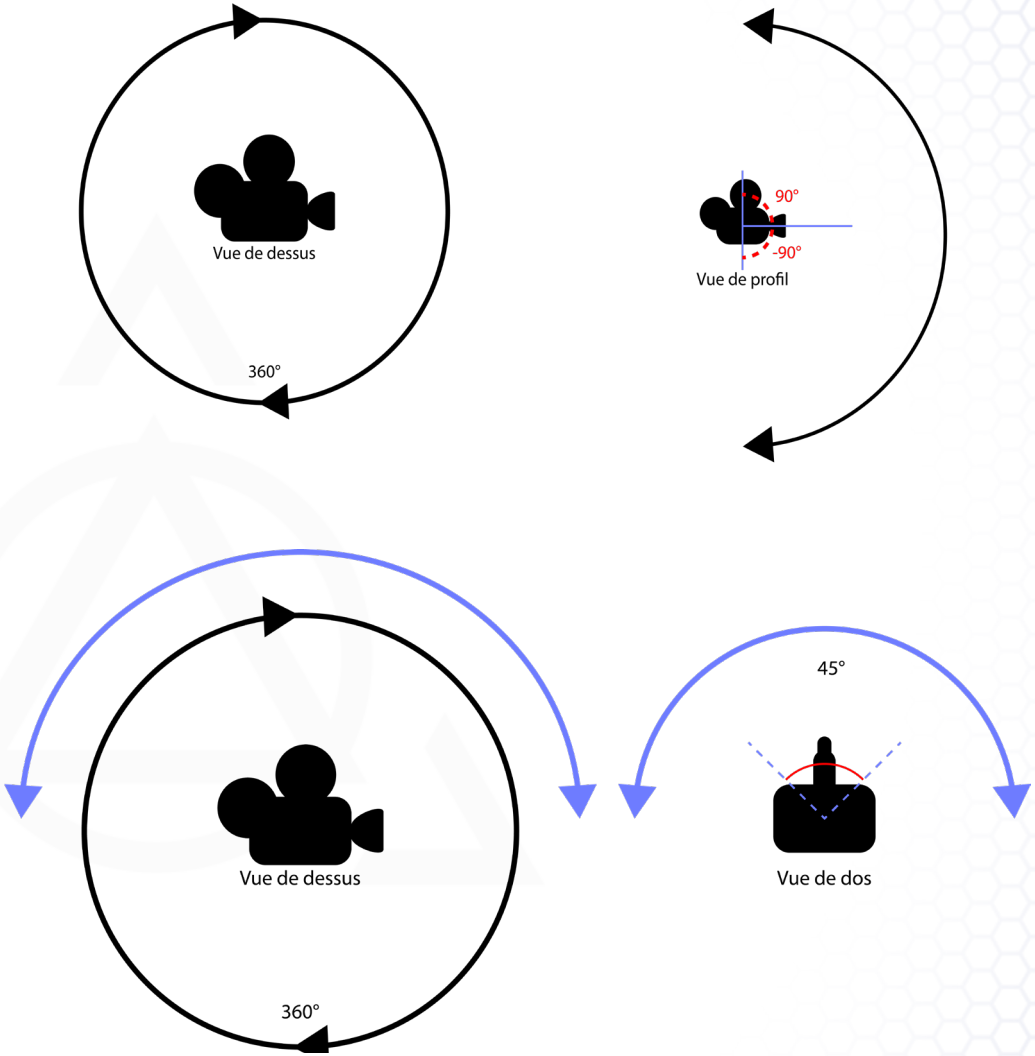


Caméra

La caméra est une caméra à la première personne, réactive aux mouvements de souris du joueur, ainsi qu'à l'état du personnage jouable. Le joueur peut la faire tourner sur l'axe Y grâce aux mouvements de souris horizontaux, et sur l'axe X grâce aux mouvements de souris verticaux. Cependant, la caméra est Clamp d'un delta de 90° sur l'axe X, et le joueur n'a pas de contrôle sur sa rotation autour de l'axe Z.

Lorsque le personnage est en glissade, la caméra pivote légèrement autour de l'axe Z en fonction de la direction de la glissade et des Inputs souris horizontaux du joueur.

Schéma Caméra



Mécaniques

Déplacement

Lorsque le joueur *Input* une combinaison de ZQSD, il se déplace dans la direction des Inputs, relativement à l'orientation de la caméra.

Les déplacements sont différents en fonction de l'état du personnage :

- S'il est en train de glisser, la direction du personnage est fixée à la direction de la glissade
- S'il est en l'air, les déplacements du personnage sont multipliés par un coefficient réducteur appelé « *Air Control* ». Il est donc plus difficile pour le joueur de modifier la direction de son personnage lorsque celui-ci ne touche pas le sol.

Lorsque le joueur cesse d'enregistrer des Inputs de déplacement, la vitesse du personnage chute à zéro très rapidement.

Glissade

Lorsque le joueur *Input LeftShift* et que le personnage dispose de suffisamment de vélocité, celui-ci enclenche une glissade dans la direction de la caméra au moment de l'*Input*.

Cette glissade augmente temporairement sa vitesse, et déplace le personnage dans la direction de la glissade de manière non-modifiable. Si le personnage ne touche pas le sol, mais est suffisamment proche de celui-ci le système va venir le « plaquer » au sol et effectuer la glissade comme si le personnage touchait le sol de base.

Lors de la glissade, la *Hitbox* du personnage est réduite de moitié, ce qui est montré par un *Feedback* de caméra.

Lorsque le personnage enclenche une glissade, le système va déclencher un *Timer* très court, durant lequel le personnage gagne de la vitesse et est bloqué dans cet état de glissade. Une fois le *Timer* ayant atteint zéro, le personnage se met à perdre de la vitesse de manière exponentielle (plus important encore que s'il était simplement à l'arrêt), mais redevient libre de ses actions (il peut arrêter de glisser, sauter, ou bien continuer à glisser si le joueur maintient l'*Input*).

Lorsque le personnage sort de l'état de glissade, il se tourne et se dirige automatiquement dans la direction actuelle de la caméra, qu'elle soit identique ou différente de celle de la glissade.

Schéma direction en glissade

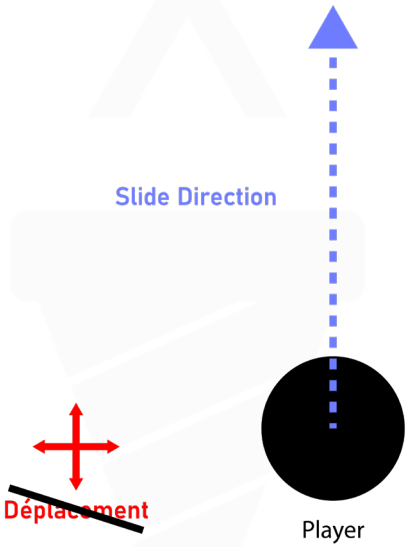
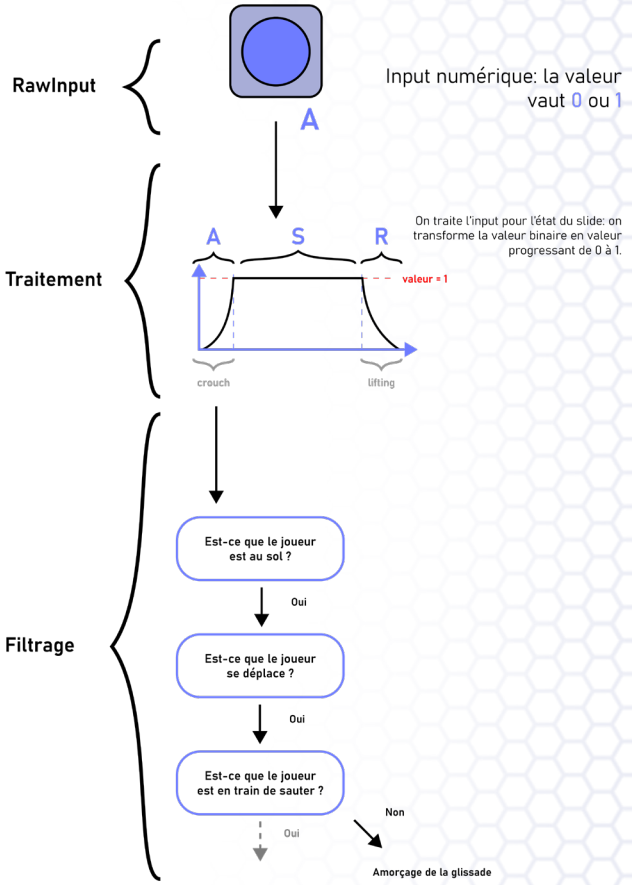


Schéma Glissade



Saut

Lorsque le joueur *Input SpaceBar*, le personnage effectue un saut.

Il quitte le sol et gagne temporairement de la vélocité verticale et horizontale. La durée du saut varie en fonction de la durée de l'*Input*, afin de proposer au joueur un saut analogique.

La variation de hauteur et de longueur du saut passe du simple au double en fonction de la durée de pression de l'*Input*.

Accumulation et conservation de vitesse

Notre intention étant de produire un jeu du genre *Fast-Shooter*, nous avons commencé par poser une définition sur ce terme.

L'aspect *Shooter* est développé plus bas.

L'aspect *Fast*, nous l'avons regroupé sous ce paradigme : proposer des mécaniques liées au déplacement permettant d'accumuler et de conserver de la vitesse. Dans ce but, il nous fallait alors gérer la vitesse avec précision.

La gestion de la vitesse du personnage est effectuée par 2 paramètres.

- Le premier est « l'*Adder* » qui constitue la vitesse principale du personnage. C'est une valeur numérique qui possède une valeur de base non-nulle (vitesse de base du personnage) et qui va augmenter grâce à certaines actions.

Cette valeur augmentée va ensuite pouvoir être conservée tant que le joueur ne commet pas d'erreur ou n'effectue pas d'actions entraînant une décélération.

- Le second est le « *Bonus* » qui constitue de la vitesse supplémentaire temporaire. Cette vitesse va être gagnée lors d'une action, mais immédiatement perdue lorsque l'action prend fin, sans qu'il n'existe de manière de la conserver.

Schéma saut analogique

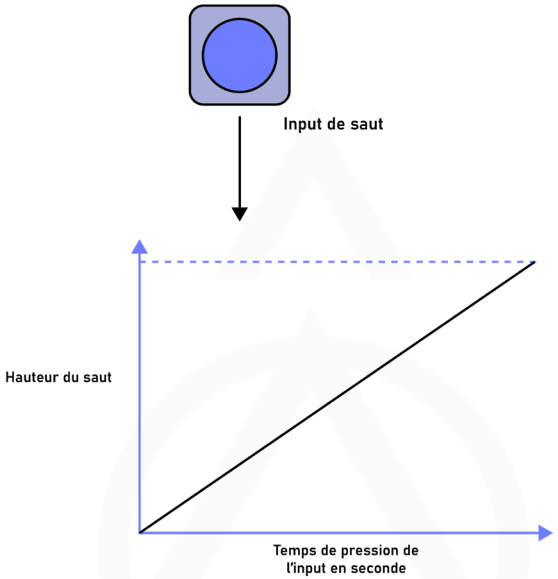
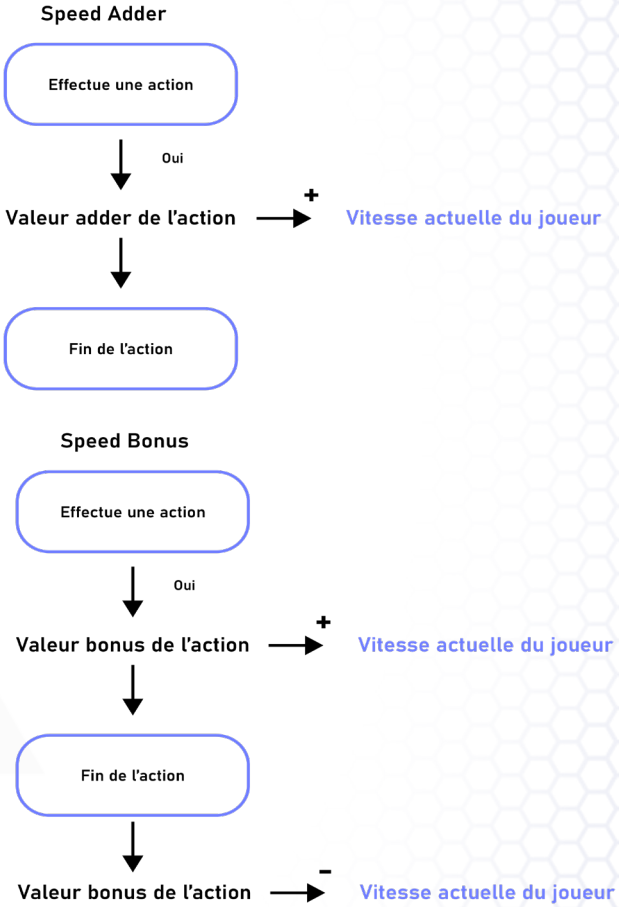


Schéma de la vitesse (Adder + Bonus)



Slide-Hopping

Lorsque le personnage saute alors qu'il est en état de glissade, alors il effectue un *Slide-Hop* et gagne de la vitesse en *Adder*.

Cette mécanique permet d'inciter le joueur à ne pas rester en glissade trop longtemps (cela le ferait fortement décélérer de toute manière) ainsi qu'à alterner des phases au sol et des phases aériennes

Frame-Hopping

Lorsque le personnage saute à l'instant où il touche le sol (*Frame Perfect*), alors il effectue un *Frame-Hop* et gagne de la vitesse en *Bonus*.

Cette vitesse en *Bonus* se dissipe dès que le personnage finit son saut, c'est-à-dire lorsqu'il touche le sol à nouveau. Il est possible de conserver cette vitesse en *Bonus* en effectuant des *Frame-Hops* à la suite (le joueur doit donc *Timer* parfaitement son Input de saut, à chaque fois que le personnage touche le sol).

Il ne s'agit pas d'une réelle « conservation » de vitesse, mais plutôt d'une compensation de la perte du précédent *Frame-Hop* par un nouveau.

Cette mécanique permet au joueur de pouvoir déplacer le personnage rapidement s'il réussit à enchaîner les bons *Timings*, et ainsi éviter les phases de *Gameplay* trop linéaires à base de course unique.

Schéma Slide-Hopping

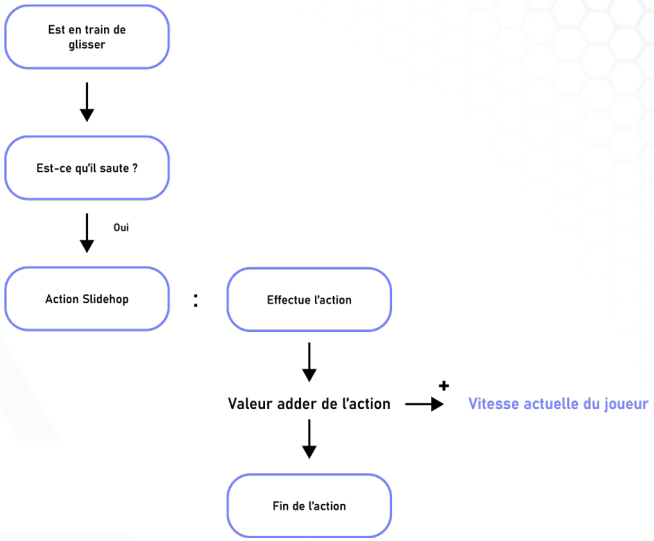
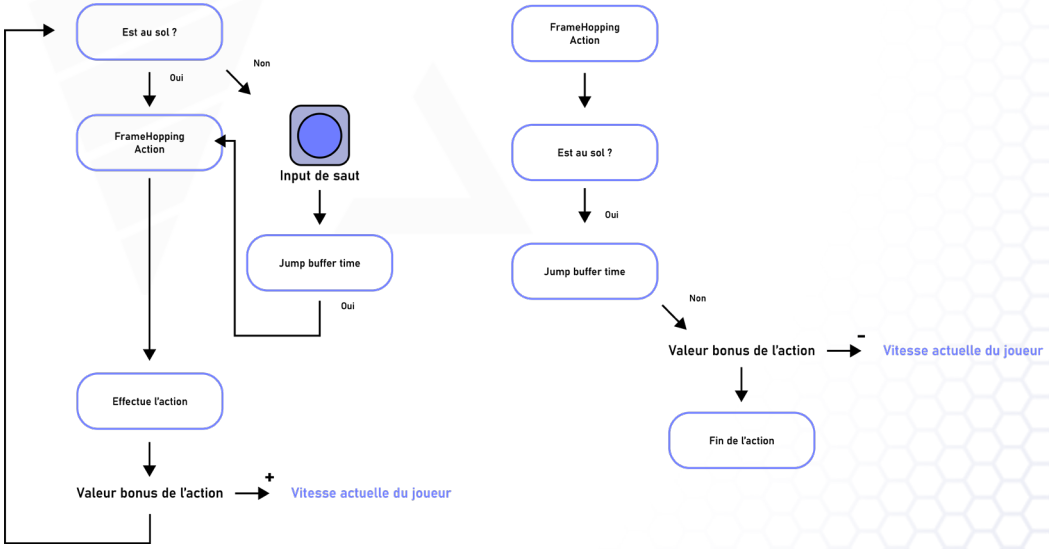


Schéma Frame-Hopping

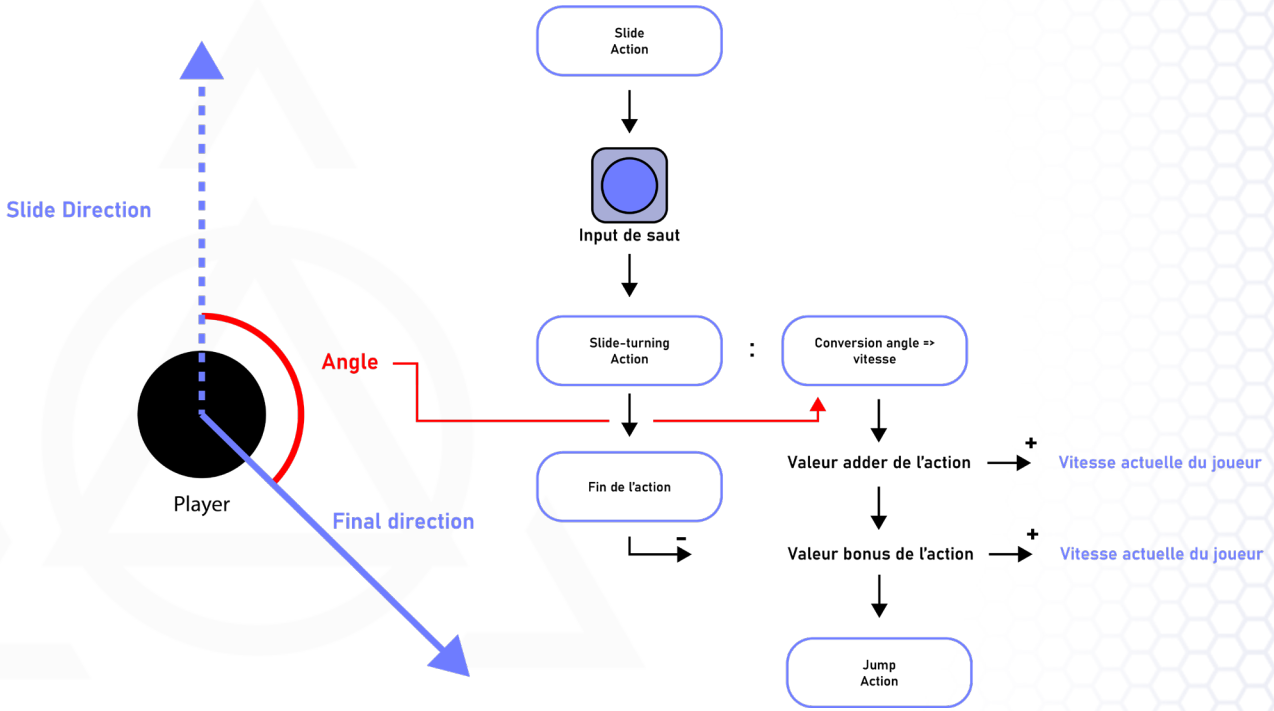


Slide-turning

Lorsque le joueur effectue une glissade, le système enregistre sa direction de départ. Dès que le joueur arrête sa glissade, le système enregistre sa nouvelle direction (direction de sa caméra). Le personnage va subir une augmentation de sa vitesse en adder en fonction du delta de rotation entre ces deux directions : plus le joueur se sera tourné lors de sa glissade (avec un maximum de 180° i.e. le personnage glisse dans une direction et regarde dans l'autre), plus le gain de vitesse sera important.

Cette mécanique vise à récompenser les joueurs qui utilisent leur glissade dans des contextes non-linéaires, et qui réussissent à conserver leur fluidité de déplacement malgré les différentes collisions.

Schéma Slide-Turning



Slide-landing

Lorsque le personnage effectue une glissade à l'instant où il touche le sol, alors il effectue un *Slide-Landing* et gagne de la vitesse en *Adder*.

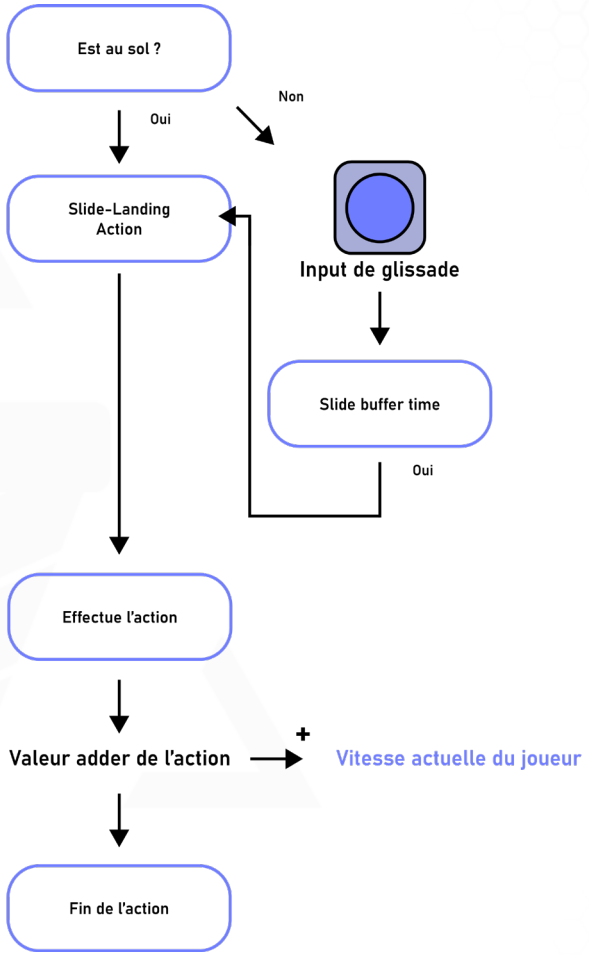
Cette mécanique vient compléter le *Frame-Hopping*, en proposant au joueur 2 *Inputs* différents lorsque son personnage touche le sol :

- Soit il saute et effectue un *Frame-Hop*
- Soit il glisse et effectue un *Slide-Landing*

Cela permet d'inciter le joueur à ne pas rester en saut trop longtemps, et à alterner les phases au sol et les phases aériennes.

De plus, cette mécanique est la réciproque du *Slide-Hopping*. En effet, si le personnage saute depuis une glissade, il gagne de la vitesse, et s'il glisse depuis un saut (si le joueur Input la glissade à la bonne *Frame*), il gagne de la vitesse.

Schéma Slide-Landing



Turbo

Le *Turbo* est une ressource que le personnage peut collecter en détruisant des « Œufs » disposés sur la *Map* en tirant dessus. Le *Turbo* se conserve dans une jauge, possédant un maximum.

Lorsque le joueur *Input CapsLock*, le personnage va se mettre à consommer en continu du Turbo, afin de gagner temporairement énormément de vitesse en *Bonus*.

Dans cet état, le personnage ne peut pas tirer ou glisser : il ne peut que courir.

Si le joueur Input un saut ou une glissade, le personnage sort de l'état de *Turbo* : il cesse de consommer du *Turbo* et perd la vitesse en *Bonus*.

L'idée de cette mécanique est de proposer au joueur une Reward consommable, qui lui permet d'esquiver, de s'échapper ou bien de rejoindre un point de l'espace bien plus vite.

Cependant, elle n'est pas conçue pour permettre au joueur d'accumuler de la vitesse. En effet, cette mécanique agit de manière isolée et ne se cumule pas avec les autres. C'est donc pour ces raisons que l'on perd la vitesse en *Bonus* à l'instant où l'on Input une action différant des déplacements simples.

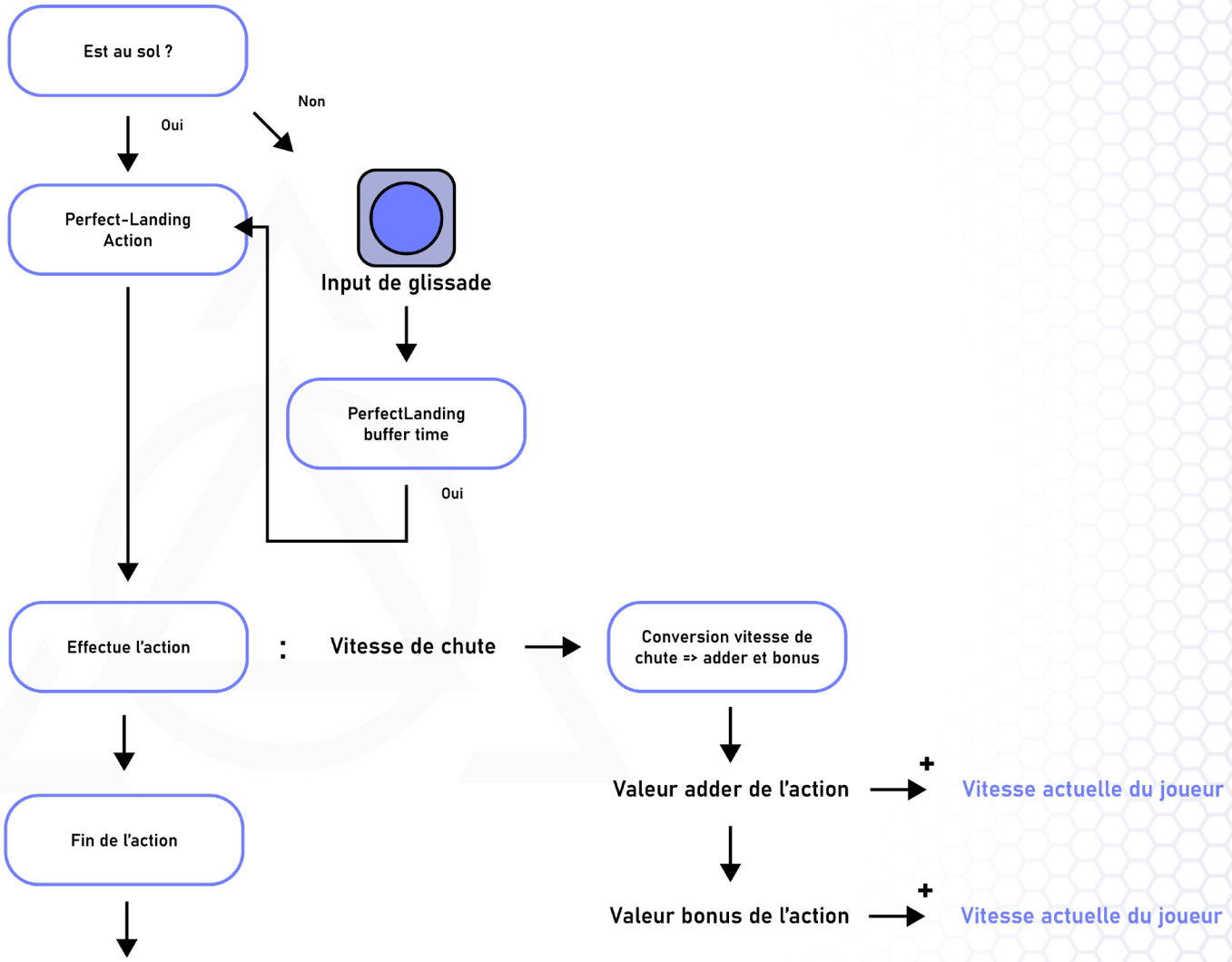
Perfect Landing

Le *Perfect Landing* est une mécanique qui fonctionne de pair avec le *Slide-Landing* et le *Frame-Hopping*. Lorsque le personnage tombe d'une hauteur plus grande que celle d'un simple saut (lorsqu'il se *Drop* depuis une hauteur par exemple), le système enregistre la vélocité verticale qu'il accumule.

Ensuite, si le joueur réussit à faire effectuer à son personnage un *Slide-Landing* ou un *Frame-Hop* (c'est-à-dire réussir à Input un saut ou une glissade à la *Frame* où le personnage touche le sol), alors le personnage va effectuer un *Perfect Landing* et gagner une proportion de sa vélocité verticale en vélocité horizontale en *Adder*.

Cette mécanique est la réciproque du *High-Fall Landing* ,afin de proposer au joueur une liberté de mouvement vertical, ainsi que des situations de *High-Risk / High-Reward*.

Schéma Perfect-Landing



Décélération

Friction au sol

Lorsque le personnage se trouve en contact avec le sol et n'est pas en état de Turbo, de glissade ou de saut, alors la friction au sol va considérablement le ralentir.

Cela incite le joueur à alterner des phases de *Gameplay* aériennes et de glissade, ainsi qu'à le pousser à découvrir les mécaniques de prise de vitesse.

High-fall landing

Le *High-Fall Landing* est la mécanique réciproque au *Perfect Landing*.

Le *High-Fall Landing* est une mécanique qui fonctionne de pair avec le *Slide-Landing* et le *Frame-Hopping*. Lorsque le personnage tombe d'une hauteur plus grande que celle d'un simple saut (lorsqu'il se *Drop* depuis une hauteur par exemple), le système enregistre la vélocité verticale qu'il accumule.

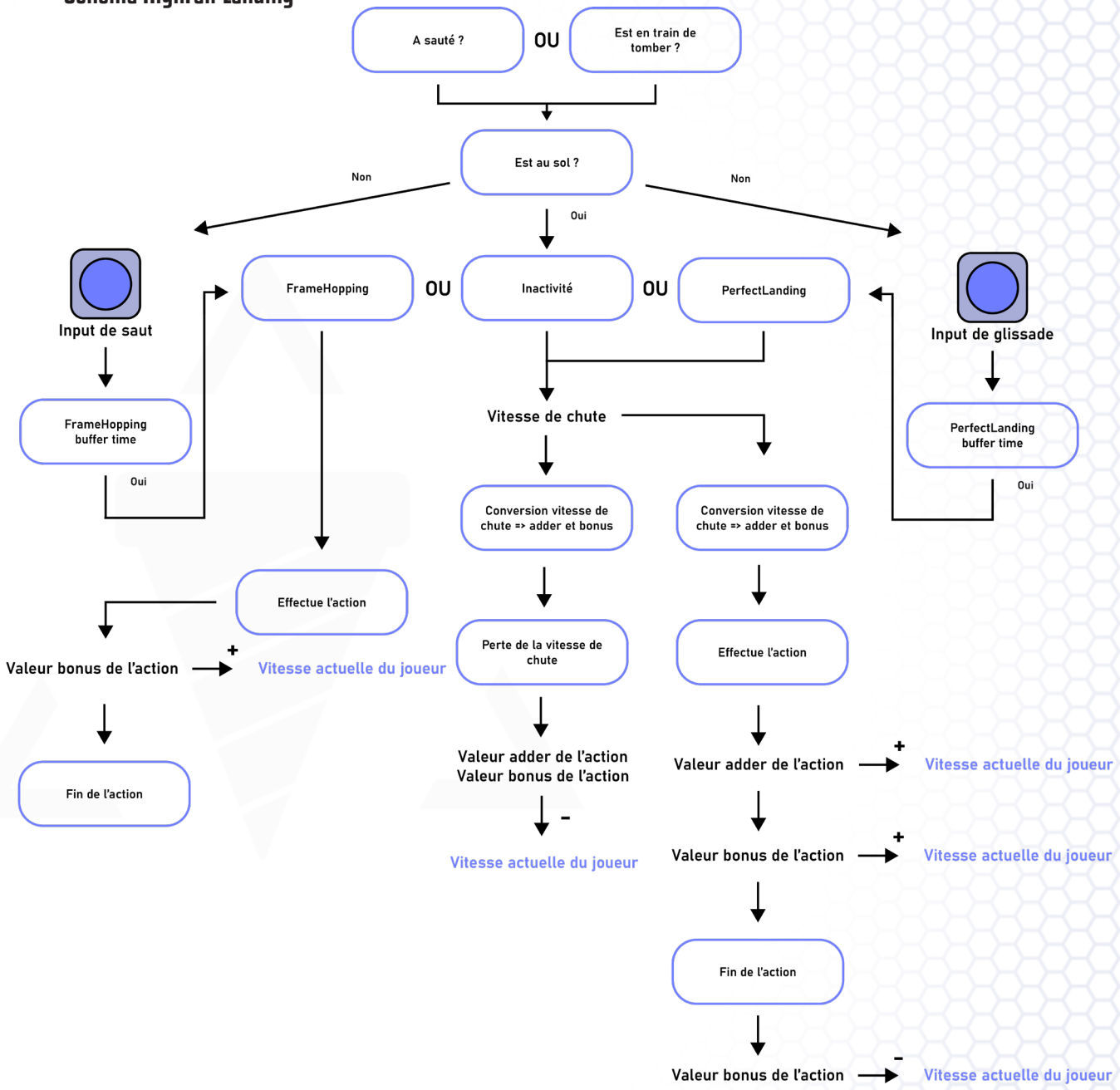
Ensuite, si le joueur échoue à faire effectuer à son personnage un Slide-Landing ou un *Frame-Hop* (c'est-à-dire échouer à *Input* un saut ou une glissade à la *Frame* où le personnage touche le sol), alors le personnage va effectuer un *High-Fall Landing* et perdre une proportion de sa vélocité verticale en vélocité horizontale en *Adder*.

Cette mécanique est la réciproque du *Perfect Landing*, afin de proposer au joueur des situations de H*igh-Risk / High-Reward* : plus le personnage tombe de haut, plus sa vélocité va être impactée par l'échec ou la réussite de l'atterrissage. De plus, plus la chute est longue, plus la fenêtre d'opportunité se réduit.

Inactivité

Lorsque le joueur cesse d'envoyer des Inputs de déplacement ou d'action au système, alors celui-ci va considérablement réduire la vélocité du personnage, jusqu'à zéro. Cette mécanique cherche à pousser le joueur à être constamment en train de se déplacer, et à ne pas être statique lors de ses tirs.

Schéma HighFall-Landing



Shooter-Core

Notre intention étant de produire un jeu du genre *Fast-Shooter*, nous avons commencé par poser une définition sur ce terme.

L'aspect *Fast* est développé plus haut.

L'aspect *Shooter,* nous l'avons regroupé sous ce paradigme : proposer une expérience de tir, avec des challenges axés autour de la précision et du *Tracking*.

COMPORTEMENT DES ARMES

Afin de répondre correctement à notre cible (*Core-Gamers*), les mécaniques de tir de notre jouet s'orientent autour de la précision du *Gunplay*.

Au préalable, beaucoup d'options ont été testées :

- Différents types d'armes
- Différents types d'Inputs
- Des temps de chargement
- Le Bullet Drop
- Le Bloom
- Le Spray pattern / Recoil

Après une phase de tri, seuls des archétypes suffisamment communs au genre FPS ont été conservés (Fusil à pompe, SMG). Ces armes disposent de propriétés permettant au joueur un grand contrôle sur son *Gunplay* : une excellente réactivité de l'arme, pas de temps de chargement, des tirs effectués en Raycasts, et pas de recul.

SYSTEME D'ARMES

Les armes sont regroupées sous 2 catégories : les armes primaires et secondaires. Il existe pour l'instant 2 armes primaires et 1 arme secondaire.

Cependant, il existera à terme 3 de chaque. Les armes primaires sont des armes archétypes du genre FPS, et dont les balles sont fournies en quantité illimitée au joueur : elles constitueront son *Gameplay* principal (à moins qu'il réussisse à améliorer suffisamment son arme secondaire lors de sa *Run* pour qu'elle devienne viable en continu.

Il s'agit des 2 armes suivantes :

- Un fusil d'assaut (AR)

Illustration AR

C'est une arme à longue portée et haute cadence de tir, disposant d'un nombre important de balles dans chaque chargeur. Elle tire via un *Raycasting* précis et ne dispose pas de recul ou de Bloom.

- Des fusils à pompe en Akimbo (Akimbo *Shotguns*)

Illustration Akimbo

Ce sont des armes à courte portée et faible cadence de tir, disposant de peu de balles dans chaque chargeur. Les projectiles possèdent une dispersion et l'arme tir au coup par coup en zone circulaire autour du *Crosshair*. L'Akimbo consiste à avoir une arme dans chaque main du personnage, et à pouvoir tirer avec chacune des armes respectivement avec le *LeftClick* et le *RightClick*.

Les armes secondaires sont bien plus puissantes, mais représentent un *Gameplay* exotique.

Il s'agit de l'arme suivante :

- Un Lance-roquettes (RL)

Illustration Lance-Roquettes

C'est une arme à longue portée et faible cadence de tir, disposant d'une seule balle dans chaque chargeur. Elle tire via un une seule balle dans chaque chargeur. Elle tire via un *Raycasting* précis et tire un projectile ayant du *Travel Time* et explosant en zone sphérique infligeant d'énormes dégâts au contact d'un obstacle.

Les armes secondaires sont limitées en termes de munitions.

Le joueur dispose donc d'armes principales au *Gameplay* « classique », mais légèrement différent pour chacune d'entre elles, afin de proposer des challenges altérés (l'AR se concentre sur la précision et le *Tracking*, tandis que les Akimbo Shotguns se concentrent sur les déplacements et le *Flicking*. Les deux restent tout de même compris dans les challenges de visée et déplacement).

Il dispose également d'armes secondaires, à l'utilisation plus rares, afin de proposer des phases de *Gameplay* exotiques et de donner au joueur un bonus temporaire, qu'il peut utiliser pour surperformer ou pour se sortir de situations délicates.

DEPLACEMENT VS VISEE

Une des problématiques de *Game Design* les plus importantes dans la création de Fast-Shooter est celle de l'opposition entre le déplacement et la visée.

En effet, le *Core-Gameplay* de ce genre de jeux est constitués de deux challenges complètement antithétiques : plus le personnage va vite, plus il est difficile pour le joueur d'être précis et réciproquement, plus le joueur souhaite être précis, moins le personnage peut aller vite.

Les mécaniques du jouet sont donc majoritairement orientées vers l'un d'entre eux : le challenge de déplacement et de prise de vitesse. Le comportement des armes est donc très épuré afin de laisser place aux mécaniques de déplacement.

De plus, c'est dans cette optique que les monstres qui emplissent l'espace de jeu sont destructibles en quelques balles seulement (2~4), afin de ne pas demander au joueur un challenge de Tracking trop important.

Ingrédients

Œufs

Les Œufs sont des ingrédients disposés en groupes d'une dizaine à des endroits stratégiques de la *Map*. Lorsque le personnage les détruit à l'aide de ses mécaniques de tir, alors chaque œuf détruit vient incrémenter la jauge de la ressource de *Turbo* du personnage.

Une fois détruits, les œufs se régénèrent automatiquement après une longue période, et uniquement lorsque le personnage se trouve au-delà d'une certaine distance.

Cela permet de s'assurer que le personnage ne reste pas trop longtemps à graviter autour d'un *Spawn* d'œufs.

Les Œufs sont placés à des endroits stratégiques demandant au joueur de faire de légers détours dans la trajectoire de son personnage, en dehors de son chemin naturel.

L'idée est de récompenser un joueur qui aurait les compétences afin d'atteindre une vitesse moyenne suffisamment grande pour se permettre un détour sans remettre en question l'accomplissement de ses objectifs.

Illustration Oeufs

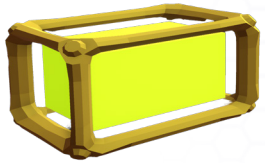
Munitions secondaires

Les munitions secondaires sont une Reward consommable donnée au joueur lorsqu'il détruit les agents hostiles qui peuplent l'espace de jeu.

Lorsque le personnage détruit un ennemi, alors celui-ci possède une chance de Drop une boîte de munitions secondaires. Si le personnage passe à proximité, alors il la récupère, et voit son nombre de munitions d'arme secondaire augmenter.

Cet ingrédient sert à créer des fenêtres d'opportunité de Gameplay exotique au joueur, en plus de récompenser le bon accomplissement du challenge de destruction des monstres.

Illustration Munitions



Signes & Feedbacks

Dans le cadre d'un jeu au *Game Feel* frénétique, nerveux et précis, il est important que l'entièreté de nos mécaniques soit indiquée et feedbackée correctement afin de communiquer au joueur la réussite ou l'échec de ses actions.

Vitesse

Gain et perte

- Le *Glow* du casque scintille en vert ou en rouge si le personnage gagne ou perd de la vitesse
- *L'Outline* du casque s'allume en vert ou en rouge si le personnage gagne ou perd de la vitesse
- Un *VFX* d'aberration chromatique s'active lorsque le personnage gagne de la vitesse
- La valeur de vitesse incrémente ou décrémente si le personnage gagne ou perd de la vitesse
- Le *FOV* augmente ou diminue si le personnage gagne ou perd de la vitesse
- Des *SFX* s'activent lorsque le personnage gagne ou perd de la vitesse

Niveau de vitesse

- Des *Speedlines* s'affichent à l'écran et varient en quantité, taille et couleur en proportion de la vitesse du personnage
- La valeur de vitesse s'affiche à l'écran en proportion de la vitesse du personnage
- La musique évolue en proportion de la vitesse du personnage
- Le *FOV* augmente en proportion de la vitesse du personnage
- Les animations de déplacement varient en proportion de la vitesse du personnage

Shooter

- Le *Crosshair* réagit à chaque tir
- Des *Hitmarkers* s'affichent lorsque le tir atteint sa cible. Ils varient si le tir est un critique, ainsi que si l'agent hostile est détruit ou non
- La jauge de munitions décrémente ou incrémente lorsque le personnage tire ou recharge
- L'*UI* liée à l'arme en cours d'utilisation est actualisée en continu
- Des *SFX* s'activent à chaque action liée à l'utilisation de l'arme (tir, recharge, inspection)
- Des animations se jouent à chaque action liée à l'utilisation de l'arme (tir, recharge, inspection)
- Les boîtes de munitions secondaires sont physicalisées et possèdent un *Glow*
- Une animation et un *SFX* se jouent lorsque le personnage récupère une boîte de munitions secondaires

Objectifs

- Des *Voicelines* sont jouées à chaque apparition et disparition d'objectifs
- *L'UI* liée aux objectifs est actualisée en continu
- Des *Waypoints* physicalisés sur les objectifs réagissent en fonction de la position du Crosshair du joueur
- Des *SFX* s'activent à chaque apparition et disparition d'objectifs
- Des animations se jouent au niveau des objectifs à chaque apparition et disparition

Player

- Une animation et un *SFX* se jouent lors de la prise de dégâts
- Une animation et un *SFX* se jouent lors de la récupération de points de vie
- Une animation et un *SFX* se jouent lors du déplacement du personnage, et varient en fonction de la vitesse et de l'état de ce déplacement (glissade, saut, chute)
- Des *Voicelines* se jouent en fonction de l'état de santé, et de vitesse du personnage
- Le *HUD* du personnage s'actualise en continu (*Display* de ses *HP*, de son *Turbo*, de son état critique, des Metrics de déplacement)
- *L'Outline* et le Glow réagissent à la récupération et la consommation de Turbo, en plus d'un *SFX* qui se joue

Roguelike

- Le *Controller* est arrêté et les cartes *d'Upgrades* apparaissent accompagnées d'un *SFX* lorsque la phase d'Upgrade se lance
- Une animation et un *SFX* se jouent lorsque le joueur survole une carte d'Upgrade
- Une animation et un *SFX* se jouent lorsque le joueur sélectionne une carte d'Upgrade, accompagnés d'une *Voiceline*
- Une *Voiceline* se joue lorsque la phase d'Upgrade se termine, la musique reprend

Tension ludique

Propriétés ludiques du jouet

Le jouet possède des propriétés ludiques axées autour de la vitesse.

Le *Controlle*r est capable de monter jusqu'à une haute vitesse de déplacement et les *feedbacks* des différentes actions associées ainsi que de la vitesse du personnage sont présents partout dans le système.

Tendances joueur

L'ensemble de ces mécaniques va inciter le joueur à se déplacer et à prendre de la vitesse, jusqu'à atteindre une vitesse suffisamment haute pour qu'il devienne difficile de la faire monter plus haut grâce à l'*Adder.*

Ensuite, le joueur va adopter des comportements lui permettant de gagner de la vitesse temporairement grâce au Bonus, mais également effectuer des actions lui permettant de conserver ladite vitesse.

Tendance système

Le système, lui, va essayer de ralentir le joueur le plus possible. Lors de ses phases d'inactivité, lors de l'échec de ses *Inputs* de déplacement, ou lorsque le personnage passe trop longtemps au sol. De plus, les chutes ainsi que les challenges de visées et de *Tracking* vont permettre de réduire la vitesse du joueur.

Tension Ludique

Notre tension ludique s'effectue donc au niveau de la prise et accumulation de vitesse. Là où le joueur va chercher à avoir un personnage de plus en plus rapide, le système va chercher à l'en empêcher.

Cohérence & émergence

Cohésion du noyau jouet

Les différentes mécaniques du jouet orbitent autour du même challenge : réussir à atteindre et conserver une assez haute vitesse, tout en effectuant de légers challenge de précision et de *Tracking* sur les monstres qui occupent l'espace de jeu.

Dans ce but, les mécaniques ont été conçues afin de pousser le plus possible le joueur à aller dans le sens de ces challenges

- Les mécaniques de *Slide-Hopping* et *Slide-Landing* se complètent, afin de récompenser un joueur qui déplacerait le personnage en alternant glissades et sauts.
- La mécanique de *Slide-Turning* permet au joueur de se créer des fenêtres d'opportunité afin d'accomplir les challenges de *Tracking* lorsqu'il glisse (puisque, pendant un court instant, la direction de son personnage est verrouillée) et vient ensuite récompenser la création de ces fenêtres par un gain de vitesse du personnage.

De plus, plus le challenge de *Tracking* a été réussi, plus la *Reward* de gain de vitesse est importante.

- La mécanique de *Perfect Landing* permet également au joueur de pouvoir tirer lorsqu'il est en l'air (ce qui fonctionne grâce à l'absence de *Bloom* ou de recul). Il peut aller chercher ces phases aériennes grâce à la verticalité du *Level Design*, sans avoir à se soucier d'être pénalisé par une haute chute : au contraire, le système récompense sa prise de risque si ses compétences sont au niveau.

Ces mécaniques incitent dont le joueur à essayer les différentes combinaisons de mécaniques de déplacement. Lorsqu'il y arrive, il est récompensé par la possibilité de créer une fenêtre d'opportunité afin d'accomplir les challenges liés au tir et donc détruire des ennemis.

Le système va ensuite récompenser le joueur en fonction de son utilisation de ces fenêtres d'opportunités, ainsi que de ses compétences.

Mécaniques émergentes

Les différentes mécaniques du jouet ont donné lieu à des comportements émergents, que nous avons pu observer chez le joueur lors de nos différents *Playtests*.

SLIDE-TURNING 180

Le *Slide-Turning 180* est le nom que nous avons donné à la technique consistant en une série de Slide-Turns effectués au maximum (180° de rotation), afin de gagner extrêmement rapidement de la vitesse.

Cela demande au joueur une connaissance quasi-parfaite du *Level Design* ainsi que des compétences techniques relativement hautes, puisqu'il va effectuer des *Slide-Hops*, *Slide-Landings* et *Slide-Turns* en boucle, et donc passer la moitié du temps avec son personnage tournant le dos à la direction dans laquelle il se déplace (puisqu'il fait des demi-tours successifs).

SLIDE TRACKING

Mécanique déjà évoquée plus haut, les joueurs ont développé un comportement consistant à tirer sur les ennemis uniquement lors de leurs glissades, afin de pouvoir alterner leur focus attentionnel entre les challenges de déplacement et les challenges de visée.

Comportement joueur & intentions

Tendances observables

Lorsque nous avons testé et fait tester notre jeu à ses débuts, l'un des points clefs qui revenaient était le fait que les joueurs appréciaient pouvoir changer d'arme, de *Gameplay* -même légèrement- et accéder à différents outils afin de détruire les agents hostiles.

Or il nous était, en termes de *Scope* et de *Design,* impossible de concevoir une multitude d'armes aussi variées et différentes dans leur comportement, tout en gardant un jeu équilibré et intéressant.

Nous avons alors eu l'idée suivante : notre joueur pourrait commencer avec une arme correspondant à un archétype classique d'arme de genre *FPS*, puis pourrait petit à petit, le long de sa *Run*, améliorer / altérer le comportement de son arme, afin d'en faire une arme unique.

C'est ainsi que nous avons décidé de Gamifier notre jouet en un *Roguelike* dans lequel le joueur pourra customiser son arme, avec de simples changements comme des améliorations de statistiques ou bien des changements complexes comme une modification du comportement des projectiles, par exemple.

Structure globale du jeu

Nous avons donc décidé d'adopter une structure de type *Roguelike* pour le moment et *Roguelite* dans le futur. Le jeu se séquencera en une alternance de phase de « *Wave* » et phase d' « *Upgrade* ».

- Les phases de *Wave* se constituent d'un objectif à atteindre pour le joueur, pris au hasard parmi 2 types d'objectifs distincts : les Vers ou les Foreuses. Dès que l'objectif est rempli, la phase de *Wave* se termine et le joueur passe en phase d'Upgrade.
- Les phases *d'Upgrade* se constituent d'un choix d'amélioration à faire par le joueur pour son personnage, parmi un certain nombre d'améliorations proposées (de 3 à 5).

Lorsque le joueur a sélectionné *l'Upgrade* qu'il désirait, la phase *d'Upgrade* se termine et le joueur passe en phase de *Wave*.

Roguelike-Core

Design des objectifs

Challenges et competences ciblées

Dans le cadre d'un jeu du genre *Fast-Shooter*, il est important que les différents objectifs à moyen terme challengent le joueur au niveau de ses *Inputs* de déplacement ainsi que de sa visée.

Les objectifs mis en place dans le jeu sont donc naturellement des objectifs incitant le joueur à se déplacer, et mettent en jeu des agents hostiles qui peuplent l'espace de jeu de différentes manières afin de pousser le joueur à augmenter ses compétences de tir.

Objectifs gameplay adaptés

Il existe 2 types d'objectifs différents, ainsi qu'un troisième qui a été conçu mais non intégré. Ces objectifs ont pour but de proposer de la variation dans les *Waves*, en challengeant plus ou moins différemment les compétences du joueur.

Les objectifs sont les suivants :

- Les Vers

Les Vers sont des agents qui vont apparaître à plusieurs points précis de la carte, tirés au hasard parmi une *Pool* de points. Une fois apparus, ils disposent d'une jauge de vie, et font apparaître à rythme régulier des ennemis.

Dès que le joueur descend leur jauge de vie à 0, les Vers arrêtent de faire apparaître des agents hostiles.

En revanche, tant que la phase de *Wave* n'est pas terminée (puisque composée de plusieurs Vers), un Ver avec ses points de vie à 0 se régénèrera lentement. S'il atteint 100% de ses points de vie à nouveau, il redeviendra actif et recommencera à faire apparaître des ennemis.

Cela pousse le joueur à ne pas *Focus* les Vers les uns après les autres, mais plutôt à naviguer entre les différents Vers pour baisser leur jauge de points de vie homogènement, afin de pouvoir tous les achever dans la même « rotation » et qu'aucun n'ait le temps de se régénérer.

De plus, au cours de sa navigation, le joueur devra tirer sur des ennemis qui occupent l'espace de jeu.

Illustrations du Ver

- Les Foreuses

Les Foreuses sont des objectifs qui vont apparaître seul ou à plusieurs sur un/des point.s précis de la carte, tiré.s au hasard parmi une *Pool* de points. Une fois apparus, un Timer se lance.

Les Foreuses disposent d'une jauge de vie, et le but du joueur est d'empêcher que cette jauge ne tombe à 0 jusqu'à la fin de l'écoulement du *Timer*.

Les ennemis vont interagir différemment en fonction de la présence des Foreuses, de leur distance, ainsi que de la position du joueur. Si une Foreuse tombe à 0 points de vie, alors la *Wave* s'arrête et le joueur a échoué.

Cela pousse le joueur à naviguer entre les différentes Foreuses, afin de s'assurer de toutes les protéger simultanément.

De plus, au cours de sa navigation, le joueur devra tirer sur des ennemis qui occupent l'espace de jeu.

Illustrations de la Foreuse

VARIATION ET REJOUABILITE

Les 2 (à terme 3) types d'objectifs permettent une forme de rejouabilité de base.

A cela s'ajoute une génération aléatoire des types de *Waves* ainsi que de l'emplacement des Vers ou des Foreuses.

Ces éléments permettent d'éviter une redondance dans le *Gameplay* et ainsi créer de la rejouabilité. Celle-ci est accrue par les Upgrades dont peut disposer le joueur et donc également la tournure que prendra sa *Run*.

Ainsi, nous sommes certains de créer de la variation et de briser la monotonie, tout en proposant des objectifs qui challengent le même set de compétences chez le joueur afin de ne pas le faire changer complètement de jeu.

Upgrades Design

Classification des Upgrades

Les *Upgrades* ont été classifiées par degrés de rareté, comme on peut le voir dans beaucoup de *Roguelike / Roguelite*. Il existe donc 4 tiers de rareté : Commun, Rare, Epique et Légendaire, par ordre de chance d'apparition.

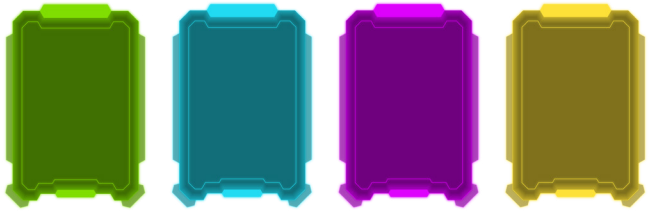
Chaque tier est plus rare que le précédent et propose également des améliorations plus puissantes. De plus, plus l'on grimpe en rareté, plus les *Upgrades* proposées sont exotiques.

Par exemple, le tier Commun ne propose que des améliorations de statistiques.

Le tier Rare permet de changer d'arme principale.

Le tier Epique propose des améliorations qui vont changer drastiquement le comportement de l'arme.

Illustrations de la rareté des cartes



Tandis que le tier Légendaire apporte des améliorations qui permettent de changer le *Gameplay* du joueur.

Formalisation et fonctionnement statistique

Le taux d'apparition des cartes a été rationnalisé en fonction du nombre d'améliorations proposées au joueur à chaque phase d'*Upgrade* ainsi que de la fréquence à laquelle les différentes rareté devaient apparaître.

Il était important de répondre à la question : « Les cartes [Rareté] doivent apparaître en moyenne combien de fois par *Wave* ? ».

A partir de là, un tableau de statistiques d'apparition des différentes raretés a été établi.

Generation procedurale

Ces probabilités préalablement établies sont programmées dans le système à l'aide d'un système de poids. Ce poids est un paramètre permettant de pondérer le taux d'apparition des cartes en fonction des cartes disponibles et de certaines propriétés uniques (certaines cartes ne sont obtenables qu'un nombre X de fois, par exemple).

Ces propriétés uniques se retrouvent dans des cartes à itération limitée (ex. 1) ou bien qui changent drastiquement le Gameplay du joueur (ex. 2).

Mob Design

Challenges et competences ciblées

Dans l'optique du jeu de genre *Fast-Shooter*, il est important de ne pas proposer au joueur un *challenge* de précision trop difficile ou de *Tracking* trop long. Les différents agents hostiles disposés sur la Map possèdent donc des points faibles mis en valeur et sont destructibles par une arme principale non-améliorée en 2~4 balles ou bien 1 balle dans un point faible.

Structure du comportement des ennemis

Les ennemis ont été conçus à l'aide d'un outil permettant de pouvoir les créer rapidement mais permettant également d'exprimer avec un algorithme relativement simple le schéma de prise de décision et du fonctionnement de leur IA.

Chaque monstre possède une suite d'actions possédant des conditions de départ et conditions d'arrêt, ainsi que trois paramètres : une priorité, une chance et un poids.

Le système va lire pour chaque entité leurs actions dans leur ordre de priorité.

Pour chaque action, si la condition de départ est remplie, le système va la « tester ». Si deux actions sont valides et de même priorité, le système choisira alors au hasard, pondéré par le poids des actions.

« Tester » une action, c'est essayer d'effectuer l'action en fonction de son paramètre « chance ». Si le paramètre « chance » est à 1, alors l'action est effectuée à 100%. S'il est à 0.5, le système va tirer au hasard et aura 1 chance sur 2 de l'effectuer.

Lorsqu'une condition d'arrêt est validée ou que le test d'une action a échoué, l'action s'arrête et le système va ré-effectuer la manœuvre depuis le début pour décider de la future action de l'entité.

Si aucune action n'est valide, alors les entités effectueront leur action par défaut, dénuée de condition de départ.

ENNEMIS FINAUX

- **Jumper**

Le *Jumper* est un ennemi qui se déplace en direction du personnage ou de l'objectif le plus proche. Arrivé à une certaine distance, il va se mettre à attaquer celui-ci en lui sautant dessus.

Si le personnage est tourné en direction du Jumper, celui-ci va se mettre à effectuer des esquives latérale.

En dessous d'une certaine distance, le *Jumper* possède une chance d'effectuer des phases d'enragement, durant lesquelles il ne bouge pas et joue une animation. Il s'agit de fenêtres d'opportunité pour que le joueur les détruise.

L'intérêt du *Jumper* est de forcer le joueur à se déplacer en plus de proposer un fort challenge de visée.

Illustrations du Jumper

- **Exploder**

L'Exploder est un ennemi qui se dirige constamment et uniquement en direction du personnage. S'il se trouve suffisamment proche du personnage, il explose et inflige des dégâts en zone au personnage et aux Foreuses.

Il possède une inertie grandissante, proportionnelle au temps passé à se déplacer dans une direction fixe. *L'Exploder* est donc capable d'accélérer jusqu'à un maximum de 40 de vitesse, ce qui correspond à une vitesse moyenne élevée du personnage. Cependant, cette inertie rend les virages difficiles à négocier et le joueur peut esquiver latéralement la trajectoire des *Exploders*.

Afin de compenser sa forte puissance, *l'Exploder* possède une large *Hitbox* et peu de points de vie. Lorsque *l'Exploder* est détruit, il explose en infligeant des dégâts en zone au personnage et aux Foreuses.

L'intérêt de *l'Exploder* est de forcer une vitesse moyenne élevée du personnage et proposer des challenges d'esquive et de *Tracking* au joueur.

Illustrations de l'Exploder

- **Colosse**

Le Colosse est un ennemi qui se déplace en direction de la Foreuse la plus proche, ou le personnage dans le cas échéant. Arrivé au contact, il va se mettre à attaquer lentement, en infligeant de très gros dégâts.

Il possède des mécaniques de protection :

- Il possède des parties de *Hitbox* qui sont invulnérables
- Il possède des comportements liés à sa survie (il possède une chance de s'enterrer, ou de protéger sa face avant)

Le Colosse est plus grand que les autres agents hostiles et possède sa zone de vulnérabilité en dessous de son abdomen.

L'intérêt du Colosse est de forcer le joueur à le prioriser lors des Waves de Foreuses, ainsi que de proposer au joueur des challenges de déplacement et de tir légèrement différents en l'incitant à faire tourner le personnage autour du Colosse ou à passer en dessous de celui-ci.

Illustrations du Colosse

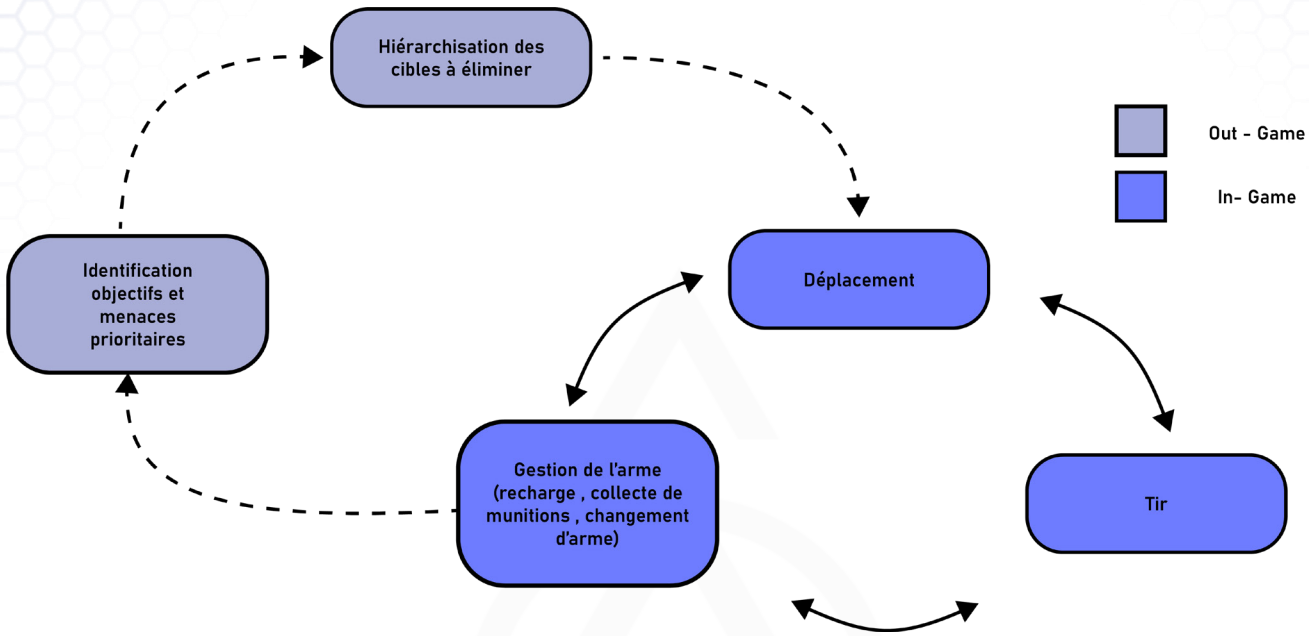
EFFETS DES ENNEMIS SUR LE PERSONNAGE

Ces différents agents hostiles ont donc pour but, du fait de leur comportement, de pousser le joueur à les considérer comme une menace suffisamment importante pour devoir les détruire, tout en gardant le personnage en mouvement.

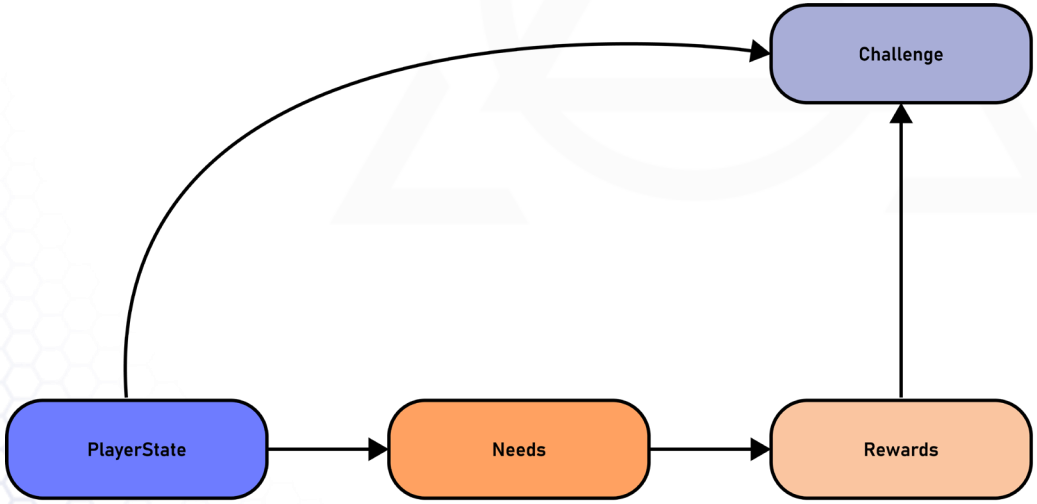
En effet, un joueur aux compétences suffisamment élevées pour conserver son personnage à une vitesse moyenne élevée pourra esquiver la quasi-totalité des attaques.

Cela lui demandera cependant d'être capable de répondre aux deux demandes de compétences simultanément (déplacement et tir) afin d'expérimenter le *Fast-Shooter* au complet.

Boucle de Gameplay



Boucle PNRC



PlayerState : vitesse du personnage , état de santé du personnage

Needs : Vitesse suplémentaire

Rewards : Meilleure capacité de destruction
des ennemis et de complétion des objectifs

Challenge : réaliser des séquences d'inputs de prise de vitesse , détruire des ennemis

L.D
Level Design

Level Design

Metrics

Etablissement des Metrics du joueur

Intentions

La promesse d'un *Fast-Shooter* s'accompagne obligatoirement d'un *Game Feel* centré autour de la vitesse. Il était donc important que les *Metrics* du joueur permettent d'accéder non seulement à une haute vitesse, mais également à une bonne marge de progression de celle-ci, afin que le joueur ait la sensation d'accélérer.

Il n'existe donc pas de Cap maximum de vitesse, cependant la progression est rendue de plus en plus difficile à mesure que le personnage gagne en vitesse.

De plus, nous voulions que le High-Level Context ajoute à cette sensation de vitesse : nous avons donc ajusté les Metrics du Controller à la taille de la *Map*. Le tout étant calibré autour d'une vitesse moyenne agréable.

Concernant les autres mécaniques, nous voulions faire ressentir au joueur le Feeling de contrôler un drone humanoïde. Les différentes Metrics du personnage devaient donc être similaires à celles d'un humain, légèrement améliorées sans devenir surréalistes.

Itérations

Itération notable 1

L'une de nos itérations a vu la vitesse maximale (il n'y a pas de maximum à proprement parler, mais on appelle vitesse maximale la vitesse suffisamment haute pour qu'il soit très difficile de la conserver et d'aller au-delà) de notre personnage être divisée par deux.

En effet, le personnage se déplaçait bien trop vite et le challenge de visée était devenu trop difficile. Nous avions contrebalancé cela à l'aide de cibles imposantes, mais cela donnait la sensation d'être très petit vis-à-vis de l'environnement.

Nous avons donc fait descendre la vitesse « maximale » de 120 à 60. Afin de compenser cette perte de vitesse, nous avons opté pour un Level Design à plus grande échelle, en multipliant les éléments permettant de créer un High-Level Context communiquant de la vitesse.

Itération notable 2

L'une de nos itérations les plus importantes s'est faite après l'implémentation de l'entièreté des mécaniques de perte de vitesse.

C'est à partir de cette itération que conserver la vitesse du personnage devenait un véritable challenge pour le joueur. Avant celle-ci, il était relativement facile de gagner de la vitesse, mais les erreurs n'étaient que très peu sanctionnées : il était difficile de décélérer.

Nous avons donc entièrement revu les valeurs des différents gains de vitesse afin de conserver des Metrics équilibrées.

Echelle et Metrics de l'environnement

Degrés de liberté du joueur

Afin de répondre à la *Fantasy* de *Fast-Shooter*, l'espace de jeu dans lequel évolue le personnage devait répondre à plusieurs critères.

- Tous les passages qui semblaient franchissables par le joueur devait être franchissables par le personnage
- Le *Level Design* devait proposer de la verticalité
- Le *Controller* doit sembler peu limité et très adapté aux Metrics de son environnement
- Le *Level Design* doit proposer de l'espace, et limiter les passages étroits

Ce sont pour ces différentes raisons que notre espace de jeu possède des modules en hauteur accessible de toutes parts, et que la quasi-totalité des hauteurs du jeu sont des différentiels de 4m, là où notre Controller peut effectuer des sauts de 5m.

Outils mis en place

Terrain

Terrain est un package disponible sur *Unity* permettant de créer des *Meshes* correspondant à des terrains organiques. C'est l'outil que nous avons utilisé au départ afin de mélanger *Mesh* organique et structures rigides.

Cependant, nous avons décidé de passer par une génération manuelle sur Blender puisque Terrain ne proposait pas d'outil suffisamment puissant pour créer des cavités, et que nous souhaitions que l'entièreté de notre *Level Design* se trouve dans un immense réseau de grottes.

Modular Set

Afin de pouvoir *Blocker* rapidement nos différentes scènes, nous avons créé un ensemble de gabarits modulaires nous permettant de représenter les structures rigides de notre scène.

Ce set modulaire nous a servi jusqu'à l'aboutissement du projet, lorsque nous avons appliqué la Direction Artistique Visuel sur chacun de ces Assets, avant de tous les remplacer dans nos scènes finales.

Itérations

Itération notable 1

L'une de nos itérations a vu quasiment tripler de taille notre Blocking, car il ne proposait pas suffisamment de grands espaces au joueur. Nous avons donc élargi les différentes circulatoires, afin de laisser respirer le joueur, et qu'il puisse profiter de grands espaces plats où être libre, y compris à haute vitesse.

Cela a également diminué le nombre d'obstacles présents dans le niveau, permettant au joueur de garder la vision sur les évènements et la foule d'agents hostiles.

Itération notable 2

Notre seconde itération notable est survenue lors de l'implémentation des mécaniques de *Perfect Landing* et *High-Fall Landing*. En effet, ces deux mécaniques jouant sur la verticalité, nous nous sommes rendu compte que notre Blocking en manquait cruellement.

Nous avons donc implémenté de la verticalité, et avons adapté les *Metrics* de gain de vitesse de ces deux mécaniques en conséquence.

Espaces & vitesse

Notre *Level Design* est donc composé de grands espaces, qui peuvent sembler impressionnants au début. Cependant, leur existence va inciter le joueur à se déplacer et à prendre de la vitesse. A terme, ces grands espaces vont permettre au joueur de ressentir ce *Feeling* de vitesse : des grands espaces parcourus rapidement semblent petits.

Cependant, ces espaces ne sont pas vides, et les différents *Landmarks* permettent de créer un *High-Level Context* propice à l'expression de la vitesse.

Circulatoires VS lignes de vue

L'une des problématiques majeures lors de la création du *Level Design* de notre jeu de genre *Fast-Shooter,* a été l'opposition entre le besoin de circulatoires agréables et espacées, et l'existence de lignes de vue complexes et intéressantes.

Cependant, après avoir fait de nombreux tests de *Blocking*, les modules trop proches les uns des autres donnaient davantage un Feeling d'exploration et de *Close-Quarter Battle* (CQB) que de vitesse et de frénésie.

Ainsi, notre *Blocking* repose majoritairement sur des grands espaces très aérés, avec de nombreuses lignes de vue afin que le joueur puisse récolter rapidement et efficacement de l'information. Les challenges restent donc focalisés autour des déplacements et de la visée et non autour du combat et du positionnement tactique.

GESTION DE LA DIFFICULTE

Equilibrage des Upgrades

Effets des Upgrades sur les statistiques

Les *Upgrades* agissent comme des modificateurs de statistiques appliqués aux statistiques de base de l'arme. Ces modificateurs sont calculés sous la forme de pourcentages, que l'on additionne avant d'en faire un coefficient multiplicateur de la valeur de base de la statistique.

FinalValue = BaseValue + BaseValue * (Somme des modificateurs / 100)

Cela permet de rendre linéaire la progression des statistiques du joueur, et ainsi éviter des effets boule de neige dans le cas où un joueur aurait la chance de tomber plusieurs fois de suite sur des améliorations similaires (auquel cas les multiplicateurs entre eux auraient fait grandir la statistique de manière exponentielle).

Il existe également des *Upgrades* qui donnent des effets ou bien changent drastiquement les statistiques de l'arme du personnage, ce sont les *Upgrades* exotiques.

Enfin, certaines *Upgrades* sont de simples incrémentations, dénuées de valeur en pourcentage, et visant certaines statistiques précises.

Classification et pondération des Upgrades

Nous avons ensuite pondéré la valeur des modificateurs apportés par les *Upgrades* de manière à ce que l'amélioration apporté par une *Upgrade* d'un tier supérieur soit inversement proportionnelle à sa rareté.

Si une *Upgrade* est 2 fois plus rare qu'une autre, alors elle donnera 2 fois plus de statistiques.

Gestion des Upgrades exotiques

Les *Upgrades* exotiques évoquées plus haut ne pouvant cependant pas être pondérées de manière classique, un coefficient leur a été attribué en fonction de leur effet ainsi que de leur rareté.

En fonction de leur puissance, elles ont ensuite été classifiée dans les tiers Epique ou Légendaire, puisqu'il serait incohérent de proposer un changement radical de *Gameplay* trop fréquemment au joueur.

Equilibrage des Waves

Procéduralisation des Waves

Les deux premières *Waves* que rencontre le joueur sont forcément une *Wave* de Vers puis une de Foreuses. Cette base permet de s'assurer que le joueur connaît les deux types d'objectifs et ne sera pas surpris à l'apparition de l'un d'entre eux.

La suite des *Waves* est tirée au hasard, entre les différents types d'objectifs (2 pour l'instant), et au sein même de la *Wave,* la disposition des objectifs est également tirée au hasard.

Courbe de progression de la difficulté des Waves

Afin d'incrémenter la difficulté des *Waves*, nous avons recourt à l'ajout de nouveaux ennemis au fur et à mesure. Certains ennemis ne peuvent apparaître qu'à partir d'une certaine *Wave*, afin de s'assurer que le joueur possède un niveau de compétences suffisamment élevé pour l'affronter.

De plus, certaines *Waves* vont également servir de palier afin d'augmenter le nombre d'objectifs simultanés proposés au joueur (nombre de Vers, de Foreuses, etc.).

Durant les 20 premières *Waves*, c'est uniquement par ces biais que la difficulté va augmenter. Au-delà, le joueur rentre dans la phase *Endless* du jeu.

Hiérarchisation des ennemis

L'ordre dans lequel les ennemis apparaissent dans les *Waves* est déterminé vis-à-vis de leur apport en termes de mécaniques.

Les Jumpers, monstres au comportement simple, sont les premiers à apparaître car ils incitent le joueur à simplement déplacer le personnage.

Les *Exploders* apparaissent en deuxième car ils demandent au joueur de déplacer rapidement le personnage, ainsi que latéralement.

Etc.

Les ennemis apparaissent donc dans un ordre qui dépend de ce qu'ils apportent en termes d'apprentissage au joueur, afin d'épouser graduellement l'évolution des capacités de déplacement et de visée du joueur.

APPRENTISSAGE & ONBOARDING

Intentions et objectifs

Public cible et Persona

Le public cible de Caverners est majoritairement constitué de *Core-Gamers*, habitués aux *FPS* et disposant de hautes compétences de visée et de déplacement en première personne.

La cible secondaire sont les joueurs de FPS légèrement plus Casual, disposant de compétences solides bien que moins élevées.

Hiérarchisation des mécaniques à enseigner

Afin de proposer un *Onboarding* intéressant, les différentes mécaniques que le joueur doit apprendre sont hiérarchisées selon plusieurs paramètres :

- La dépendance : dans quel ordre les mécaniques doivent-elles être montrées, et lesquelles sont interdépendantes ou dépendantes de mécaniques antérieures ?
- Les connaissances a priori du joueur : vis-à-vis de notre cible, à quel point l'apprentissage d'une mécanique doit être insistant ou non ?
- L'importance de la mécanique : à quel point la mécanique est-elle primordiale au *Gameplay* ? Le joueur peut-il s'en passer et la découvrir plus tard en jouant, ou doit-il absolument en avoir connaissance ?

A partir de ces différents paramètres, les mécaniques ont été classifiées de cette manière, qui correspondra à l'ordre dans

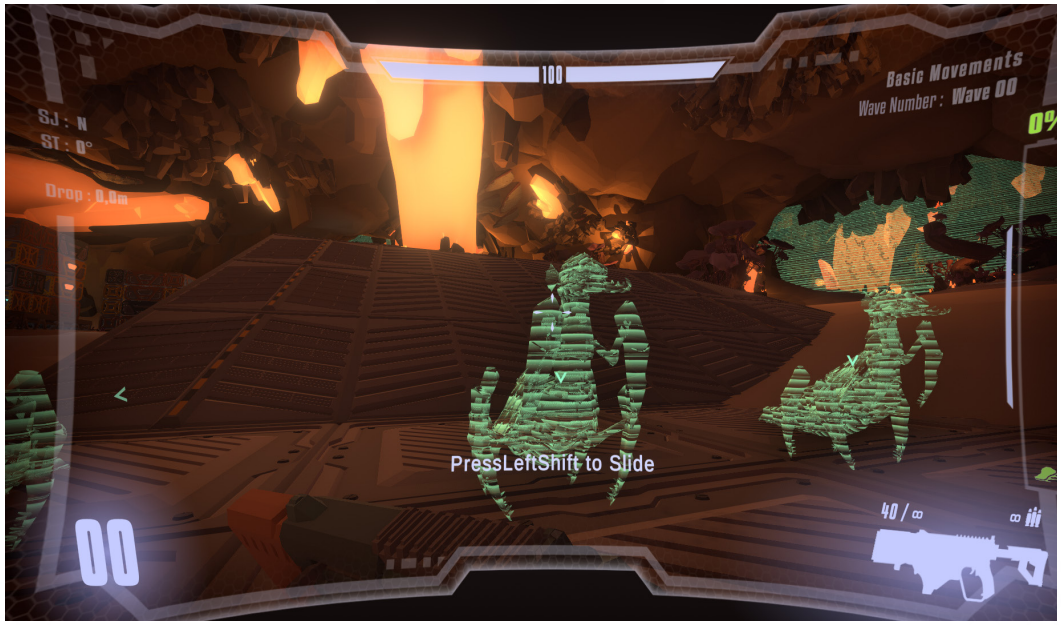
Immersion VS tutorat

L'une des problématiques que nous avons rencontré lors de la création de l'*Onboarding* a été celle de l'opposition entre immersion et tutorat.

Notre public cible étant composé de *Core-Gamers* ayant déjà une forte expérience de jeux du genre, il était fort probable qu'une grande majorité ne souhaite pas de tutorat. En effet, notre *Persona* possède des compétences déjà élevées et une appétence à la découverte par lui-même des différentes mécaniques de jeu.

C'est dans cette optique que le jeu propose une case « *Do Tutorial* », activée par défaut, dans le menu de lancement du jeu, que le joueur peut décocher lors de sa première *Run* s'il le souhaite.

OnBoarding dans le jeu avec des hologram



G.P
Game Programming
36

Game Programming

Philosophie d'implémentation

Lors du développement de notre projet, notre philosophie d'implémentation était axée autour des membres du groupe disposant de capacités de programmation moins importante.

Il était donc primordial de rendre accessible le développement aux non-programmeurs, via deux méthodes :

- La création d'outils et d'interfaces clairs et organisés, permettant la manipulation de méthodes et de comportements
- L'implémentation par ScriptableObjects, car ils sont répétables, facile d'accès et polyvalents

Processus d'implémentation

Afin d'implémenter correctement les différentes *Features* du projet, nous avons également dressé une liste de comportements à suivre afin d'être les plus efficaces possibles :

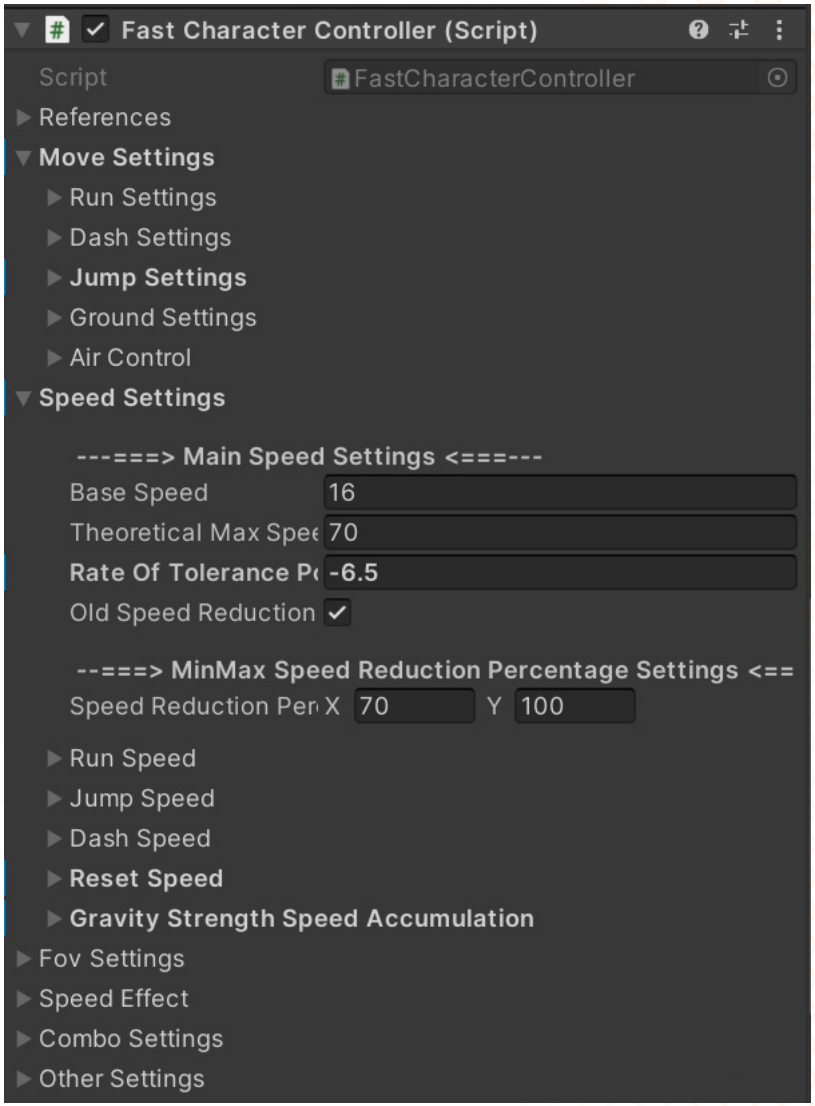
- Implémenter au plus vite : le but est de pouvoir tester puis faire fonctionner la *Feature* le plus rapidement possible
- Factoriser : réduire le code et les redondances, s'assurer qu'il y ait le moins de duplication de code possible. Ainsi, en cas de changements, il nous est facile de repérer la partie du code qui gère la fonction en question, et n'avoir qu'un minimum de changements à faire
- Débugger : tester un maximum de cas, et s'assurer que le tout fonctionne même lors de cas extrêmes ou valeurs particulières
- Optimiser : effectuer la programmation orientée performances. Nous souhaitons réduire le nombre d'instanciations, de Callbacks et d'opérations à effectuer (complexité des algorithmes)

Ce processus s'est appliqué à tous les ajouts de *Features* à partir du moment où les différents cœurs du jeu étaient implémentés.

En ce qui concerne l'implémentation de ces différents cœurs, il a fallu préparer et anticiper en amont avant d'implémenter afin d'obtenir une structure polyvalente et généralisée qui permettait en son cœur l'ajout de nouvelles mécaniques.

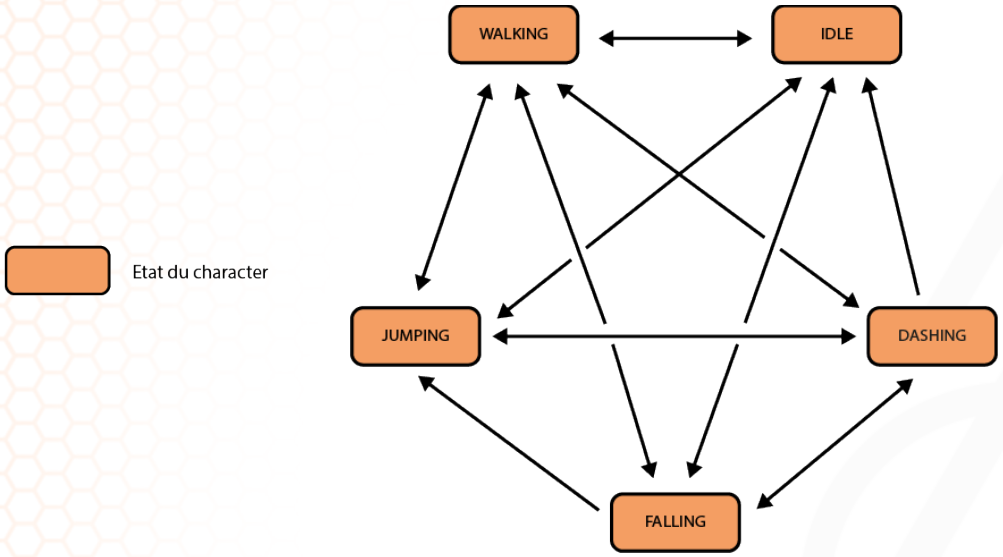
Ce sont ces différents cœurs de *Gameplay* que nous allons détailler par la suite.

Script Character Controller



Accessibilité

Schéma transition State



Afin de rendre accessible à l'équipe de *Game Design* cette partie du code, nous avons mis plusieurs méthodes en place :

- Une structure regroupant par famille les paramètres des différentes *Features* du *Controller* pour faciliter la modification du personnage
- Un affichage des éléments dans l'inspecteur structuré en parties et sous-parties afin de retrouver rapidement les paramètres recherchés
- Des options désactivables (booléennes) pour certaines fonctionnalités afin de proposer plusieurs solutions au choix lors de la réalisation de tests
- Des valeurs de Débug afin de pouvoir suivre l'évolution de certaines valeurs au cours des *Playtests*

State Machine

Notre personnage jouable évolue selon la structure d'une *State-Machine*. Celle-ci est composée d'états passifs, actifs ainsi que d'une liste de paramètres influant sur les transitions entre ces différents états.

Les états passifs sont des états qui sont vérifiés et exécutés à chaque Frame.

- *Idle* : lorsque le joueur ne rentre aucun Input et que le personnage est immobile
- *Walking* : lorsque le personnage est au sol et qu'il se déplace uniquement grâce aux Inputs de déplacement horizontal (ZQSD)
- Falling : lorsque le personnage est en train de tomber sans avoir sauté au préalable

Les états actifs sont des états constamment vérifiés mais qui ne s'exécutent qu'au moment où le joueur remplit les conditions à l'activation de ces états, et ce, une seule fois.

- *Jumping* : lorsque le joueur appuie sur l'Input de saut
- *Sliding* : lorsque le joueur appuie sur l'Input de glissade

La *State-Machine* est schématisée afin de regrouper et vérifier toutes les conditions requises à l'activation de chaque états, ainsi que les différentes transitions entre ceux-ci. Ces transitions sont pondérées par les valeurs de certains paramètres, et des Inputs du joueur. Voici une liste des différents paramètres pris en compte lors de la vérification des états / transitions :

- Les *Inputs* enclenchés, maintenus et relâchés
- Le *Buffering* des Inputs enclenchés
- Le contact au sol, le maintien au sol et la sortie de l'état au sol du personnage
- L'état aérien du personnage (en l'air ou non)
- La vitesse de déplacement du personnage
- La direction du personnage
- La présence d'obstacle au-dessus du personnage
- Les états précédent et actuel du personnage
- L'action actuellement effectuée par le personnage
- La possibilité d'effectuer une action à un temps t précis

Système de gain de vitesse et feedback sur la caméra

Pour chaque action du joueur, une valeur est ajoutée à la vitesse actuelle de son personnage. Cette valeur est déterminée par un calcul de gain de vitesse, soit en *Adder* soit en *Bonus*, déterminés comme ceci :

- Gain de vitesse en *Adder* : le gain de vitesse est une valeur ajoutée à la vitesse actuelle du personnage.

Cette valeur est multipliée par un coefficient, qui évolue de façon exponentielle et inversement proportionnelle à sa vitesse actuelle, en fonction de deux valeurs minimum et maximum de la vitesse.

Le coefficient de départ est 1, et tend vers 0 à mesure que la vitesse du joueur se rapproche de la vitesse maximum théorique.

Formule du coefficient de gain de vitesse :

$$e^{rate} * [(currentSpeed - minSpeed) / (maxSpeed - minSpeed)]$$

e = nombre d'Euler (coefficient à croissance exponentielle)

rate = multiplicateur

minSpeed = vitesse de déplacement minimum

maxSpeed = vitesse de déplacement maximum

currentSpeed = vitesse de déplacement actuelle

- Gain de vitesse en *Bonus* : le gain de vitesse en *Bonus* est une valeur brute ajoutée à la vitesse actuelle du personnage. Ce gain est temporaire et ne dure que la période p de l'action, et est perdu au moment de la fin / annulation de l'action.

Des paramètres de gain de vitesse sont établis pour chacune des mécaniques de déplacement du *Controller*, ainsi que pour certaines combinaisons d'actions comme « Sauter en se déplaçant » ou « Sauter durant la glissade ». Ainsi, nous pouvons proposer au joueur différentes manières plus ou moins efficaces de prendre de la vitesse, et effectuer des choix de *Design* intéressants.

Ancien système de réduction de vitesse

Il existe une option activable (booléenne) « *Old Speed Reduction System* » qui permet d'utiliser ce coefficient pour calculer les pertes de vitesse également.

Lorsqu'elle est activée, le système va remplacer ce coefficient par un autre multiplicateur qui diminue de manière linéaire en fonction de la vitesse actuelle du personnage.

La valeur de ce multiplicateur variait de 0.3 à 0 de manière linéaire, proportionnellement au différentiel de vitesse entre la vitesse actuelle du personnage et les vitesses minimum et maximum autorisées par le système.

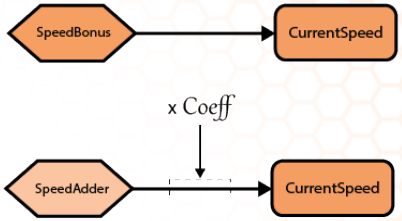
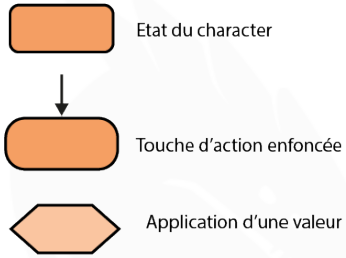
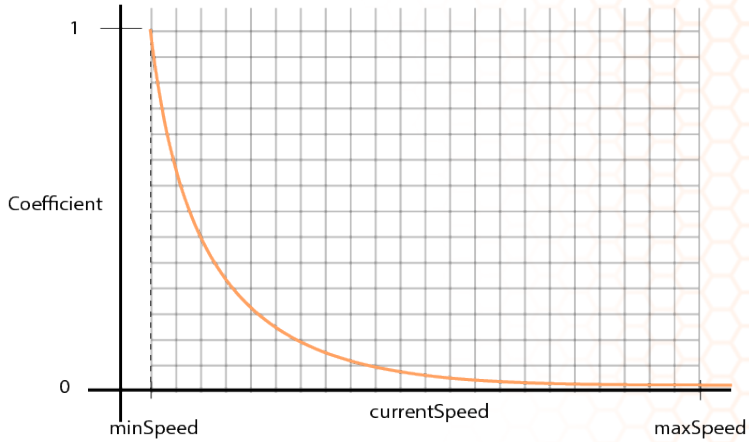


Schéma Speed gain

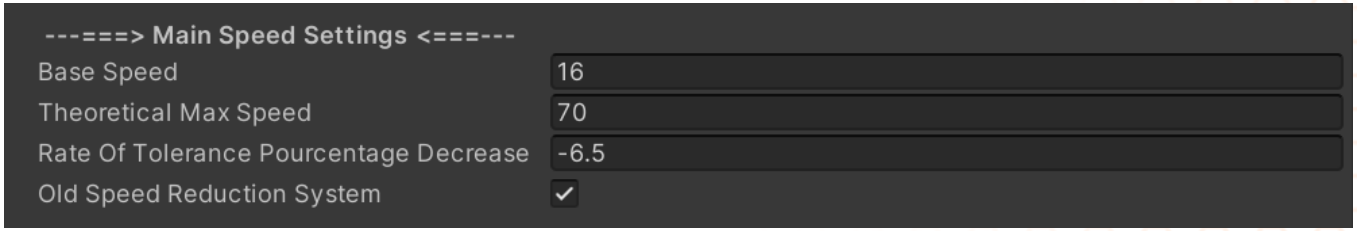




$rate = -7$

$$coefficient = e^{-rate \frac{currentSpeed-minSpeed}{maxSpeed-minSpeed}}$$

Intégration Inspecteur



Effets sur la caméra

Field of view

Le champ de vision (*FOV*) est affecté par la vitesse actuelle du personnage.

Sa modification est structurée de la même manière que les gains de vitesse, à l'aide de deux types de gains :

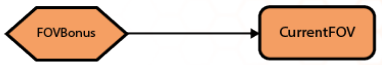
- Le *FOV* passif : est une valeur qui varie d'un minimum à un maximum proportionnellement à la variation de vitesse du personnage entre sa vitesse minimale et maximale
- Le *FOV* bonus : est une valeur ajoutée au *FOV* actuel de la caméra. Ce gain fonctionne de la même manière que le gain de vitesse en Bonus

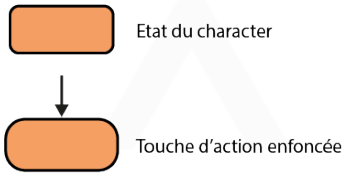
Effets visuels

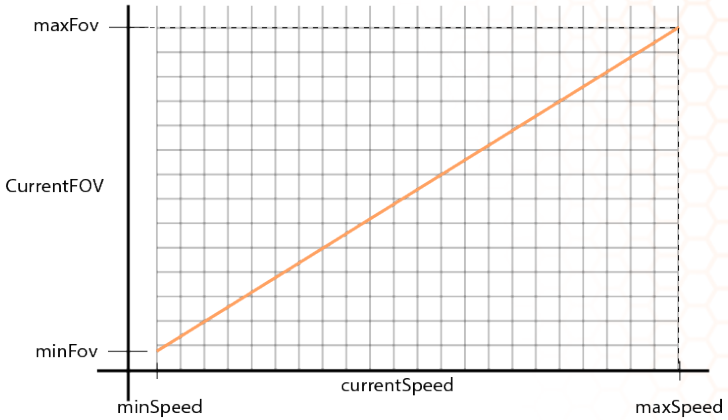
Les différents gains de vitesse vont également jouer des effets visuels sur la caméra :

- Les particules : des particules qui suivent la direction de déplacement du personnage sont affichées à l'écran. L'intensité des particules est gérée en fonction de la vitesse actuelle du personnage, allant d'un minimum à un maximum défini en fonction des vitesses minimale et maximale du personnage. En-dessous d'un certain seuil de vitesse, les particules sont désactivées.
- Les *VFX* d'aberration chromatique et de *Bloom* : deux effets de *Post-Processing* sont appliqués lorsque le joueur effectue des actions apportant un gain de vitesse en *Bonus*. Ces effets s'activent instantanément puis se désactivent progressivement, afin de donner une sensation d'impact.

Schéma FOV gain

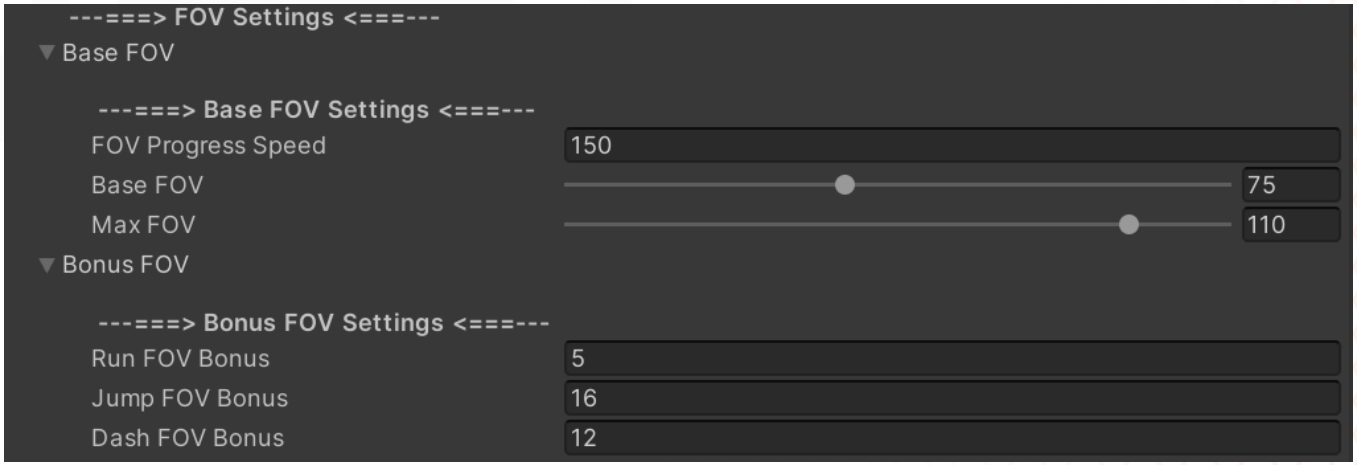






$$fov = \frac{currentSpeed - minSpeed}{maxSpeed - minSpeed} \bullet maxFov - minFov$$

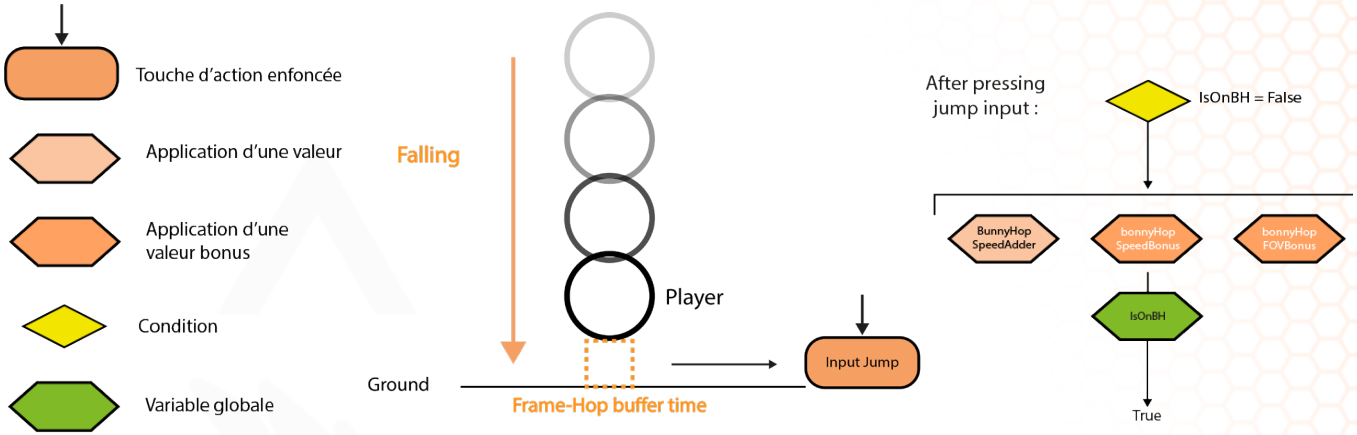
Intégration Inspecteur



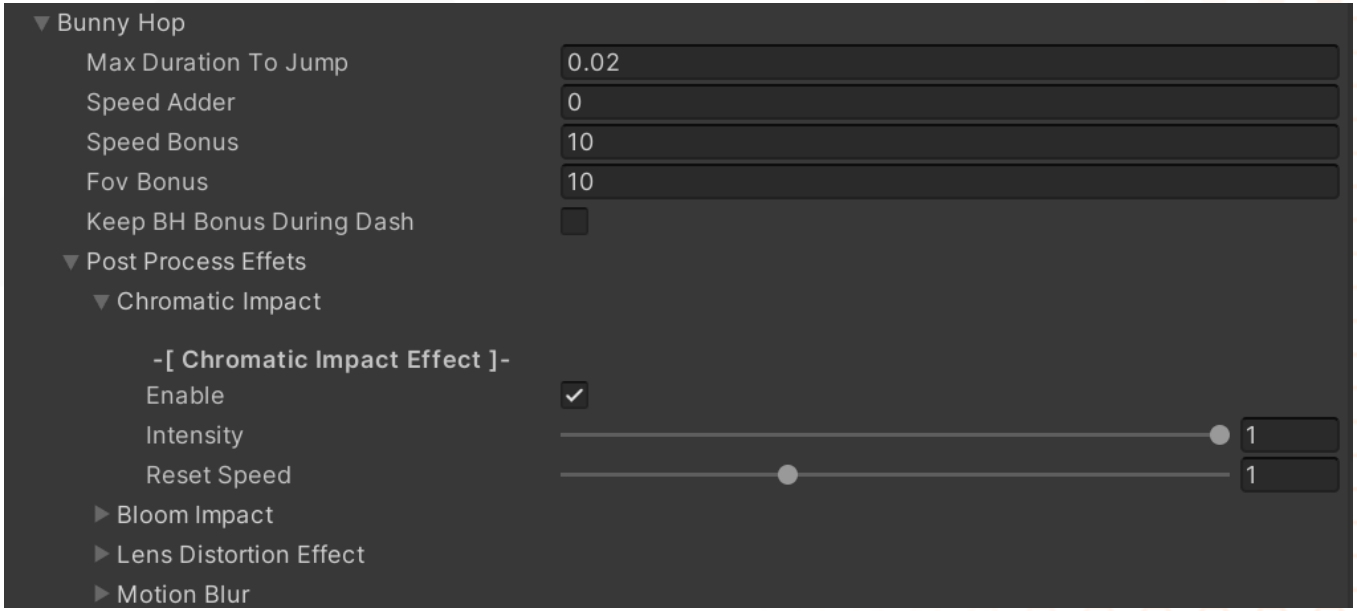
Frame-Hopping

Dans une optique de test de différentes options, un booléen « *KeepBHDuringDash* » est activable, permettant de maintenir les effets de gain de vitesse de la mécanique lorsque le personnage est également en glissade.

Schéma Frame-Hopping



Intégration Inspecteur

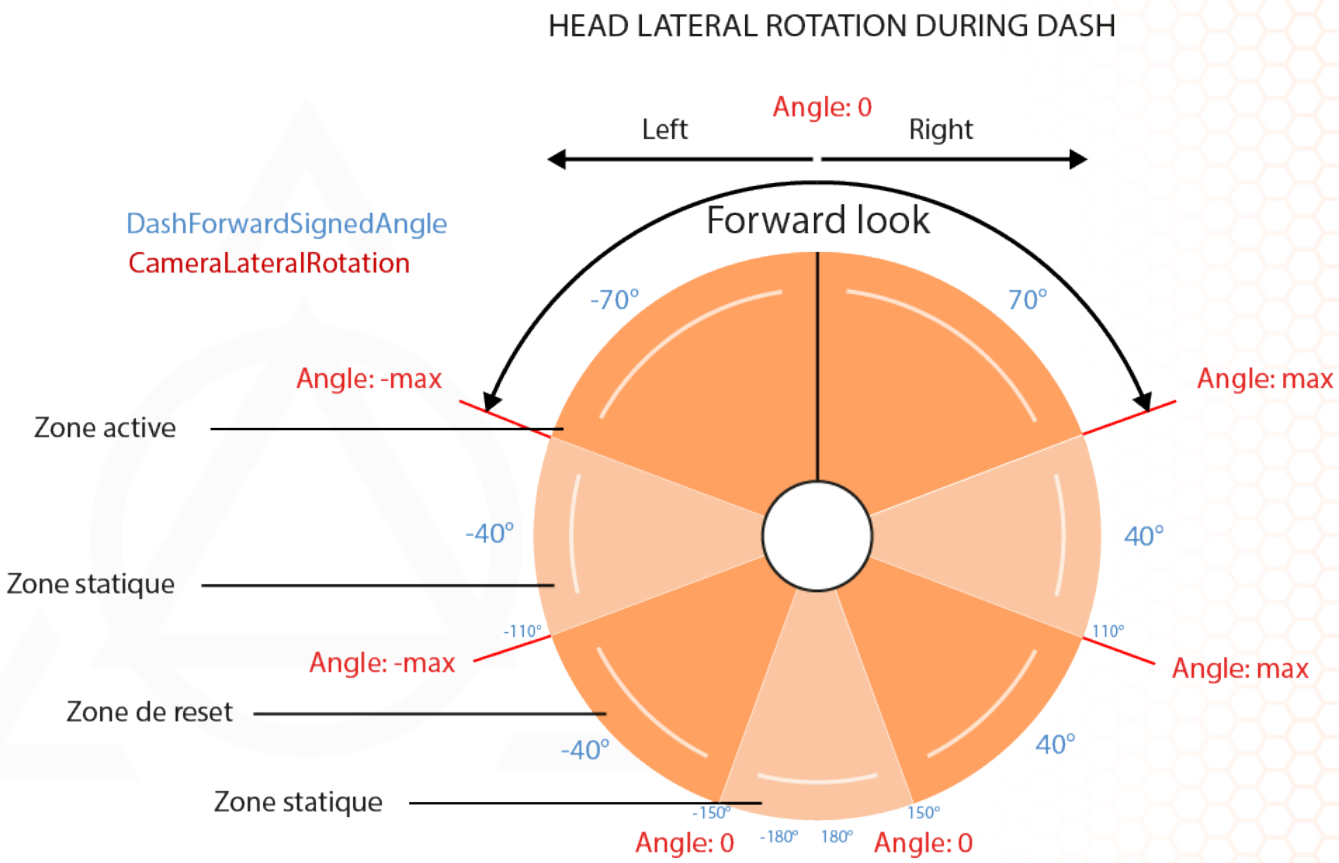


Glissade

Lors de la glissade, la caméra se penche sur les côtés (rotation sur l'axe Z) en fonction des Inputs souris horizontaux du joueur. Un calcul d'angle est effectué entre la direction de la glissade et l'orientation de la caméra. Ce différentiel d'angle va ensuite être catégorisé parmi 4 paliers en fonction de sa valeur.

- Zone active : la caméra subit une rotation sur l'axe Z proportionnelle à l'Input souris horizontal du joueur, entre un minimum et un maximum établi.
- Zone neutre : si l'Input souris horizontal du joueur dépasse le maximum de la zone active, alors la rotation de la caméra passe en zone neutre, dans laquelle sa rotation sur l'axe est Z est fixe et définie au préalable.
- Zone *Reset* : si l'Input souris horizontal du joueur dépasse le maximum de la zone neutre, alors la rotation de la caméra passe en zone Reset, dans laquelle la caméra revient progressivement vers sa valeur de rotation initiale sur l'axe Z.
- Zone nulle : si l'Input souris horizontal du joueur dépasse le maximum de la zone *Reset*, alors la rotation de la caméra passe en zone nulle, dans laquelle sa rotation sur l'axe Z est nulle. La zone nulle atteint son maximum lors d'une rotation de 180°, différentiel maximal entre deux angles.

Schéma Head Rotation



Slide-Turning

Les effets de *Post-Processing* sont incrémentés par paliers en fonction de l'intensité du *Slide-Turn*.

Voici le schéma de la mécanique en code

Schéma Slide-Turning Passif

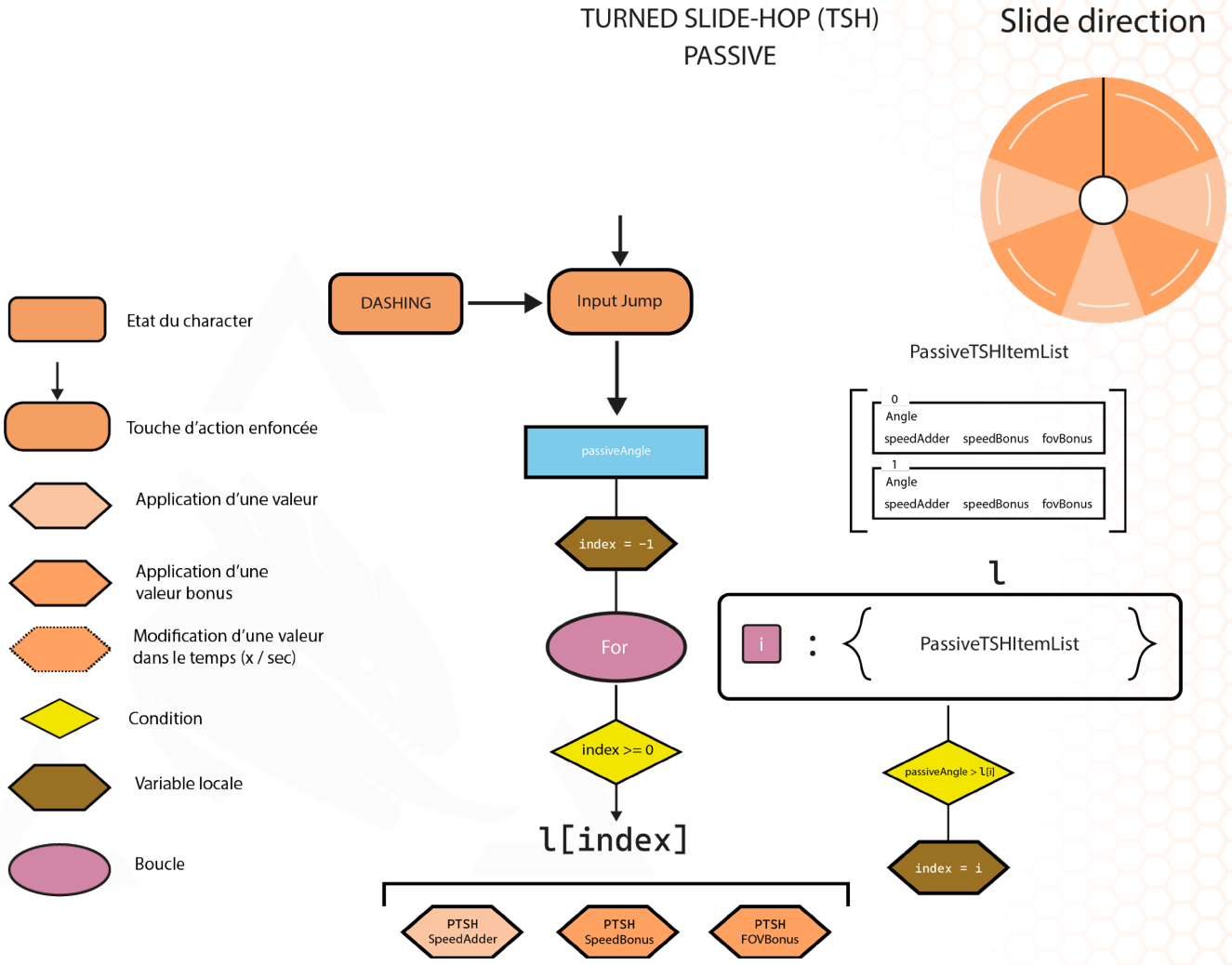
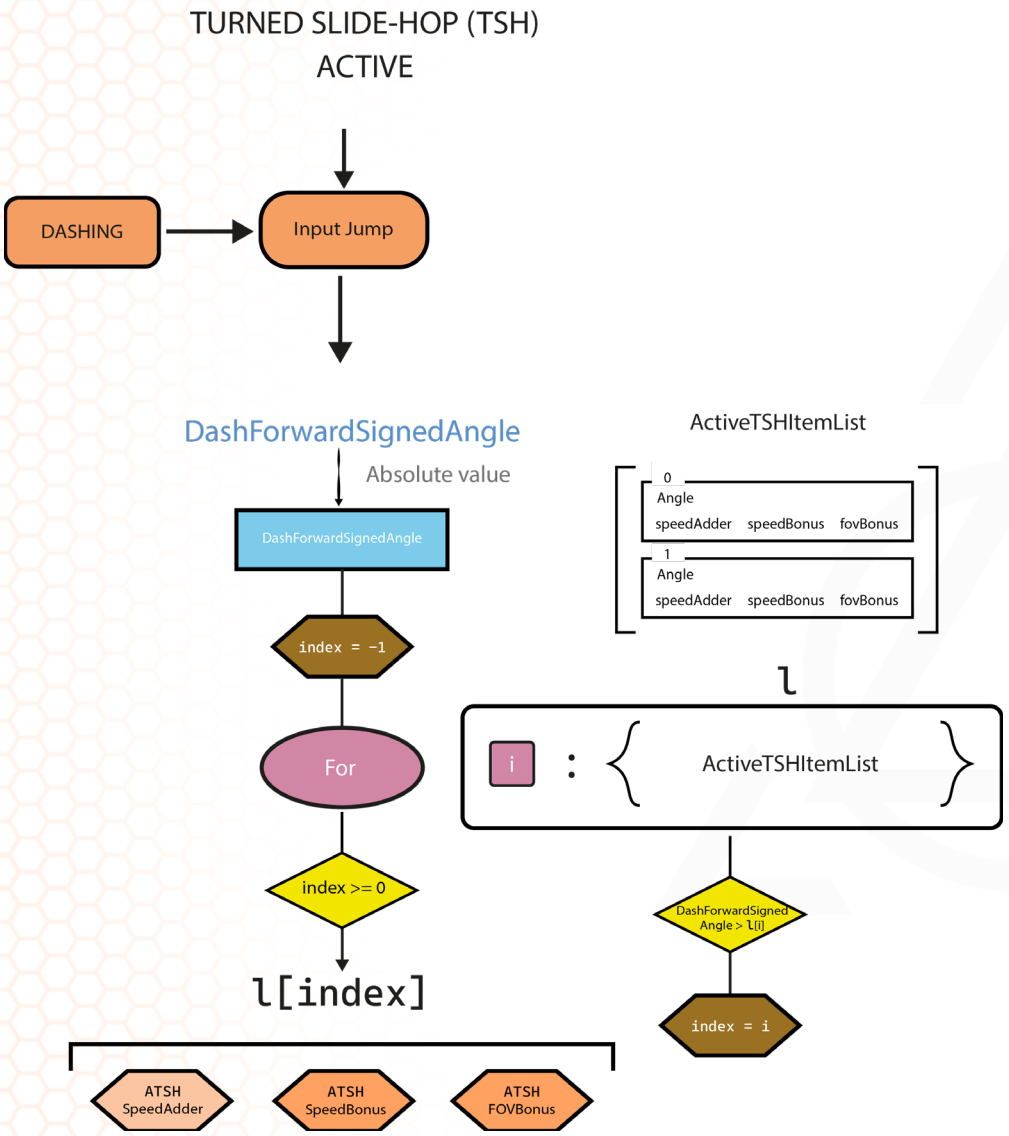
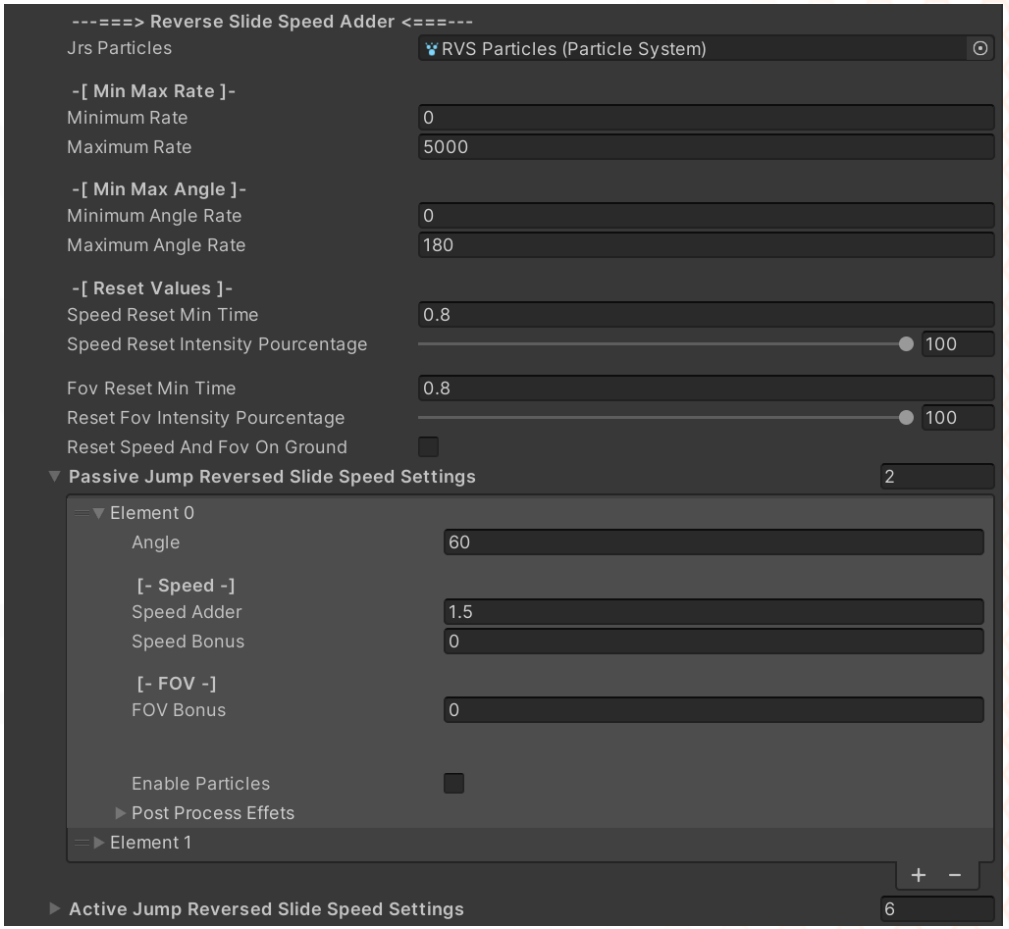


Schéma Slide-Turning Actif



Intégration Inspecteur



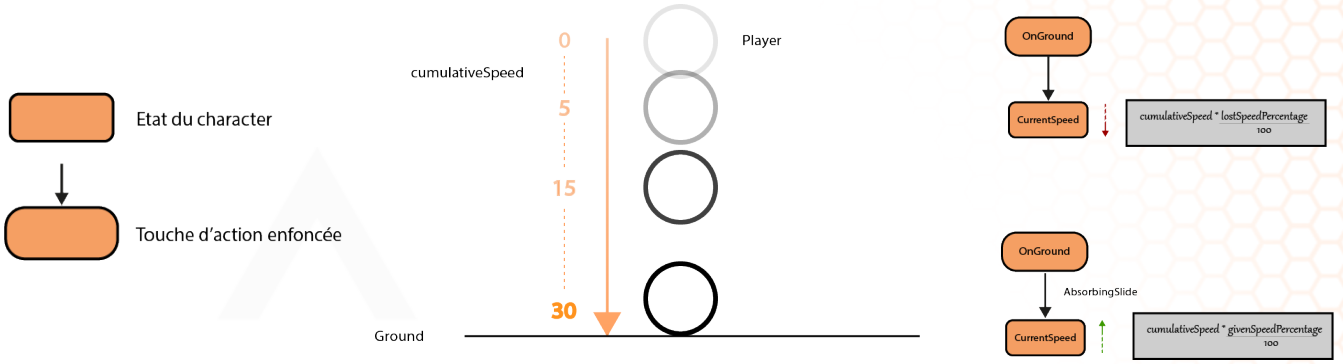
Accumulation de la vitesse de chute

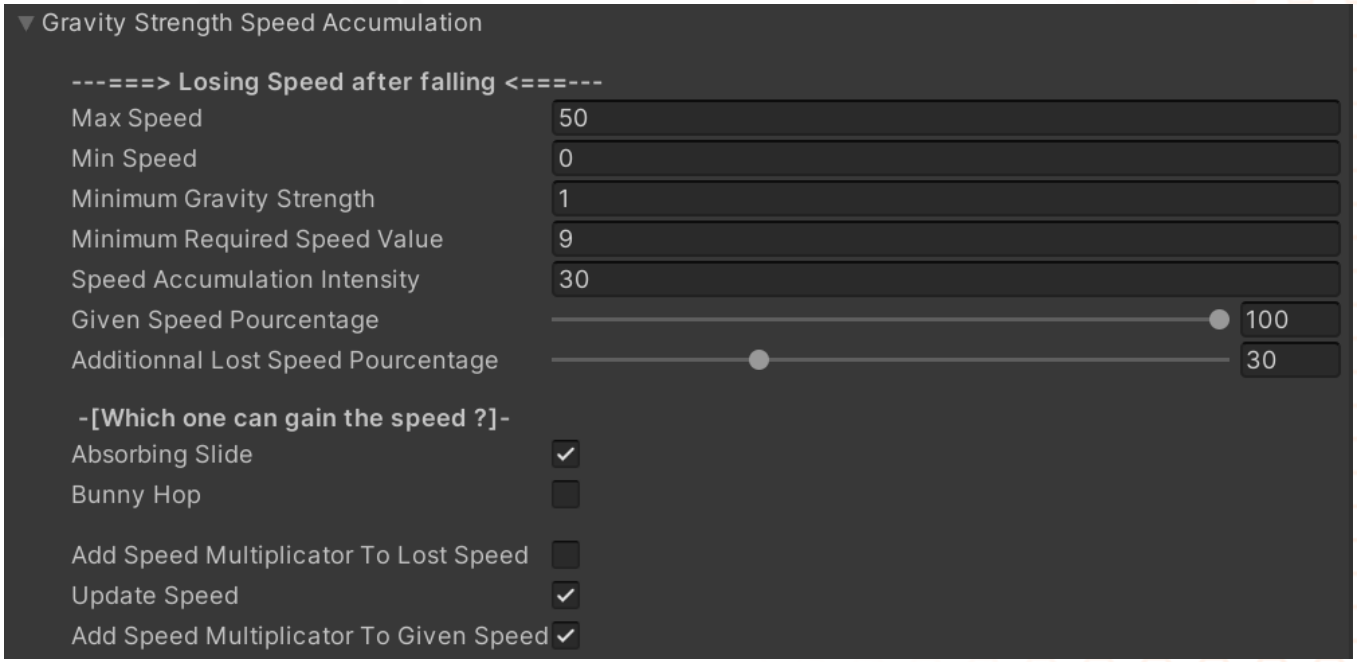
Lors de la chute du personnage, celui-ci va subir une accélération dépendant de sa vitesse de chute.

Cette accélération est paramétrable afin de pouvoir en contrôler plusieurs aspects :

- La vitesse de chute à partir de laquelle le personnage commence à accélérer
- La vitesse maximale cumulable

Schéma Cumulative Speed





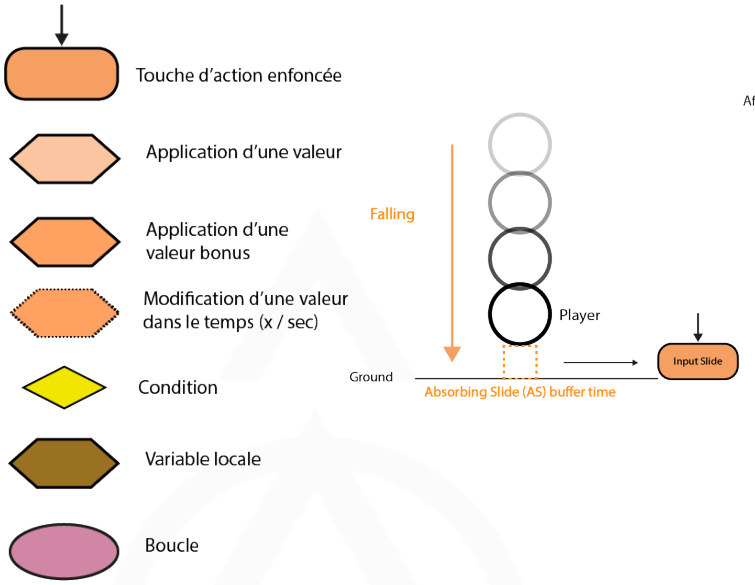
Vitesse de chute & Landing

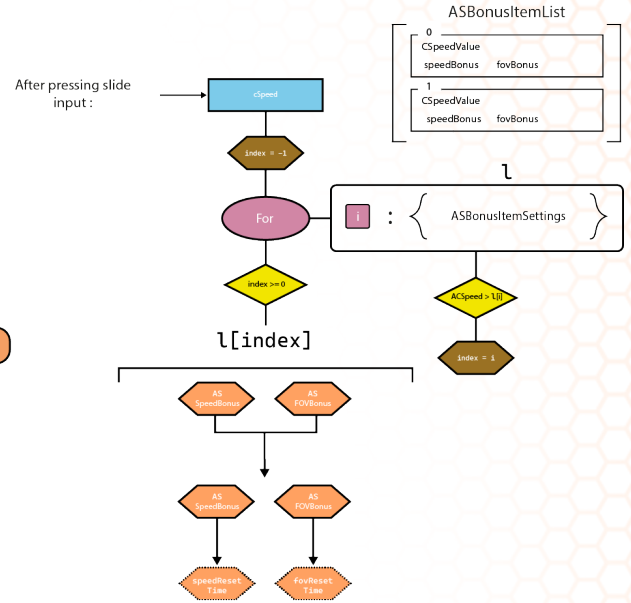
Il est possible de paramétrer la vitesse de chute afin de régler le pourcentage de cette vitesse qui sera utilisée pour les gains et pertes de vitesse lors des *Landings (Perfect et High-Fall)*.

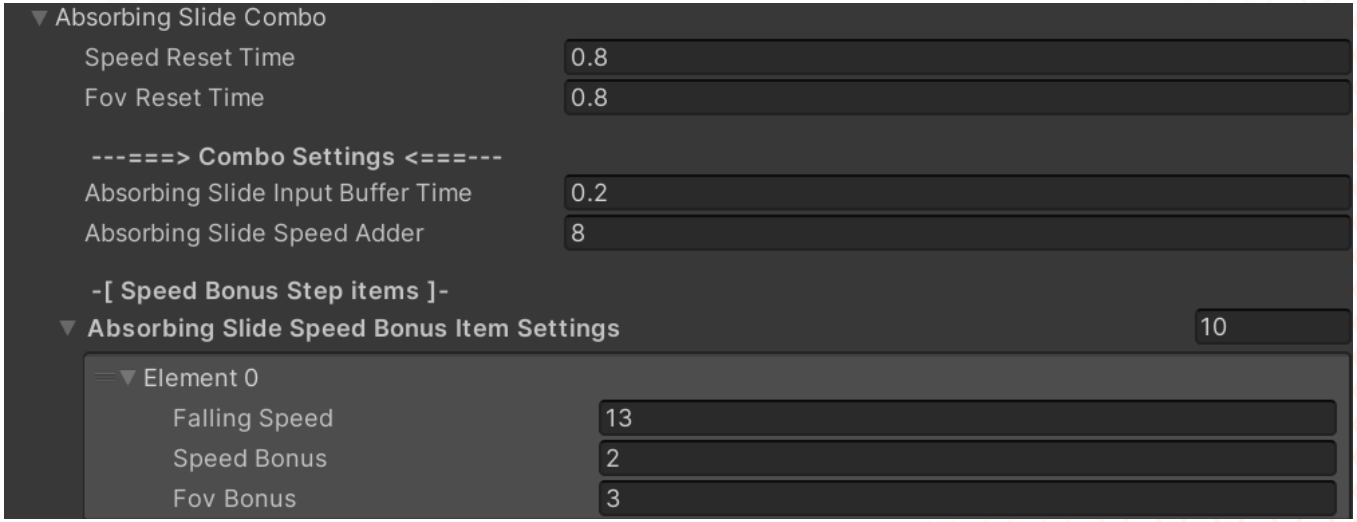
Il est également possible de déterminer quelles autres mécaniques de déplacement permettent de conserver cette vélocité verticale cumulée et de la transformer en vélocité horizontale.

Enfin, un booléen « *AddSpeedMultiplicator* » permet d'appliquer le coefficient de gain de vitesse ou non lors des gains et pertes de vitesse liés aux *Landings*.

Schéma Absorbing Slide







Speedlines

Les effets de particules signifiant la vitesse du personnage sont appliqués en continu. L'émetteur de particule est orienté selon la direction de déplacement du personnage, et le taux d'émission des particules varie en fonction de la vitesse actuelle du personnage, selon cette formule :

rate = minRate + (currentSpeed – minSpeed + misSpeedForEffect) / (maxSpeed – (minSpeed + minSpeedForEffect) * (minRate – maxRate)

minRate = taux d'émission minimum

maxRate = taux d'émission maximum

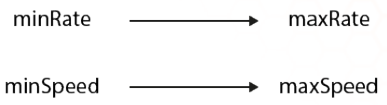
minSpeed = vitesse de déplacement minimum du personnage

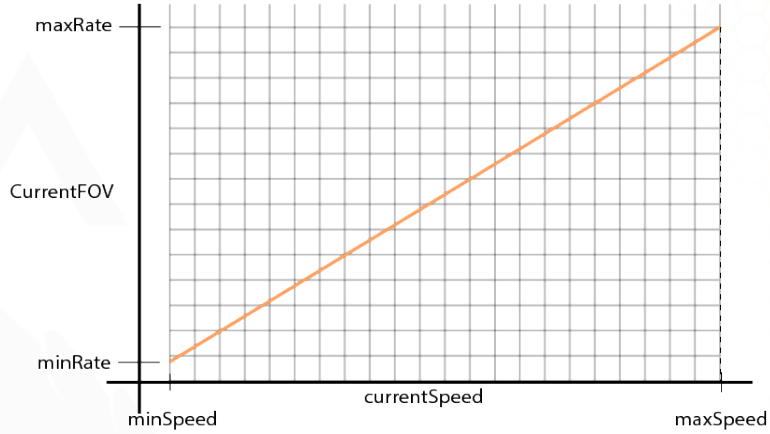
maxSpeed = vitesse de déplacement maximum du personnage (théorique)

currentSpeed = vitesse de déplacement actuelle du personnage

minSpeedForEffect = vitesse minimum requise pour que l'effet s'active

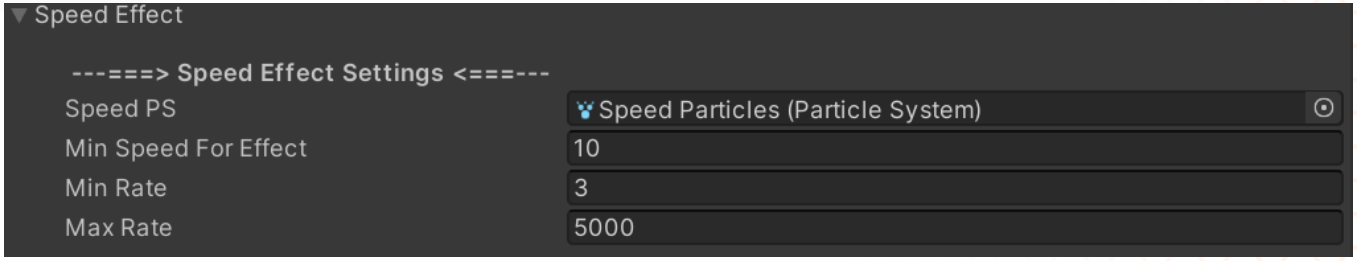
Schéma SpeedEffect





$$fov = \frac{currentSpeed - minSpeed}{maxSpeed - minSpeed} \bullet maxFov - minFov$$

Intégration Inspecteur



Accessibilité

Afin de rendre accessible à l'équipe de *Game Design* cette partie du code, nous avons dressé une liste de besoins :

- Des outils de création d'armes, de customisation d'armes, et d'*Upgrades d'armes*. Ces outils devront être accessibles et lisibles
- Il doit être possible d'itérer rapidement sur plusieurs types d'armes et simultanément
- Toutes les options doivent être modifiables depuis l'inspecteur
- Pour répondre à ces besoins, voici les solutions que nous avons mises en place et employées au cours du projet :
- Toutes les informations sont situées dans un seul et même script
- Ce script se sert des *Headers*, *Structures* ainsi que des Space afin que les variables soient lisibles et ordonnées
- Les armes et Upgrades utilisent des *ScriptableObjects* afin de pouvoir facilement les sauvegarder, les entreposer et les déplacer, aussi bien dans l'inspecteur qu'en temps réel
- Un script d'inspecteur a été conçu afin de permettre l'édition et l'affichage des *ScriptableObjects* lorsqu'ils sont référencés

Weapons

Générateur d'armes de base

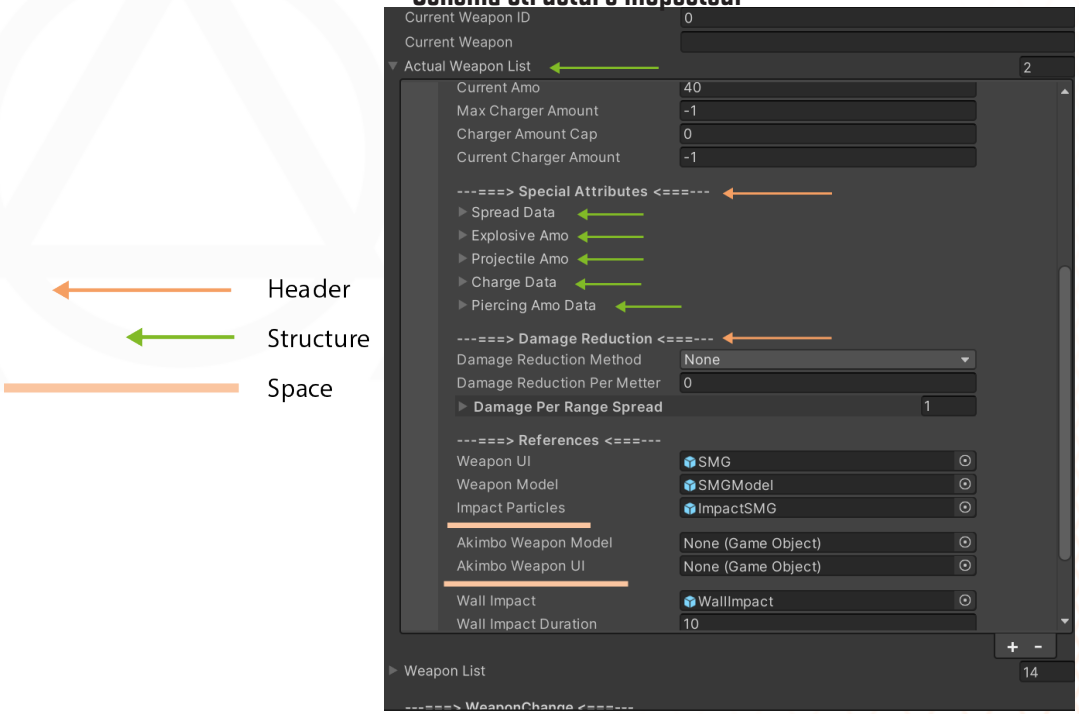
Stocker lesarmes

Les différents modèles d'armes disponibles dans le jeu sont stockés dans deux tableaux :

- Le premier contient les armes avec lesquelles le joueur commence le jeu, et s'actualise au fur et à mesure de sa *Run*
- Le second contient l'entièreté des armes créées et disponibles en jeu

Ces modèles d'armes prennent la forme de *ScriptableObjects*, modifiables directement dans l'inspecteur. Ils contiennent toutes les informations nécessaires au fonctionnement complet de l'arme : Modèle 3D, Metrics, Animations, Options, Références.

Schéma structure inspecteur



Détection de l'input

Par défaut, les armes fonctionnent avec un *Input* de type « *Press* » : le tir est déclenché à l'instant où le joueur appuie sur son *Input* de tir.

Cependant, afin d'englober différents Feelings d'armes et de pouvoir proposer une expérience de customisation complète au joueur, il existe de multiples types *d'Input* :

- Press : la détection se fait au moment de la pression de *l'Input* (I*nput.GetKeyDown*). Faire un appui continu n'effectuera qu'un seul tir
- *Hold* : la détection se fait en continu, tant que *l'Input* est pressé *(Input.GetKey)*. Faire un appui continu tirera en continu
- *Charge* : la détection se fait au moment du relâchement de *l'Input (Input.GetKeyUp)*. Ce type d'Input enregistre également la durée du maintien de la touche.

La détection de *l'Input* dépend du type, mais également d'autres paramètres modifiables en fonction de l'arme :

- Le *Cooldown* de tir
- Le nombre de munitions restantes
- La durée de l'*Input* (s'il y en a une)
- L'état du personnage / l'action actuelle du personnage

L'Input peut également altérer le comportement de l'arme. Ceci est réalisé via une structure contenant toutes les variables susceptibles d'être modifiées par *l'Input*.

Ces variables sont chacune associées à un multiplicateur de valeur 0, qui est modulé en fonction de l'influence de l'Input sur le comportement de l'arme.

(ex : un *Input* de type *« Hold »* peut modifier les dégâts qu'inflige l'arme. Alors le paramètre de durée de *l'Input* fera incrémenter proportionnellement le modificateur associé à la variable « *Damage* » de l'arme)

Tirer

Le tir fonctionne via *Raycasting*. Le système détecte ensuite les cibles qui ont été touchées, en filtrant les résultats par *Tag* et par *Layer* afin de ne pas prendre en compte les éléments d'interfaces et autre indésirables.

S'il s'agit d'un objet, un mur ou un sol, le système termine immédiatement le processus de tir et ajoute les *Feedbacks* correspondants (impacts de balle, son d'impact, etc.).

S'il s'agit d'un ennemi *(Tag == « Target »)*, le système remonte à la source du Transform en récupérant successivement les différents parents, afin de trouver le script *TargetBehaviour*. Le script T*argetBehaviour* détermine les différents comportements des ennemis et possède des fonctions qui vont être appelées afin de signaler à l'entité qu'elle a été touchée, a pris X dégâts, que la zone touchée est la suivante, etc.

Gestion des munitions

Lors du processus de tir, la première *Frame* va venir décrémenter le nombre actuel de munitions de l'arme active du personnage d'un montant équivalent au nombre de munitions tirées.

Si ce nombre est supérieur au nombre de munitions restantes dans le chargeur de l'arme active du personnage, alors le processus de tir est enclenché avec un nombre de munitions équivalent au nombre de munitions restantes dans le chargeur. Si ce nombre est 0, alors le processus de tir ne s'enclenche pas.

Si le joueur *Input* R pour recharger, le processus de rechargement se lance. Le personnage ne peut plus tirer, et le nombre actuel de munitions dans le chargeur de son arme active est remplacé par le nombre maximal de munitions dans le chargeur de son arme active. Le nombre de chargeurs disponibles de son arme active est décrémenté de 1.

Changement d'arme

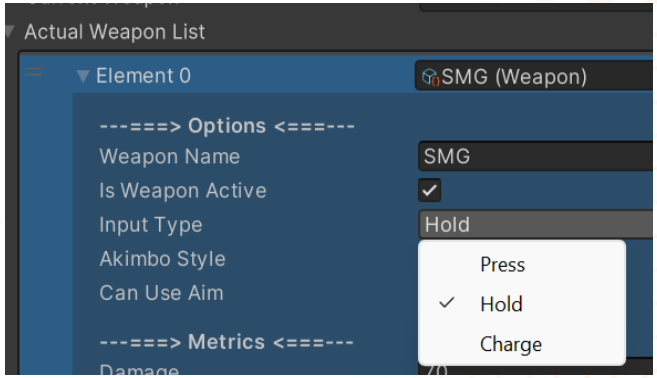
Par défaut, le système considère que le personnage possède deux armes simultanément, stockées dans un tableau d'armes donc les ID correspondent au type des armes (primaires ou secondaires).

Si le joueur I*nput mouseWheel* pour changer d'arme, le système va incrémenter ou décrémenter l'ID actuel de l'arme du personnage. Il va ensuite *Unload* l'arme actuelle du joueur et *Load* l'arme correspondant au nouvel ID de l'arme du personnage.

Les *Loadings* et *Unloadings* concernent l'entièreté des variables et objets attachés à l'arme :

- Le modèle 3D
- Le Canvas et les Feedbacks 2D
- Les Audio Events
- Les Particle System
- Etc.

Vue de l'inspecteur de la liste d'arme



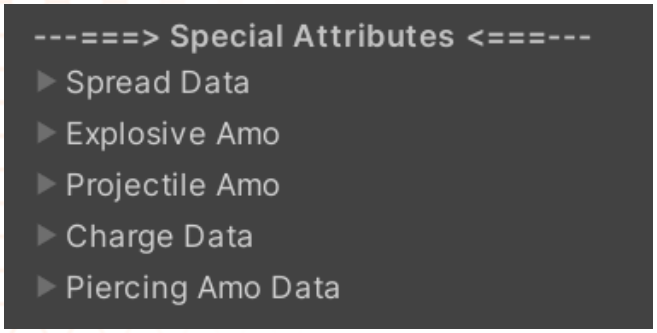
Customisation

La customisation d'arme permet de créer des combinaisons de comportements uniques afin de proposer des armes intéressantes au joueur. Ces comportements sont activables et désactivables dans l'inspecteur, et sont entièrement modifiables.

Les différentes fonctions présentes dans ces comportements *Overrident* les fonctionnements de base de l'arme.

Voici les différentes comportements ajustables des armes.

Vue de l'inspecteur des différentes custimisations



Spread

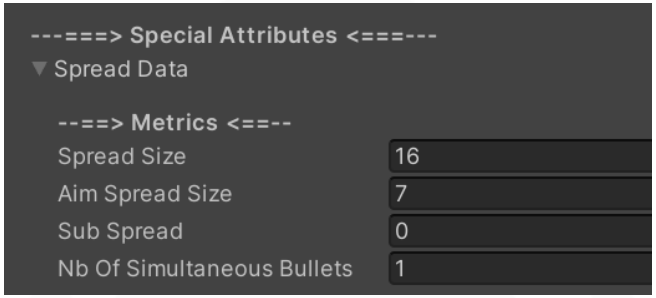
Ce comportement simule une dispersion de balles.

Le système commence par déterminer un point de l'écran qui agira comme nouvelle cible du *Raycast* de l'arme. Cette nouvelle cible est déterminée par une fonction aléatoire basée sur le centre de l'écran : il s'agit d'un point pris au hasard dans un rayon donné autour du centre de l'écran.

Ce comportement s'accompagne d'une modification du nombre de balles tirées simultanément. Une variable permet de modifier le nombre d'itérations du Raycast, simulant un tir simultané de plusieurs balles dispersées.

Il est possible de déterminer l'étendue du *Spread* en modifiant la variable *SubSpread*, qui détermine l'espacement maximal entre deux *Raycasts* successifs. Ainsi, il est possible de créer des *Spreads* plus ou moins concentrés.

Vue de l'inspecteur du spread



Tirs explosifs

Ce comportement simule une explosion ayant lieu à l'impact de le balle.

Le système commence par détecter toutes les entités situées autour du point d'impact dans un rayon donné (*SphereCast)* puis les trie par *Tag* et par *Layer*.

Toutes les entités correspondant à des ennemis *(Tag == « Target »*) seront remontées jusqu'à la source, et la fonction de prise de dégâts sera appelée.

Les dégâts de l'explosion sont calculés suivant cette formule :

$$((maxDMG - minDMG) * (distance / radius)) + minDMG$$

minDMG = les dégâts minimaux de l'explosion pour toute entité touchée

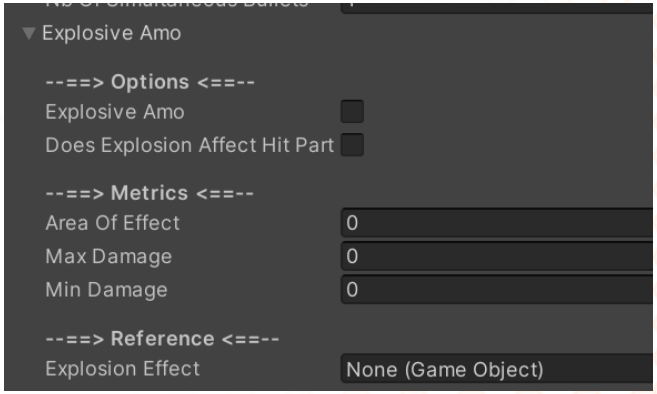
maxDMG = les dégâts maximaux de l'explosion pour toute entité touchée

radius = le rayon d'effet de l'explosion

distance = la distance séparant le centre de l'explosion et l'entité touchée

Ainsi, l'explosion inflige des dégâts variants dans une plage définie en fonction de la distance au centre de l'explosion.

Vue de l'inspecteur des type de munitions



Tirs Transperçants

Ce comportement vient modifier le comportement du tir de base de l'arme : au lieu de s'arrêter au premier objet touché, le système va enregistrer une valeur de pénétration, qui va décrémenter à chaque objet touché par le *Raycast*.

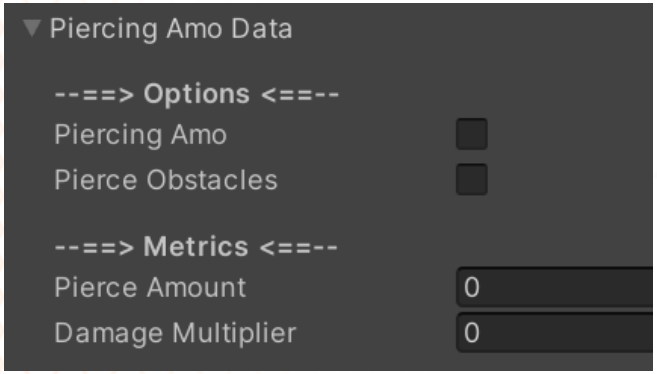
Les entités touchées par le *Raycasts* sont ensuite récupérées par la fonction Physics.*RaycastAll*, qui renvoie un tableau de *Colliders*. Ces *Colliders* sont ensuite triés par distance, afin de déterminer quelles sont les premières entités à avoir été touchées.

Cette option permet donc de simuler des balles qui traverseraient un nombre prédéterminé d'obstacles / entités.

Il existe une variable « *Damage Multiplier* » qui correspond à un coefficient multiplicateur des dégâts infligés, appliqué entre chaque instance de pénétration. Cela permet par exemple de décrémenter les dégâts en fonction du cardinale de l'entité touchée.

Il est important de référer dans un tableau annexes les différentes entités déjà touchées afin de les ignorer lors de l'utilisation du *Physics.RaycastAll* : les entités étant composées de plusieurs Colliders, elles ne doivent pas être touchées plusieurs fois par le même *Raycast*.

Vue de l'inspecteur du Piercing Amo



Tir de projectiles

Ce comportement simule des projectiles physicalisés à la place du *Raycasting*. Cela implique plusieurs paramètres :

- Le *Travel Time*
- La gravité
- La distance maximale de tir / la durée maximale d'existence du projectile
- Les rebonds sur les différents obstacles
- Les tirs téléguidés ou *Homing*

Lors de l'Input de tir, le système instancie un objet et lui attribue une vélocité initiale. Dès lors qu'il est instancié, l'objet va ensuite agir de manière autonome.

Il se détruit lorsque son *Timer* est expiré ou s'il rentre en contact avec un obstacle, dépendamment de l'option choisie. Une fois que le projectile est entré en contact avec un *Collider,* il se comporte comme un *Raycast* normal.

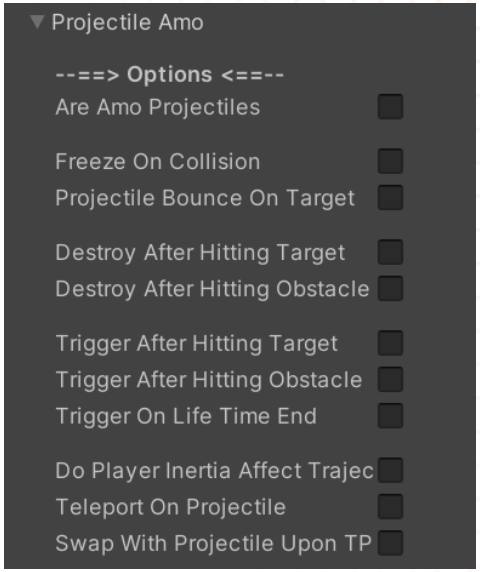
Cas particulier : Le Homing

Le *Homing* est un cas particulier de tir de projectile. Il s'agit d'un comportement qui simule un projectile téléguidé / à tête chercheuse.

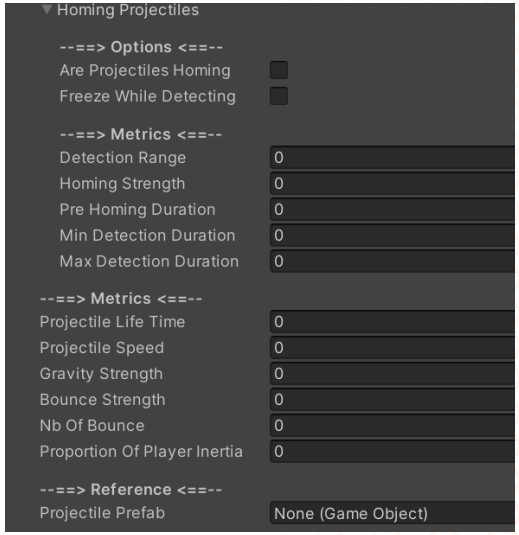
Afin de fonctionner correctement, le projectile effectue une détection d'ennemis dans son périmètre *(SphereCastAll)* en temps réel et détermine sa cible comme l'ennemi le plus proche sur sa trajectoire. Il va ensuite modifier celle-ci pour se diriger en direction de la cible.

Il est possible de modifier le délai avant l'activation du *Homing*, la vitesse d'adaptation du projectile ou encore l'inertie donnée au projectile.

Vue de l'inspecteur des différentes custimisations



Vue de l'inspecteur du Homing



Mode Akimbo

Ce comportement est un changement drastique du comportement de base des armes, il s'agit d'un *Override* complet.

Ce comportement simule une paire d'armes, disposées dans les mains respectives du personnage. Il utilise les mêmes types d'Input que les autres armes, mais considère une nouvelle variable correspondant au *MouseClick* du joueur : le *LeftClick* fait réagir l'arme de gauche et le *RightClick* fait réagir l'arme de droite.

Le système utilise le même fonctionnement inhérent de l'arme, mais fait référence à des sets de données différents en fonction de *l'Input* du joueur. Ce mode demande donc la création d'un set de variables séparé afin de suivre l'évolution des données de la seconde arme.

Ce set de variables est créé en instanciant une copie, dont on garde une référence attachée à l'arme active actuelle du personnage.

Statistiques & Upgrades

Afin de gérer les *Upgrades*, une paire de variables est associée à chaque caractéristique : l'une représente la valeur brute et l'autre représente le modificateur.

La valeur brute correspond à la valeur initiale de la statistique. Il peut s'agir de *Float*, de *Vector2* dans le cas d'une plage de valeurs ou d'une *List<Float>* dans le cas d'une *Pool* de valeurs fixe. Cette valeur ne change jamais et détermine l'aspect initial de l'arme.

Le modificateur est une valeur variable, représentant la modification en pourcentages de la valeur initiale de la statistique. A chaque *Upgrade*, le modificateur va incrémenter ou décrémenter de la valeur indiquée sur la carte *Upgrade* choisie.

La valeur finale de la statistique en jeu est calculée selon cette formule :

$$FinalValue = BaseValue + BaseValue * (Mod / 100)$$

Les modificateurs ne sont pas référencés sur l'arme mais sur l'index de l'arme. Ainsi, toutes les armes primaires ont des modificateurs communs, et les secondaires aussi. Ces valeurs n'évoluant que lors d'une Run, cela implique que toutes les armes de même index subissent les changements apportés à une arme. Ainsi, si le joueur décide de sélectionner une Upgrade lui modifiant son arme primaire ou secondaire, celle-ci conserve ses modificateurs.

Variables du code des Upgrades

```
[Serializable]
17 références
public struct StatModifiers
{
    [Header("- Base Metrics -")]
    public StatAccessor damageMultiplier;
    public StatAccessor shootCooldownMultiplier;
    public StatAccessor reloadDurationMultiplier;
    public StatAccessor chargerSizeMultiplier;

    [Header("- SpreadSize -")]
    public StatAccessor spreadSizeMultiplier;
    public StatAccessor subSpreadSizeMultiplier;
    public StatAccessor nbOfBulletsMultiplier;

    [Header("- Explosion -")]
    public StatAccessor explosionAOEMultiplier;
    public StatAccessor explosionDamageMultiplier;

    [Header("- Piercing Amo -")]
    public StatAccessor pierceAmountMultiplier;


    [Header("- Projectile -")]
    public StatAccessor projectileLifetimeMultiplier;
    public StatAccessor projectileSpeedMultiplier;
    public StatAccessor projectileGravityStrengthMultiplier;
    public StatAccessor projectileBounceMultiplier;

    [Header("- DamageReduction -")]
    public StatAccessor rangeDistanceModifier;
    public StatAccessor rangeFlatDamageReductionModifier;
    public StatAccessor rangeDamageModifierModifier;
    public StatAccessor rangePercentageLostPerMetterModifier;
    public StatAccessor rangeminDamageModifier;
```

Feedbacks

Animations

Chaque arme est liée à un *Animator* qui suit la même structure d'états, et qui est mis à jour en temps réel par le biais d'une variable.

De plus, toutes les armes sont liées à l'*Animator* du *Crosshair,* qui vient récupérer les données de chaque tir. En fonction des paramètres du tir (manqué, touché, touché dans un point faible, achevé, etc.), la variable indiquant l'état du Crosshair évolue, afin de jouer l'animation correcte.

On distingue donc 6 animations différentes :

- *Hit* : la cible est touchée
- *CritHit* : la cible est touchée sur un point faible
- *Kill* : la cible est achevée
- *CritKill* : la cible est achevée sur un point faible
- *HitEgg* : la cible est un œuf
- *Miss* : le tir est manqué

Feedbacks Sonores

Les *Feedbacks* sonores sont instanciés au niveau de l'arme ou au niveau du point d'impact du *Raycast* en fonction de l'effet désiré.

Les sons d'impact sont instanciés puis libérés sur le coup.

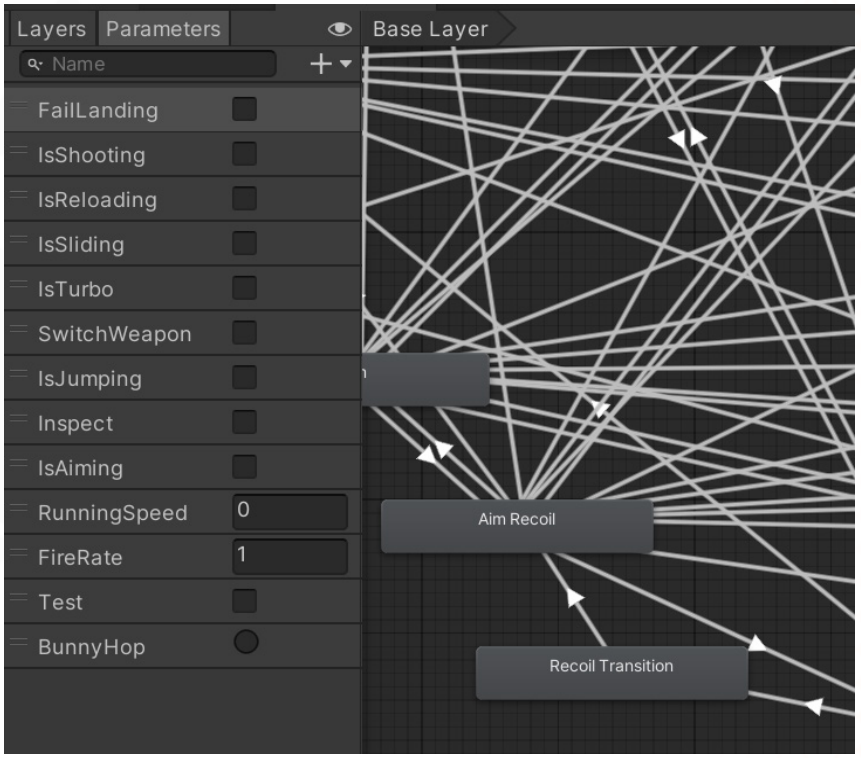
Les sons de l'arme en revanche, étant réutilisés fréquemment, possèdent leurs *Audio Events* alloués au changement de l'arme et libérés au chargement de la suivante. Cela permet d'optimiser les performances du système en gardant chargée une poignée *d'Events* plutôt que de les instancier de nombreuses fois.

Autre Feedbacks

Parmi les autres formes de *Feedback,* il existe les *Lights*, les *Particle System,* etc.

Ceux-ci sont référés directement dans le *ScriptableObject* et instanciés / appelés au cas par cas.

Parametres de l'animator du Player



Accessibilité

Afin de rendre accessible à l'équipe de *Game Design* cette partie du code, nous avons mis en place plusieurs méthodes :

- Mise en place d'un outil de création *d'IA* par *ScriptableObjects*
- Système d'ennemis entièrement modulable et customisable depuis l'inspecteur
- Structure commune à tous les ennemis

Réussir à créer une structure unique capable de prendre en compte tous les types d'ennemis a été le challenge le plus difficile de programmation concernant les ennemis.

Structure des ennemis

Hiérarchie

Les ennemis sont implémentés dans le projet comme des objets, et suivent la hiérarchie suivante :

- Un parent, qui possède le script de comportement ainsi que les composants qui se rapportent au NavMesh
- Un enfant, qui sert à gérer l'orientation par rapport au terrain
- Un sous-enfant qui contient l'*Animator* et l'ensemble des modèles 3D et Colliders de l'entité

Comportement

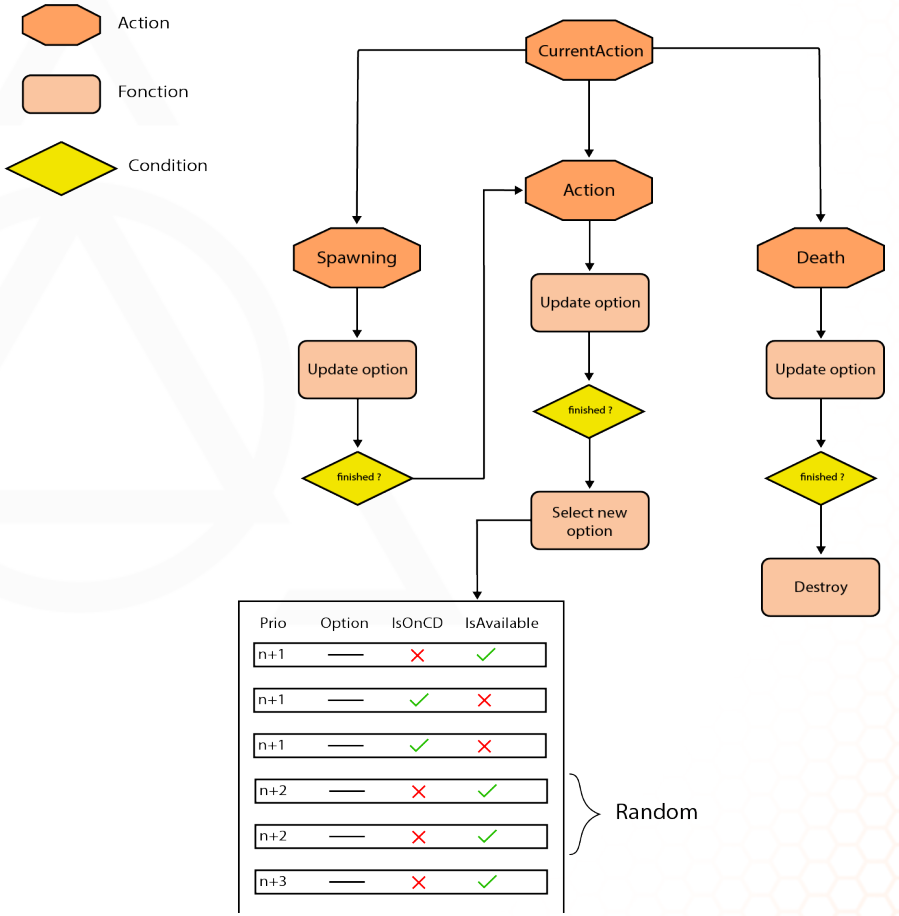
Structure Globale

La structure du comportement des agents hostiles est toujours la même. Elle est structurée à l'aide de 3 types de briques élémentaires, de logique ou d'actions :

- Les conditions d'entrée : des tests booléens de vérification de propriétés. Une condition d'entrée est un appel de fonction renvoyant vrai ou faux
- Les conditions d'arrêt : des tests continus en temps réel de vérification de propriétés. Elles sont vérifiées à chaque *Frame*
- Les actions : sont un ensemble comprenant un comportement et une liste de conditions d'arrêt. Un comportement est une action effectuée par *l'IA* des monstres

On assemble ces briques par 3 dans des blocs appelés « Options ».

Schéma State Machine des mobs



L'ensemble des options attachées à un agent hostile représente donc l'ensemble des actions qu'il peut potentiellement faire, ainsi que les conditions sous lesquelles il les fait / il s'arrête de les faire.

Les options possèdent donc :

1. Une liste de conditions d'entrée, correspondant à la possibilité ou non de rentrer dans l'option
2. Une liste d'actions à effectuer jusqu'à ce qu'elles soient finies (conditions d'arrêt vérifiées). Lorsque l'action est finie, on sort de l'option
3. Un niveau de priorité (plus la valeur est basse, plus le niveau de priorité et la chance que l'option soit sélectionnée sont hauts)
4. Un poids, qui permet de départager deux options de priorités équivalentes. Plus le poids est élevé, plus la chance que l'option soit sélectionnée est haute
5. Un *Cooldown*, représentant le temps minimum avant qu'une action soit répétable

A chaque *Frame*, le comportement des ennemis est actualisé.

Un ennemi peut-être dans deux états différents : en train de faire une action, ou non.

Si l'agent est en train d'accomplir une action, alors il vérifie à l'aide de ses conditions d'arrêts si l'action doit continuer à la Frame suivante, ou si elle doit s'arrêter.

Dans le cas spécifique d'une *DeathAction* qui est un type d'action correspondant à la mort d'un ennemi, l'entité se détruit dès la fin de l'action.

Si l'agent n'est pas en train d'accomplir une action, alors le système entame une procédure de choix afin de déterminer la future action de l'agent :

- Pour chaque palier de priorité en partant de la plus haute (0), le système va lister les options possibles (celles dont les conditions d'entrées sont vraies, et dont le *Cooldown* est à zéro)
- S'il en existe au moins une dans le palier, alors le système va tirer au hasard l'une d'entre elle, en pondérant les différentes probabilités par le poids de ces options
- Si aucune d'entre elle n'est valide, le système passe au palier de priorité suivant (n+1) jusqu'à trouver un palier dans lequel au moins 1 option est valide
- Si le système a parcouru l'entièreté des options de chaque palier et qu'aucune d'entre elles n'est valide, alors l'option par défaut va être sélectionnée, et le système va continuer en temps réel à vérifier les paliers jusqu'à ce qu'une autre option devienne valide

Dans le cas spécifique d'une option contenant une *DeathAction*, l'option est sélectionnée à la *Frame* où l'agent tombe à 0 points de vie ou moins, sans tenir compte de l'état de l'action précédente (finie ou non).

Conditions d'entrée

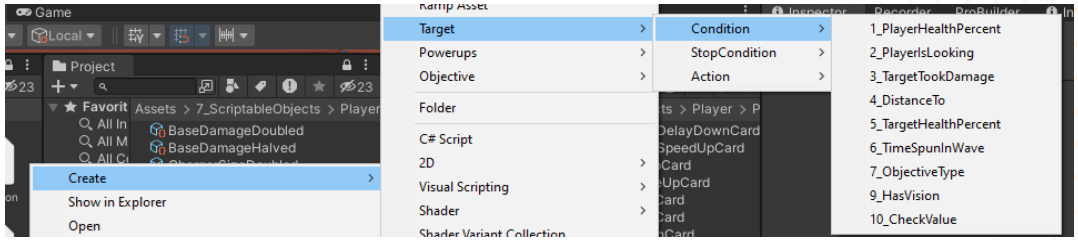
Chaque condition d'entrée hérite d'une classe commune « Condition » qui implémente une méthode de validation. Chaque condition va ensuite *Override* cette méthode afin de déterminer sa propre valeur de retour.

En outre, afin de maximiser les options disponibles à l'équipe de *Game Design* et de réduire le nombre de scripts, des options de logique sont implémentées dans la classe (comme l'opérateur *NOT*, par exemple).

Conditions d'Arret

Les conditions d'arrêt sont structurées de la même manière que les conditions d'entrée, et ont été optimisées afin d'intervenir sur la durée. Une action se termine si au moins l'une de ses conditions d'arrêt retourne vrai.

Création de condition d'entrée scriptables objects



Voici les différentes conditions d'entrée disponibles :

- *PlayerHealthPercent* : récupère le taux de points de vie du personnage, et retourne vrai si celui-ci se trouve dans une plage donnée
- *PlayerIsLooking* : retourne vrai si le personnage est orienté dans la direction de la cible, en prenant en compte une marge paramétrable
- *TargetTookDamage* : retourne vrai si la cible vient de subir des dégâts non-nuls
- *DistanceTo* : récupère la distance entre le personnage et la cible, et retourne vrai si celle-ci se trouve dans une plage donnée
- *TimeSpentInWave* : récupère le Timer de la Wave en cours, et retourne vrai si le temps écoulé se trouve dans une plage donnée
- *ObjectiveType* : récupère le type de la Wave en cours, et retourne vrai s'il est identique à celui référencé dans l'inspecteur
- *HasVision* : retourne vrai s'il existe une ligne de vue dégagée entre le personnage et la cible (vérifie que la cible est la première cible touchée par un Raycast partant de la caméra vers la cible)
- *CheckValue* : récupère une variable par nom (String), et retourne vrai si sa valeur est identique à celle référencée dans l'inspecteur

Création de condition d'arrêt scriptables objects

Voici les différentes conditions d'arrêt disponibles :

- *Timer* : crée un Timer interne et retourne vrai une fois que celui-ci est écoulé
- *AnimationEnded* : récupère l'état actuel de l'Animator de l'agent et retourne vrai si celui-ci est différent de celui référencé dans l'inspecteur (signifiant que l'animation est terminée)
- *StopByAnimation* : récupère une variable modifiable par Event directement dans l'Animator, et retourne vrai si la variable identique à celle référencée dans l'inspecteur est modifiée. Cette condition d'arrêt permet d'appeler l'arrêt de l'action directement depuis l'Animator, afin de ne pas avoir à régler chaque Timing au cas par cas

Actions

Les actions possèdent une structure plus complexe. Elles héritent d'une classe commune « Action » et sont chacune composée de méthodes uniques qui viennent Override les méthodes que l'on désire changer.

On distingue 3 méthodes majeures :

- *Initialize*
- *Update*
- *Uninitialize*

Ces méthodes ont comme simple but d'appeler les méthodes virtuelles correspondantes dans chacun des scripts qui héritent de la classe « Action ».

La méthode Initialize met en place les valeurs et méthodes de base de toutes les actions, puis appelle la méthode virtuelle *Initialize_* qui est celle qui est *Overridée* dans les scripts héritant de la classe « Action ».

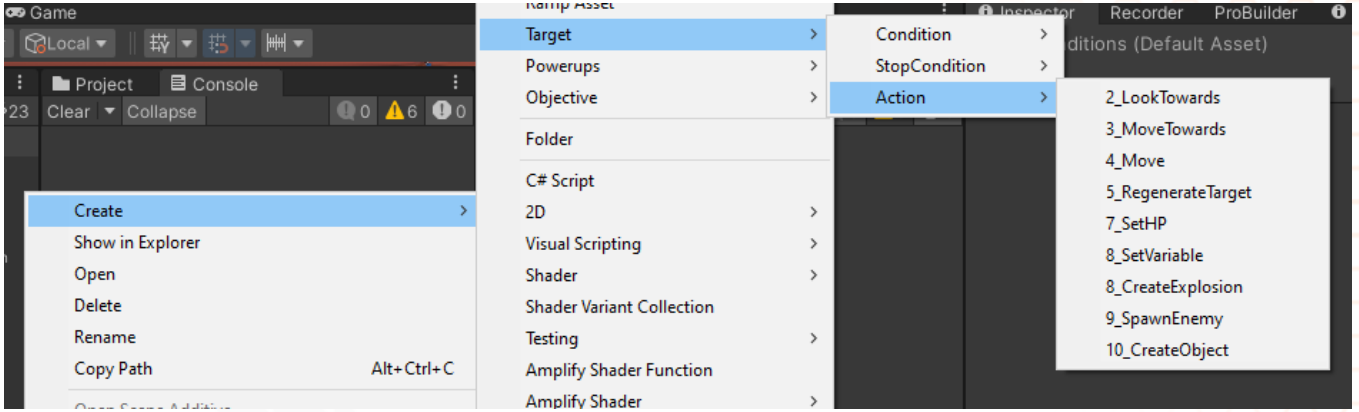
La méthode *Update* est appelée à chaque *Frame*, et retourne un booléen en fonction de l'état de la *Frame* en cours : elle retourne vrai si la Frame en cours est la dernière de l'action en cours. Elle appelle la méthode virtuelle *Update_* qui est celle qui est *Overridée* dans les scripts héritant de la classe « Action ».

La méthode *Uninitialize* s'assure du bon *Unloading* des différentes valeurs et méthodes puis appelle la méthode virtuelle Uninitialize_ qui est celle qui est *Overridée* dans les scripts héritant de la classe « Action ».

Les actions peuvent également être associées à un délai de commencement, qui permet de temporiser le début de l'action. Ce délai est créé par une section de la phase d'action en trois méthodes :

1. *Start()* : représente la toute première Frame appelée
2. *Continuous()* ou *Update()* : représentent toutes les Frames suivantes
3. *End()* : représente la dernière Frame

Création d'action scriptables objects



Voici les différentes actions disponibles :

- *LookTowards* : immobilise l'entité et l'oriente vers la cible
- *MoveTowards* : utilise le système de PathFinding de Unity afin que l'entité emprunte le chemin le plus court vers la cible
- *Move* : déplace l'entité relativement à son orientation ou à la position du personnage
- *RegenerateTarget* : modifie la valeur des points de vie de l'entité sur une durée prédéfinie
- *SetHP* : attribue une valeur brute aux points de vie de l'entité
- *SetVariable* : attribue une valeur brute à une variable de l'inspecteur
- *CreateExplosion* : crée une zone d'effet qui inflige des dégâts au personnage, aux Foreuses ainsi qu'aux autres entités sélectionnées dans la zone
- *SpawnEnemy* : instancie un ennemi sélectionné parmi une Pool d'ennemis
- *CreateObject* : instancie un Prefab à l'endroit référencé

Game Managing

Spawning

Le système de *Spawn* des ennemis est géré par un script de GameManaging situé sur un Empty de la scène. On distingue 3 formes de *Spawning* :

- Les *Spawns* principaux, comme les Vers dont l'élimination met fin à la *Wave*
- Les *Spawns* passifs, générés par la *Wave* elle-même
- Les *Spawns* actifs, générés par des entités qui en invoquent d'autres

Le *GameManager* possède une paire de variables correspondant à un nombre maximum d'entités *Spawnables* respectivement avec les *Spawns* actifs et passifs. Si un *Spawn* actif ou passif est appelé alors que le nombre d'entités a atteint ce maximum, alors le *Spawn* n'a pas lieu.

Afin de faire apparaître une entité, le système cherche par itérations successives un point de l'espace qui satisfait les conditions de *Spawn* et instancie l'entité à l'endroit donné.

Pour ce faire, le système génère un point aléatoire dans un rayon autour du personnage, et essaye de le rattacher au *NavMesh* de la scène. S'il y arrive, alors il instancie l'entité, sinon, il génère un autre point.

Gestion des points de vie

Deux paramètres renseignés dans l'inspecteur sont pris en compte pour gérer l'état de santé des agents hostiles.

- Le nombre de points de vie maximum : le nombre de points de vie de départ d'un monstre, plus cette valeur est élevée plus il est long à tuer
- La liste des *Colliders* : un tableau renseignant tous les Colliders de l'entité, appairés à leurs coefficients multiplicateurs de dégâts respectifs. C'est cette liste qui va permettre de gérer les coups critiques, ou les coups aux dégâts nuls afin d'établir les *Feedbacks* du *Crosshair* de l'arme

Tous les Colliders de la liste possèdent le *Tag « Target »*. Lorsque le Raycast de tir du personnage rencontre l'un des nombreux *Colliders* du monstre, il remonte alors l'information au script du parent, qui associe le *Collider* touché au multiplicateur correspondant.

Il calcule ensuite les dégâts subis par l'agent hostile, et renvoie son état après le traitement de cette valeur.

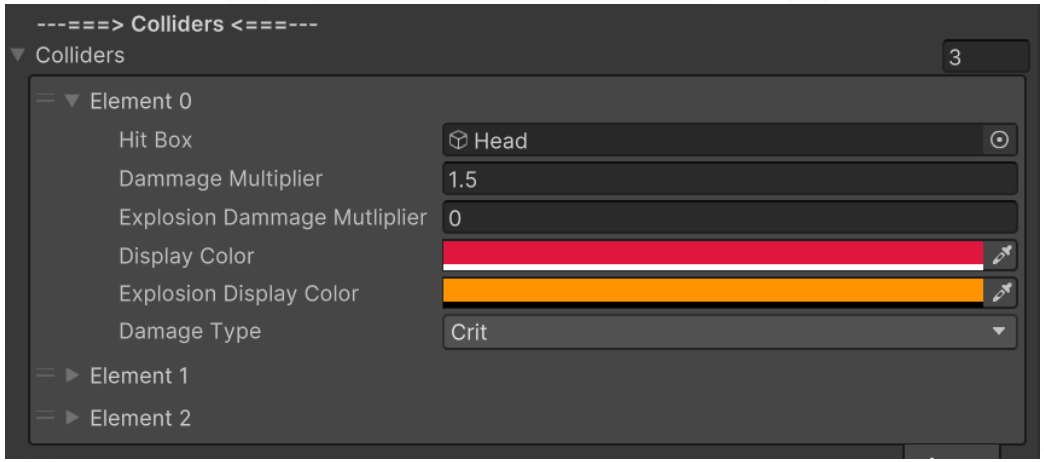
Il renvoie un état et une valeur différents en fonction de s'il a été touché, touché sur une zone critique, s'il est détruit, etc.

Optimisation & contraintes techniques

Il existe plusieurs points sur l'équipe de *Game Programming* souhaiterait travailler afin d'améliorer l'optimisation et les performances du projet :

- L'instanciation des entités est très coûteuse, il serait possible de développer deux solutions. Une première serait de brider le processus de recherche de points dans l'espace en fixant une valeur maximum d'itération. Une seconde serait de stocker l'entièreté des entités dans un objet sous la scène, afin de n'avoir à les instancier qu'au lancement de celle-ci. Ensuite, un *Spawn* de monstre ne serait plus qu'un déplacement de Transform, tandis qu'une mort serait un déplacement de *Transform* accompagné d'un *Reset* des valeurs.

Vue de l'inspecteur des Colliders d'un mob



Menuing

Transitions

Les transitions entre les différentes scènes s'effectuent via le *SceneLoader*, un objet défini afin de ne pas être *Unload* en quittant la scène. En outre, il s'agit d'un Singleton, afin de s'assurer qu'il n'en existe une seule instance et que les données qu'il contient soient constantes.

Layers

Les menus fonctionnent par *Layers* artificiels : chaque écran du menu est regroupé dans son propre *Canvas*, et chaque *Canvas* est référencé dans le *Manager* global de la scène de menus.

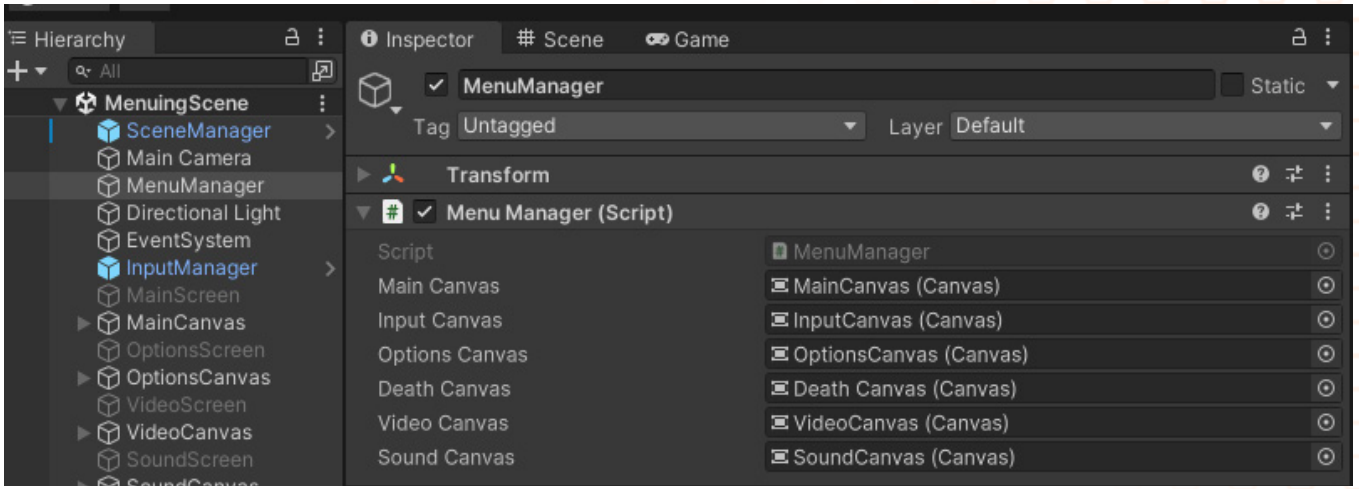
Ainsi, le système active et désactive des écrans entiers, ce qui permet de basculer de menu en menu en une seule action. Ces fonctions de basculement étant référencées de manière publique dans le *Manager*, il est possible d'y accéder depuis des éléments *d'UI* présents dans la scène.

Contenu

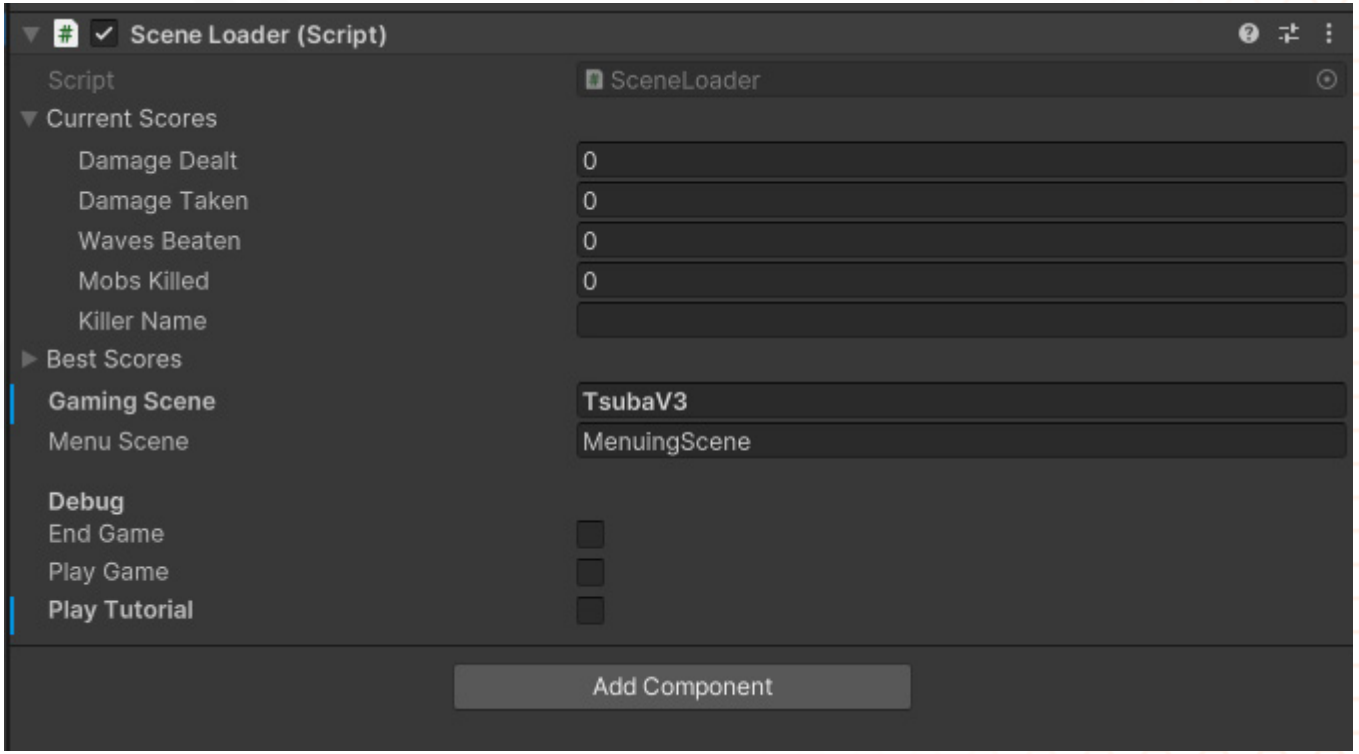
Le *Menuing* est construit à l'aide des outils *UI* de *Unity* : des Sliders pour les volumes, une section de *Scrolling* pour les Inputs, un *Toggle* pour activer / désactiver le tutoriel, etc.

Ces fonctionnalités sont reliées au *MenuManager* grâce à ses différentes méthodes. Une fois les données mises à jour, il appelle les méthodes du *SceneManager* qui permet d'effectuer les changements de scène.

Inspecteur Menu Manager Script



Inspecteur Scene Loader



Game Managing

Waves

Fonctionnement

Les *Waves* sont gérées par le *GameManager*, à travers différents scripts :

- Le *MobSpawner* s'occupe de faire apparaître et disparaître, ainsi que gérer l'intégralité des entités en temps réel
- L'*ObjectiveManager* s'occupe de la gestion des différents objectifs et de l'enchaînement correct des *Waves*

Chaque *Wave* est implémentée et stockée grâce à un *ScriptableObject,* héritant de la classe *« Wave ».*

Cette classe contient plusieurs procédures, qui sont appelées successivement, grâce à la fonction UpdateWave :

- *_PrepareWave* : est une fonction virtuelle appelée lors de l'anticipation de la *Wave*
- *BeginWave* : contient toutes les méthodes générales à appliquer, ainsi qu'un appel à *_BeginWave* qui est définie pour chaque héritier de manière unique
- *UpdateWave* : est appelée à chaque *Frame* et contient un appel à *_UpdateWave* qui est définie pour chaque héritier de manière unique
- *EndWave* : est une fonction virtuelle qui permet de réinitialiser le système afin de pouvoir correctement passer à la *Wave* suivante

Type de Wave

Il y a pour l'instant 3 types de *Waves* :

- Le *Tutorial* : cette *Wave* ne contient pas d'anticipation et ne possède donc pas *d'Override _BeginWave*. Son *UpdateWave* décrit une suite d'étapes à parcourir par le personnage, jalonnée de tests *d'Inputs*

Bien que la structure soit globalement la même pour chaque étape du tutoriel, cette *Wave* est Hard-codée

- Les DrillDefense : cette Wave correspond à la défense de Foreuses
  - *_PrepareWave* : les Foreuses se déploient, leurs *Colliders* s'activent et leurs points de vie s'initialisent
  - *_UpdateWave* : les points de vie des Foreuses sont vérifiés. Si l'une d'entre elle possède 0 point de vie ou moins, le système déclenche la condition de défaite et *Load* l'écran de fin de partie. Si la durée de la *Wave* est écoulée, le système déclenche la condition de victoire et met fin à la *Wave*
- *_EndWave* : les Foreuses se replient, leurs *Colliders* se désactivent
- Les *NestsOffense* : cette *Wave* correspond à la destruction de Vers
  - *_PrepareWave* : les Vers sont instanciés, les sons qui les accompagnent sont spatialisés aux points d'apparition et leurs points de vie s'initialisent
  - *_UpdateWave* : les points de vie des Vers sont vérifiés. S'ils sont tous simultanément à 0 point de vie ou moins, le système déclenche la condition de victoire et met fin à la *Wave*
- *_EndWave* : rien à faire

Chaque *Wave* possède ses propres comportements qui sont définis par des *Overrides* des différentes méthodes du parent. Certaines méthodes sont communes à toutes les *Waves,* comme le compte à rebours de départ, ainsi que la disparition de tous les agents hostiles lors de la fin d'une *Wave.*

Progression

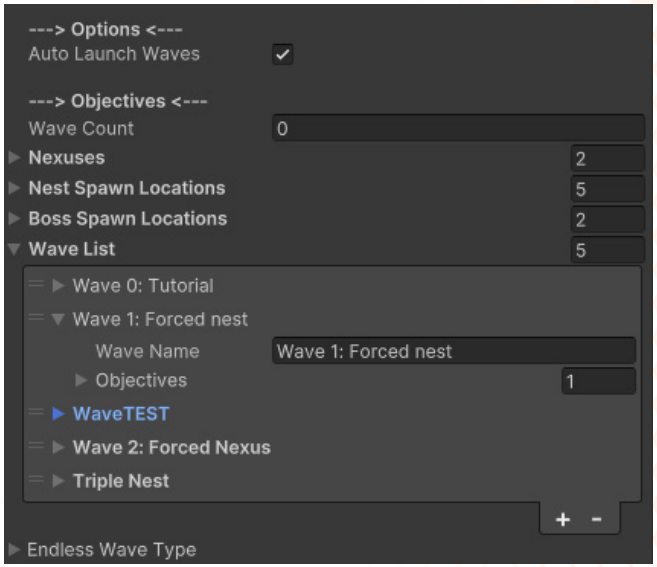
Waves

Les différentes Waves sont paramétrables directement dans leur *ScripableObject* : il est possible d'y régler leur type, ainsi que le Spawn passifs des entités, en y référant des points de Spawn dans la scène ainsi que des agents hostiles à faire apparaître.

L'organisation d'une Run est gérée par un tableau situé dans l'*ObjectiveManager,* dont chacune des cases correspond à l'ensemble des *Waves* pouvant être lancées à cet index.

Cela permet d'avoir un contrôle bien plus précis qu'une génération procédurale, sur la variation de *Gameplay* au sein d'une *Run* et sur la progression de la difficulté des *Waves.*

Inspecteur Objective Manager



Upgrades

Les phases d'*Upgrade* mettent le jeu en pause à l'aide de la méthode Time.Scale. Les cartes d'Upgrades sont instanciées à l'aide de valeurs enregistrées dans un *ScriptableObject* et suivant la structure d'un Prefab de carte.

Les cartes sont tirées au hasard parmi un *Pool* de cartes défini par la rareté de celles-ci. Si aucune carte n'est disponible dans un tier de rareté, le système prend automatiquement dans celui inférieur.

De plus, le système retient les cartes précédemment tirées afin de respecter certaines règles :

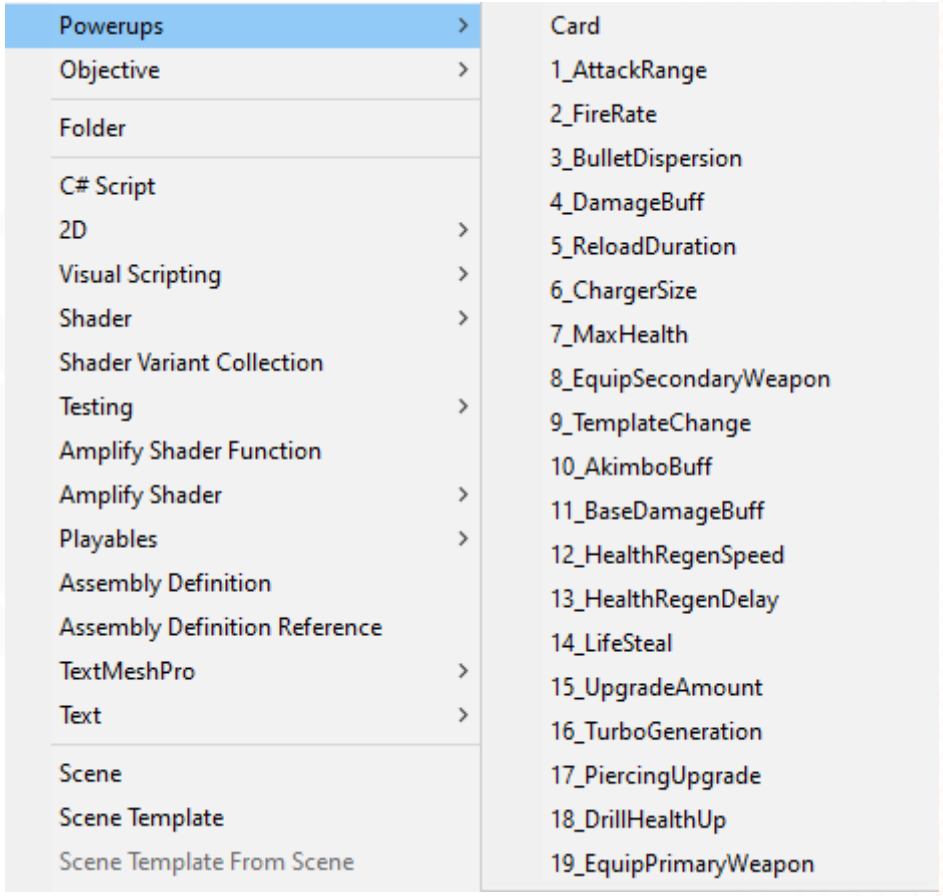
- Une même carte ne peut apparaître qu'en un seul exemplaire dans un tirage
- Certaines cartes ont un nombre limité d'occurrence à travers une *Run*

Scoring & Fin de Partie

Un système d'évaluation de performance du joueur est implémenté dans le jeu. Il récupère un ensemble de données durant la *Run* du joueur, et le stocke dans le *SceneManager*.

Lorsque le joueur perd, il est amené vers le menu de fin de jeu, dans lequel les valeurs stockées sont affichées, afin de donner un *Feedback* au joueur et augmenter la rejouabilité.

Upgrade disponible en Scriptable Object



D.A.V
Direction Artistique Visuelle

Direction artistique Visuelle

Pitch

Dans Caverners, le joueur contrôle un drone de défense humanoïde doté de capacités surhumaines et étant équipé d'armes à feu. Ce drone est envoyé en missions dans les souterrains d'une planète extra-terrestre, afin d'éradiquer les créatures qui l'occupent et protéger les installations humaines récupérant des minéraux précieux.

Intentions Générales

Les mécaniques surhumaines nous ont induit une DAV Science fiction.

La science-fiction est un genre narratif, principalement littéraire (littérature et bande dessinée), cinématographique et vidéoludique créé en 1927.

Il nous faut créer un univers sci-fi cohérent tout en étant dans des codes un peu différents en termes de couleurs avec des couleurs chaudes

Les mécaniques étant pseudo-humanoïdes, l'idée d'un joueur qui contrôlerait un drone humanoïde à distance nous a plu. Cela donne du contexte au jeu, en plus d'être légèrement méta. Ce contexte méta nous a également fait naître l'intention suivante : nous voulions qu'un maximum de mécaniques, feedbacks et signes soient intra-diégétiques, afin que la barrière entre le joueur qui joue au jeu, et le personnage qui contrôle le drone soit la plus fine possible.

Nous avons donc décidé de rendre l'expérience la plus immersive possible, tout en lui donnant un sens, et cette astuce explique également la rejouabilité de notre jeu.

Pour ce qui est l'interface , pour coller à notre pitch elle sera très robotique en affichant plein de données utiles au gameplay.

Logo Caverners

Inspirations

Dans un souci de production, nous avons opté pour une direction en Low-Polys ainsi que des textures constituées d'aplats de couleurs.

Notre inspiration majeure pour la création d'Assets a été Deep Rock Galactic.

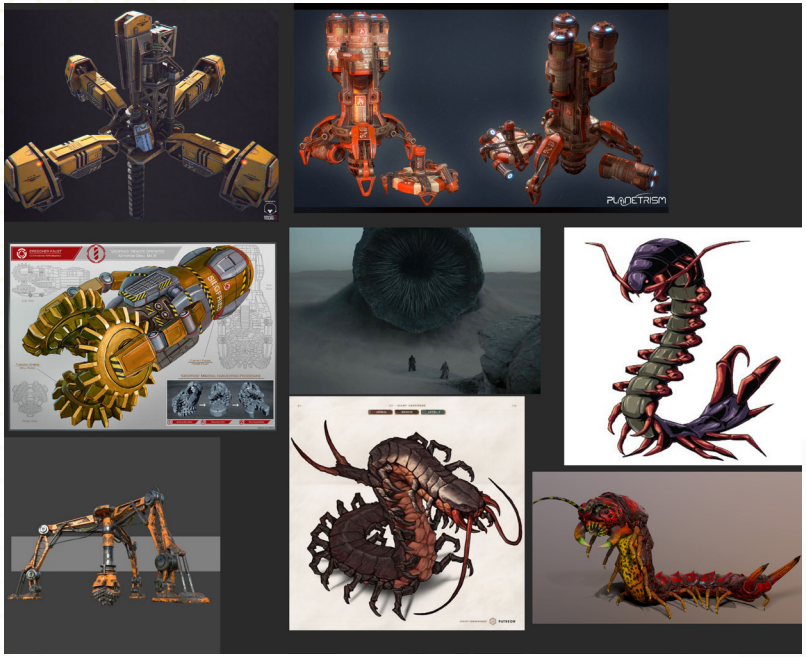
Pour ce qui est de l'UI, nous nous sommes inspirés de différents FPS pour ce qui est de la répartition des informations dans l'écran et pour les différents signes et feedbacks.

Nos références majeures pour simuler l'effet "pilote de drone humanoïde" ont été Metroid et l'armure de Tony Stark dans Iron Man.

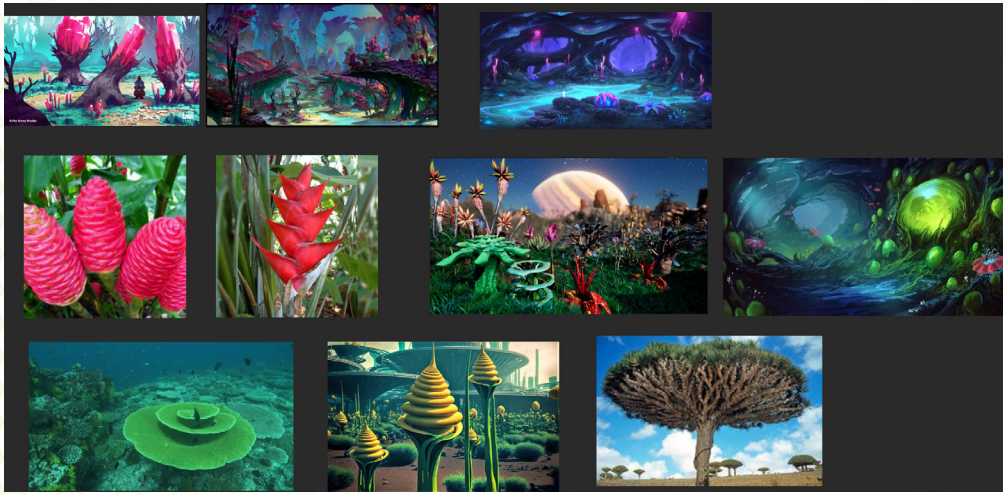
Deep Rock Galactic

Metroid Prime Remastered

Mood Board des objectifs

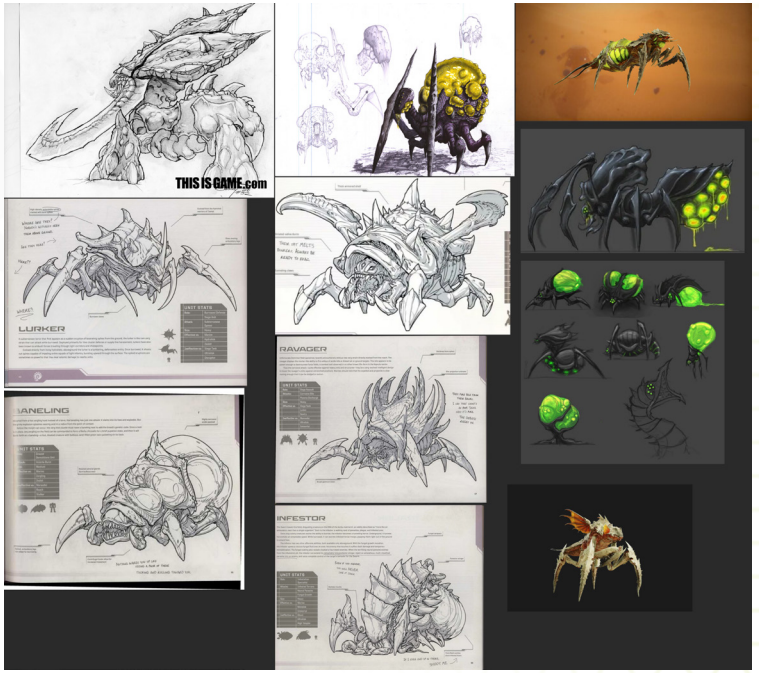


Mood Board Végétation Global et Alien

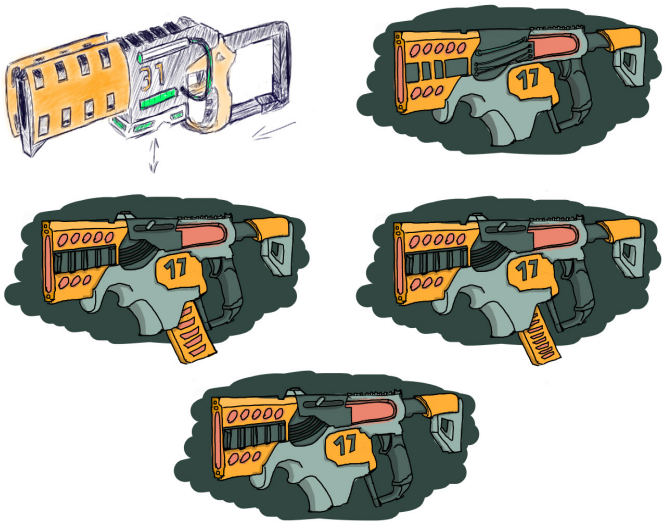


Mood Board Environement Global

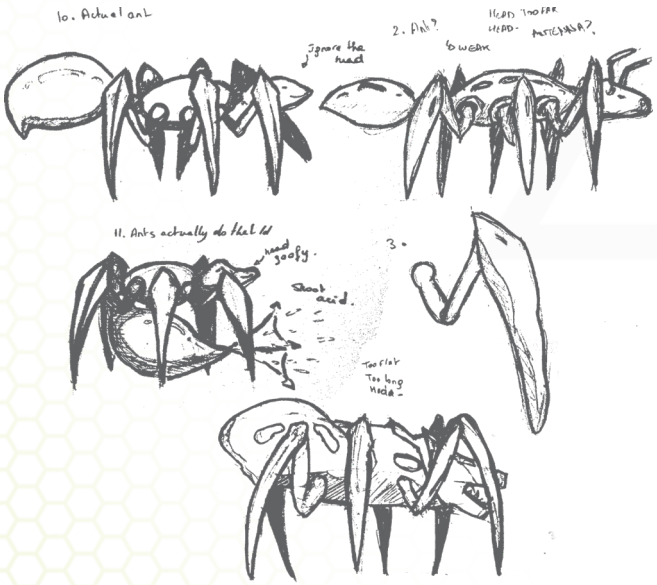
Mood Board Végétation Global et Alien



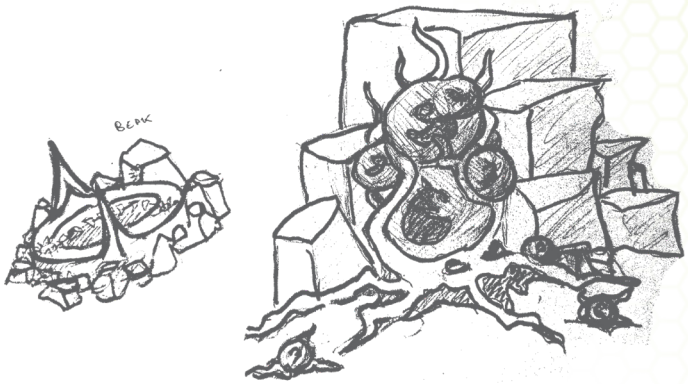
Concept Arts de l'arme principale



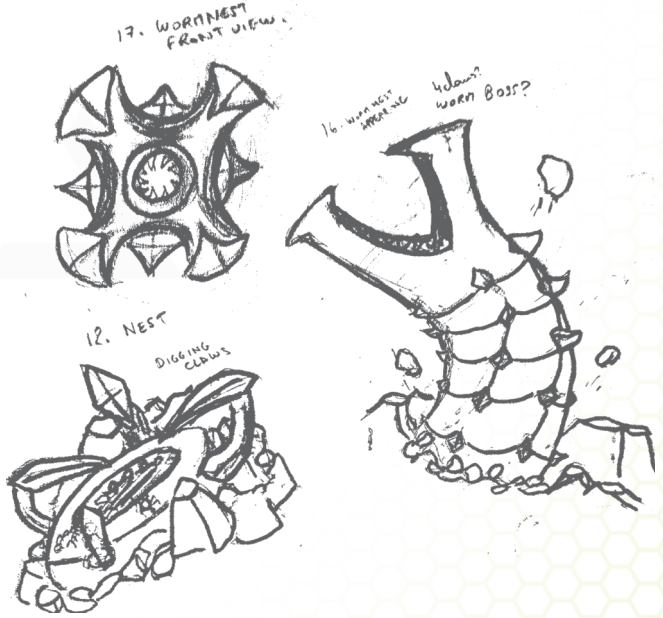
Concept Arts Forme des Mobs



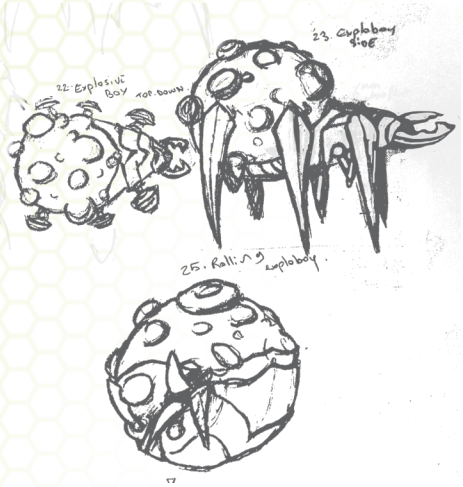
Concept Arts Eggs

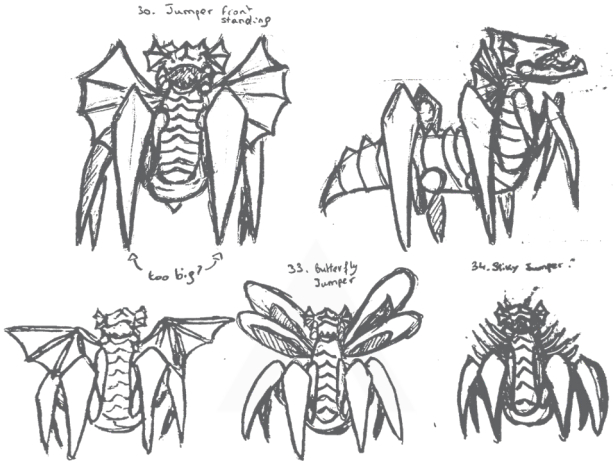


Concept Arts Nest

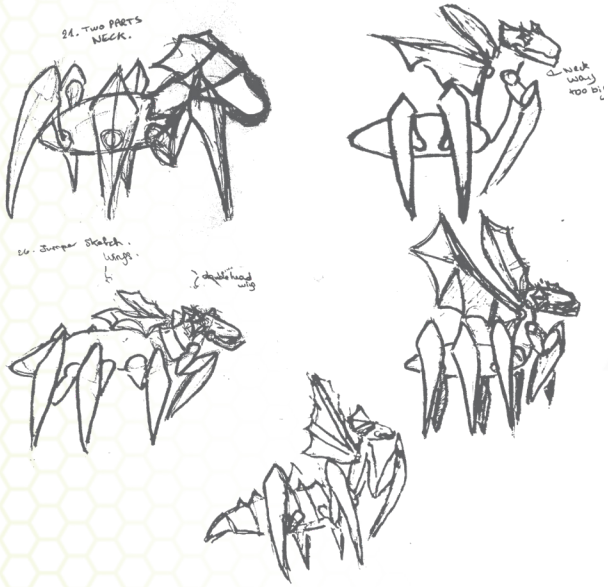


Concept Art Exploder

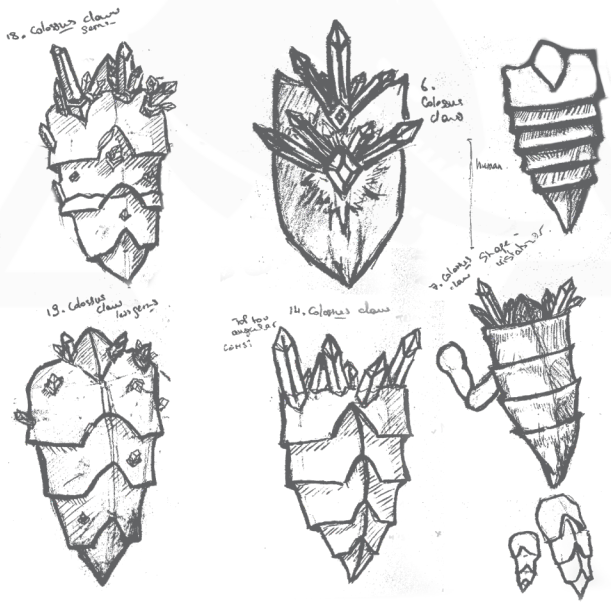


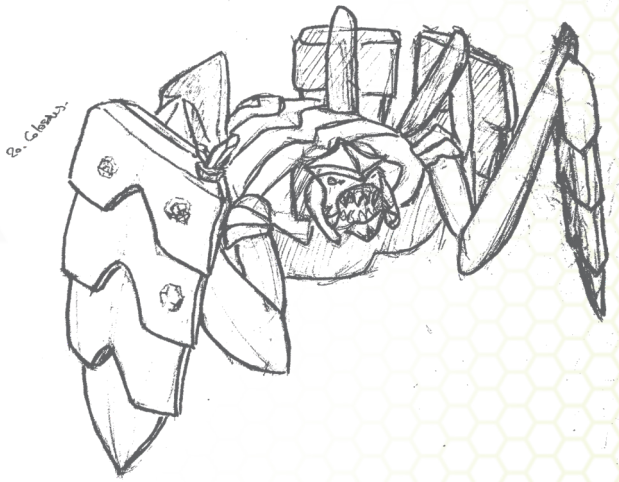


Concept Arts Jumper



Concept Arts Collose





Itérations

Itération Direction Artistique visuelle

Afin de créer l'univers qui nous convenait, nous sommes passés par beaucoup d'itérations. Il nous fallait un monde qui exprime la confrontation entre l civilisation humaine et l'espèce extra-terrestre, qui soit signifiée autant par les formes que les couleurs.

Iteration 1

Dans cette itération, nous souhaitions trouver la couleur des cristaux qui occupent la grotte, ainsi que créer un début d'environnement à l'aide de la dizaine d'Assets que nous avions. A ce stade, nous trouvions les rochers et la grotte trop peu naturels.

Itération 2 :

Dans cette itération, bien plus proche de la version finale, nous nous sommes concentrés sur le Lightning et la disposition des Assets sur la Map, dont le Level Design avait été établi. Les formes et tailles des Assets étant trop variés et trop peu régulés, nous sommes repartis à 0 lors de la disposition des Assets.

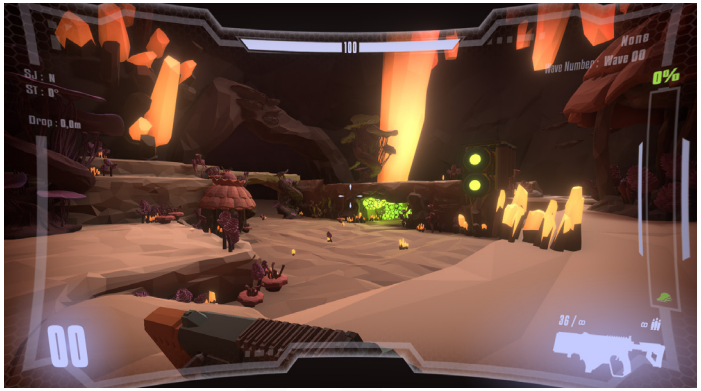
Itération 3 (finale)

Voici la version finale du jeu.

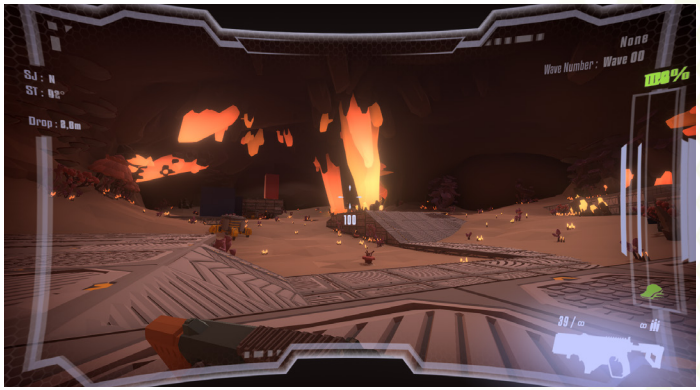
Les Assets ont été répartis selon des règles intrinsèques, afin de solidifier la cohérence de l'univers :

- Les Assets humains sont très droits et carrés
- Les Assets extra-terrestres sont organiques et courbés
- Un faux écosystème a été créé : certaines plantes ne poussent qu'à certains endroits, ou sur certains rochers, etc.
- Les Assets humain expriment de la fonctionnalité, avec des motifs répétables et des structures modulaires
- Les Assets extra-terrestres expriment de l'étrange, avec des anatomies biologiques inspirées de mélange de plantes (Une plante aquatique et une plante terrestre, ou deux végétaux du même type : champignons, arbustes)

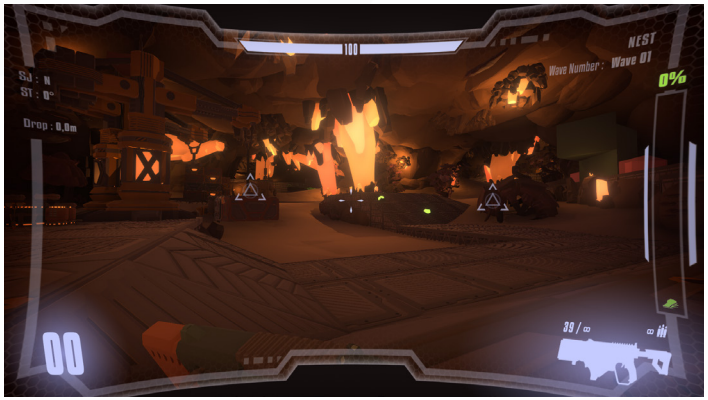
Itération 1



Itération 2



Itération 2



Contraintes Gameplay

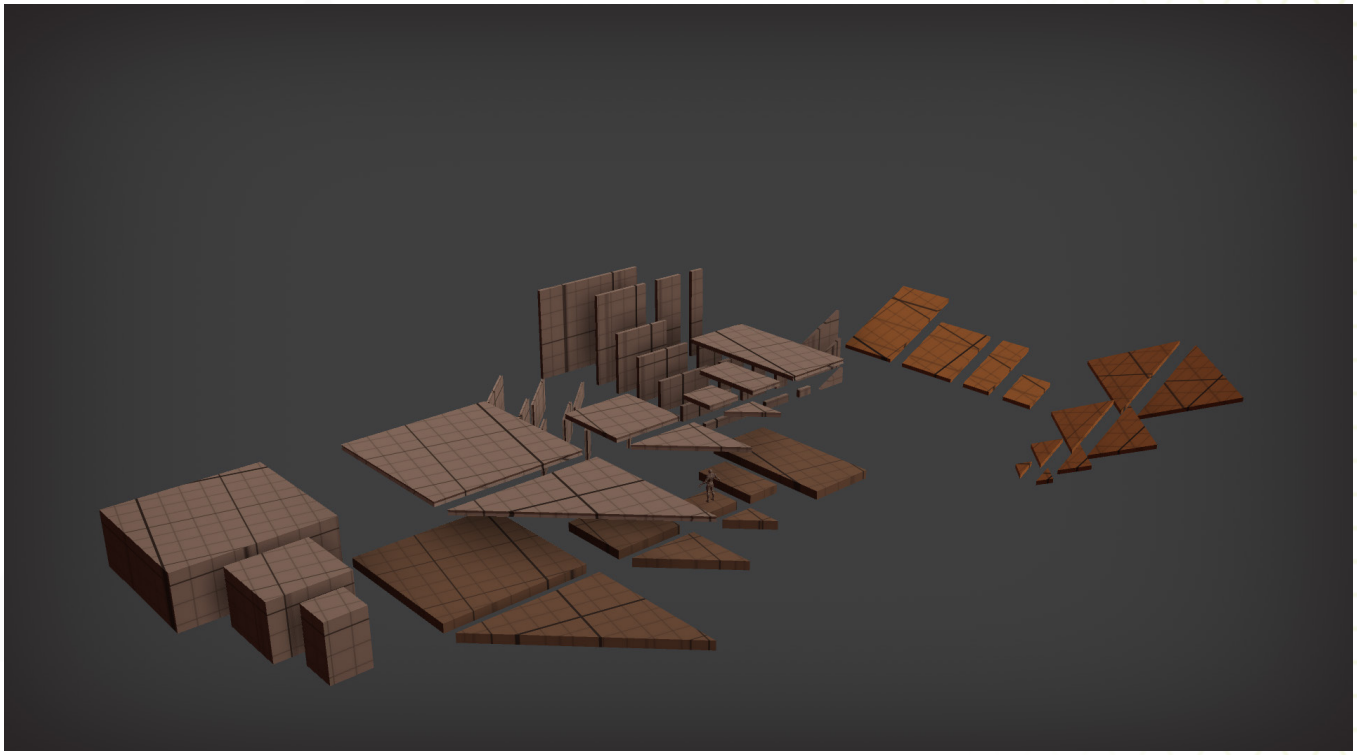
Nous nous sommes heurtés à de nombreuses contraintes de Gameplay, notamment vis-à-vis du Level Design.

Nous avons notamment conçu un Modular Set afin de pouvoir itérer rapidement en Level Design, et chaque élément de l'ensemble possède une HitBox propre et simplifiée afin de fournir de meilleures collisions et un Bake du NavMesh le plus net possible (Pathfinding).

Les autres Assets possèdent une HitBox dépendante de leur taille :

- S'ils sont plus grands que le personnage, leur HitBox est prédéterminée par le Level Design
- S'ils sont plus petits que le personnage, ils ne possèdent pas de HitBox

Modular set



Visuels

Afin de gagner en productivité, nous avons établi une liste d'Assets à produire et avons établi un Scope.

On retrouve sur cette liste les différents noms des Assets, s'ils ont des variations, les références utilisées ainsi que l'étape de production dans laquelle il se trouve.

Liste: https://docs.google.com/spreadsheets/d/1SmHC3q9zq7PXpGur7PnKEngh_MmeeIAL4PrgWjOA4sQ/edit?usp=sharing

Monde Extra-Terrestre

Le monde extra-terrestre correspond à la planète.

Les différents Assets végétaux ont été créés en combinant des plantes différentes afin de créer des anatomie rappelant des végétaux, tout en restant suffisamment éloigné de la vie sur Terre.

L'entièreté des Assets de ce monde ont été pensés avec pour but de pouvoir facilement les disposer dans n'importe quelle salle du Level Design, et déclinés en plusieurs variations afin d'éviter la répétitivité, et d'ancrer solidement la richesse de l'écosystème de la planète, pour favoriser l'immersion.

Monde Humain

Les Assets du monde humain ont été créés afin de pouvoir remplacer le Blocking mis en place par modules par l'équipe de Level Design. L'envie que le monde humain se démarque (puisqu'il est étranger à la planète) a été accentué via l'utilisation de couleurs plus froides que le monde global.

Personnage

Les Assets liés au personnage (Bras, Mains, Armes) ont été créés en suivant une ligne directrice de cohérence globale. Ainsi, ils utilisent comme munition des cristaux que l'on trouve dans la grotte, mais également du Turbo, que l'on récupère en détruisant des agents hostiles ou leurs œufs.

Ces liens sont signifiés par une palette de couleur bien distincte, et des animations liées. Cela permet de renforcer la cohérence de notre univers, en plus d'expliquer intradiégétiquement l'amélioration de notre drone.

Tous ces Assets ont été Riggés avant d'être envoyés à l'équipe d'animation, afin de pouvoir être correctement animés dans Unity. Certains Rigs sont plus complexes que d'autre, dépendamment des besoin d'animation.

Mobs

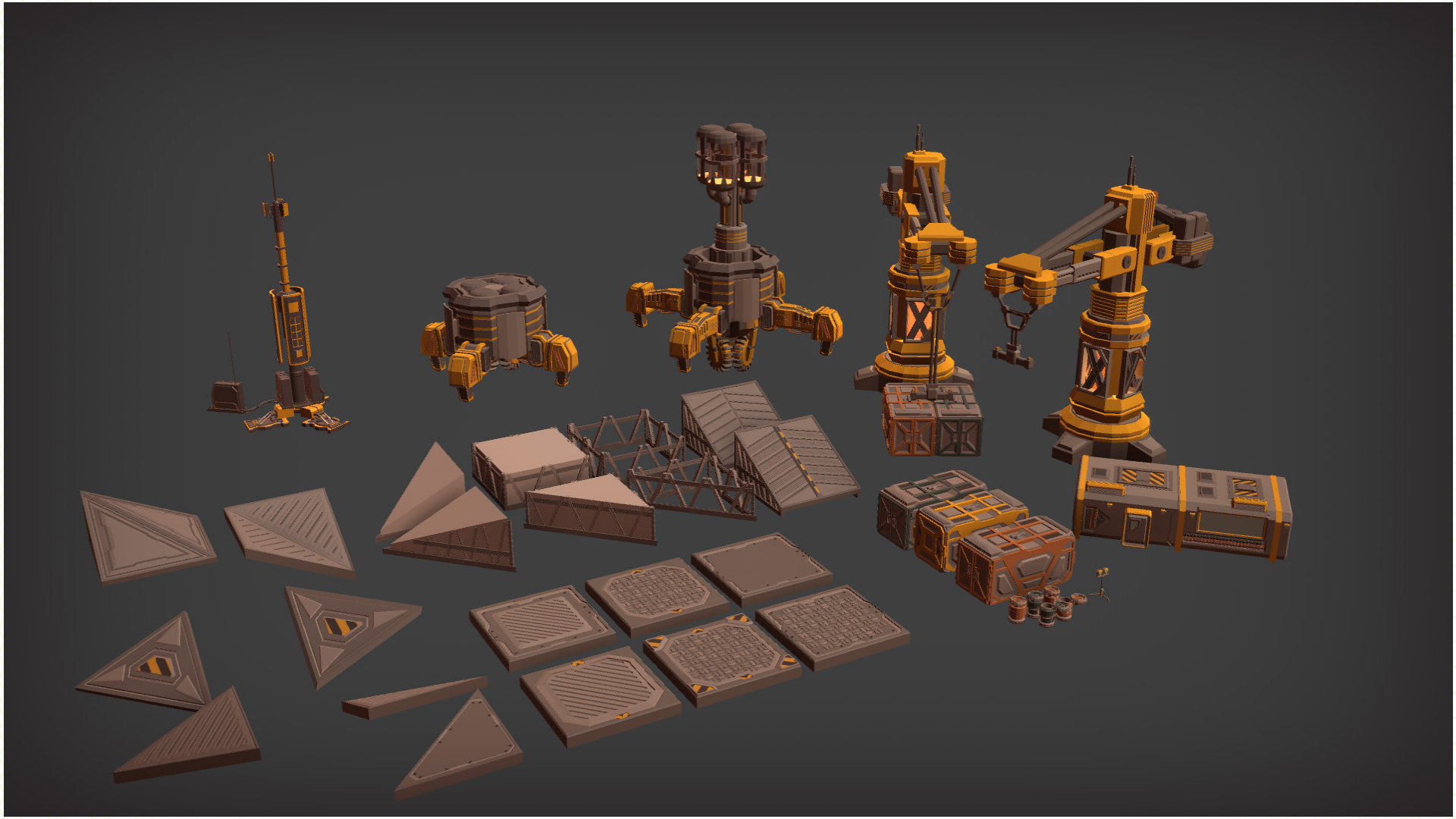
Les agents hostiles ont été conçu via des Concept Art en prenant en référence des reptiles et des insectes de toutes catégories, que nous avons ensuite combiné. Nous avons ensuite créé des variations, pour pouvoir proposer une plus grande diversité d'ennemis en jeu.

Tableau d'asset pour la production

Nom des Objets	Image Références	Variation entre 3 et 6	Topo	Textures	Validation Textures	Intégrés	Autre Références	Autre Références
World Global								
Champignon mural	https://c8.alamy.com/compfr	☑						
Herbe roche lumineuse	https://i.pinimg.com/564x/36	☑						
Plante moyenne 1	https://encrypted-tbn0.gstati	☑						
Arbre	https://www.deco.fr/sites/def	☐						
Amat Roche avec peu de cristaux et sans	https://i.pinimg.com/564x/76	☑						
Roche lumineuse Plafond	https://i.pinimg.com/564x/0e	☑					https://i.pinimg.com/564x/e7/da/83/	
Plante moyenne 2	https://i.pinimg.com/564x/b0	☑						
Plante Grande	https://i.pinimg.com/564x/e9	☐					https://jardin-eternel.fr/cdn/shop/pro	https://media.istockphoto.com/id/6
Plante Moyenne 3	https://i.pinimg.com/564x/a2	☑						
Plante Petite		☑						
Alien World								
Plante Grande	https://i.pinimg.com/564x/89	☐						
Autre Plante Moyenne	https://i.pinimg.com/564x/2b	☐						
Petite Plante	https://i.pinimg.com/564x/72	☐						
Jumper Mobs		☑						
Collosse Mobs		☐						
Nest Mobs		☐						
Plante moyen	https://i.pinimg.com/564x/18	☑						
Oeufs D'Alien		☑						
Human World								
Sol en metal		☑						
Baril	https://preview.free3d.com/i	☑						
Antenne Radio	https://i.pinimg.com/474x/97	☐					https://i.pinimg.com/474x/4b/02/f5/4	
Lampe industrielle sur pieds	https://i.pinimg.com/564x/a8	☐						
Lampe Industrielle grande	https://www.pinterest.fr/pin/2	☐						
Lampe industrielle Murale		☐						
Nexus		☐						
Contenaire	https://i.pinimg.com/564x/df/	☑						
Grue	https://i.pinimg.com/474x/2e	☑					https://i.pinimg.com/474x/5f/d2/a7/5	
Player								
Bras		☐						
Armes principale		☐						
Lance Rocket		☐						
Téléportation Gun		☐						

Asset du monde global présent dans le jeu

Assets du monde Humain présent dans le jeu avec son kit de Modular Design



Assets des armes et des bras de notre avatar

Assets des différents mobs présent dans le jeu

Visuels 2D

Inspirations

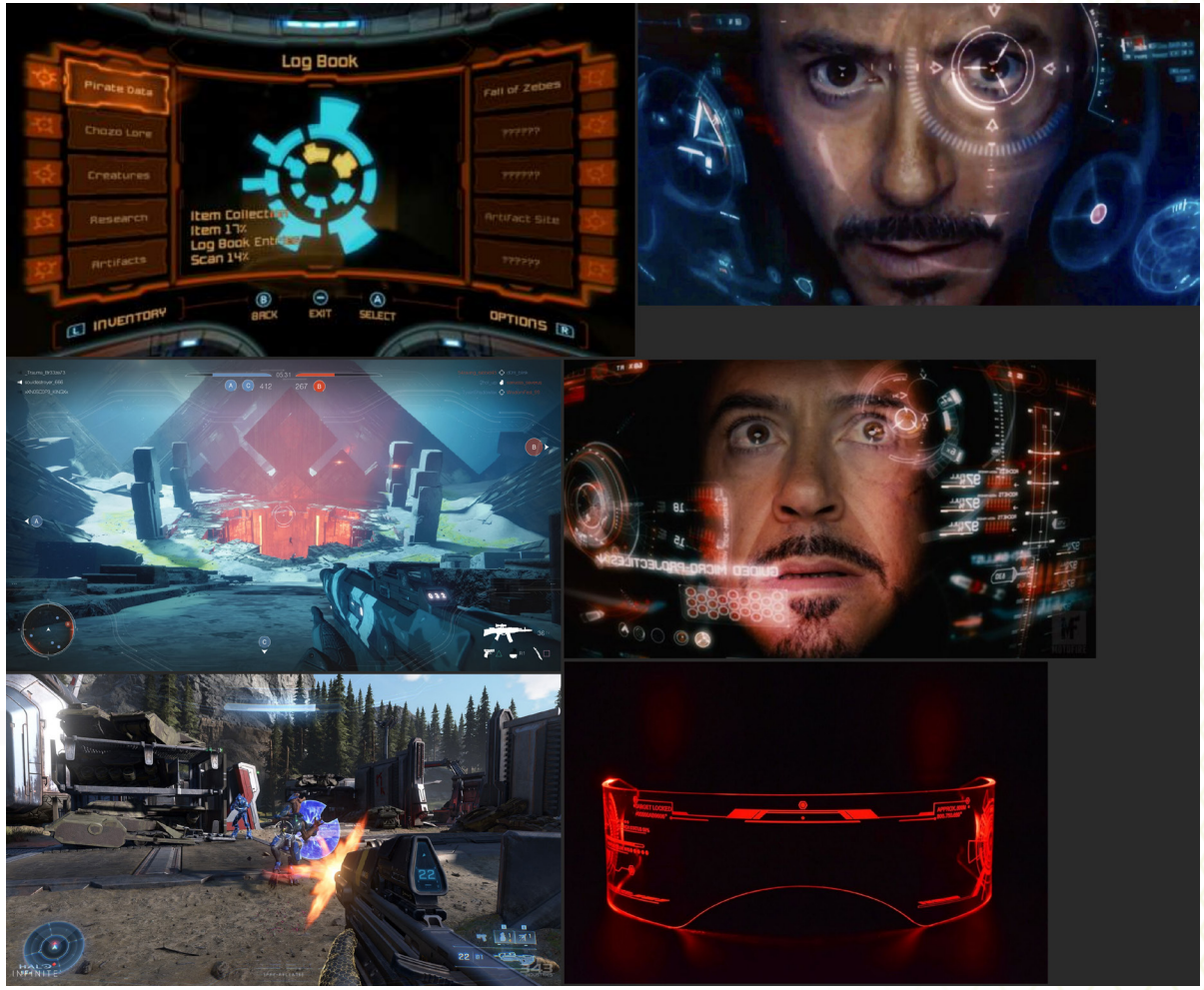
Notre idée principale était de recréer une interface robotique d'un drone de haute technologie. Nous avons utilisé pour ça plusieurs inspirations, telles que les interfaces des armures d'Iron Man et de Samus dans Metroid, ou encore les lunettes futuristes de Tron.

En ce qui concerne la répartition des informations à l'écran ainsi que des différents signes et Feedbacks, nous avons pris pour références des FPS classiques du genre (Call of Duty, Destiny, Metroid).

Moodboards

Moodboard UI

Mood Board Interface

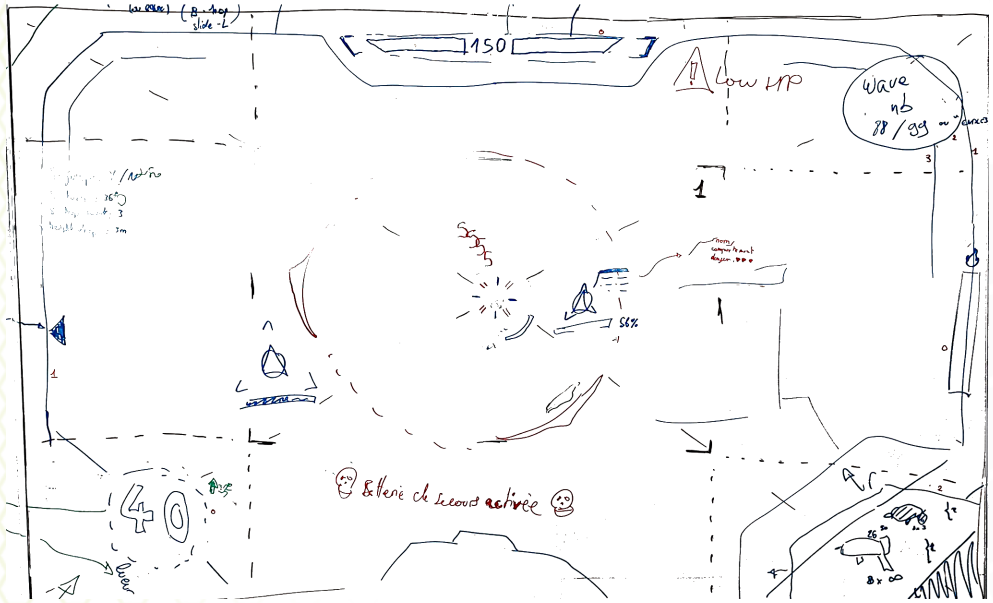


Itérations

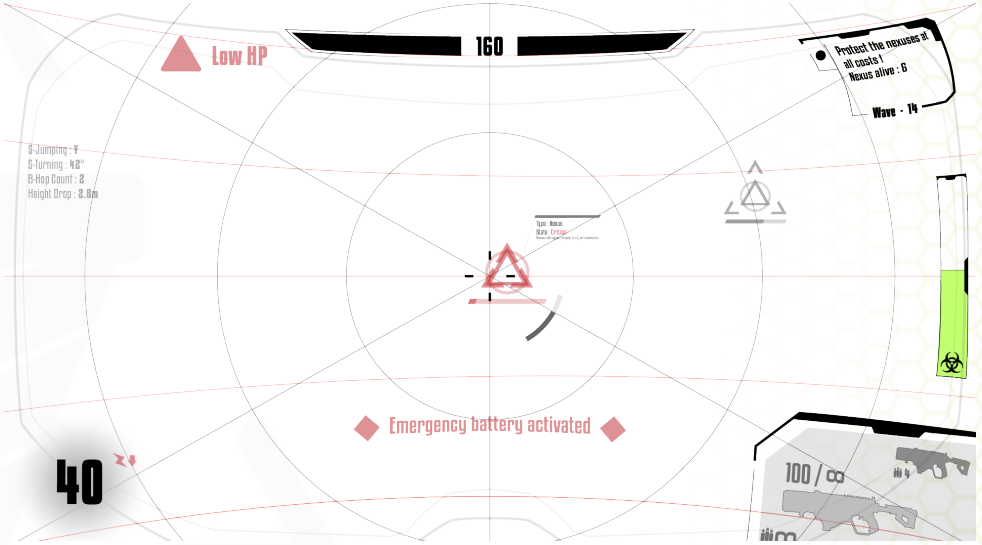
Iteration 1

Dans cette itération, nous avons tout d'abord commencé par faire un schéma de la répartition de nos informations, en gardant à l'esprit une idée des différents Feedbacks nécessaires.

Brainstorming Interface



Blocking Interface Illustrator



Iteration 2

Dans cette itération, nous avons voulu tester l'aspect « visière de casque » en créant un effet de distorsion sur les différents éléments de l'UI. Nous avons donc manuellement conçu une grille de distorsion sur Illustrator, afin de pouvoir faire des tests.

Iteration 3 (Non finale)

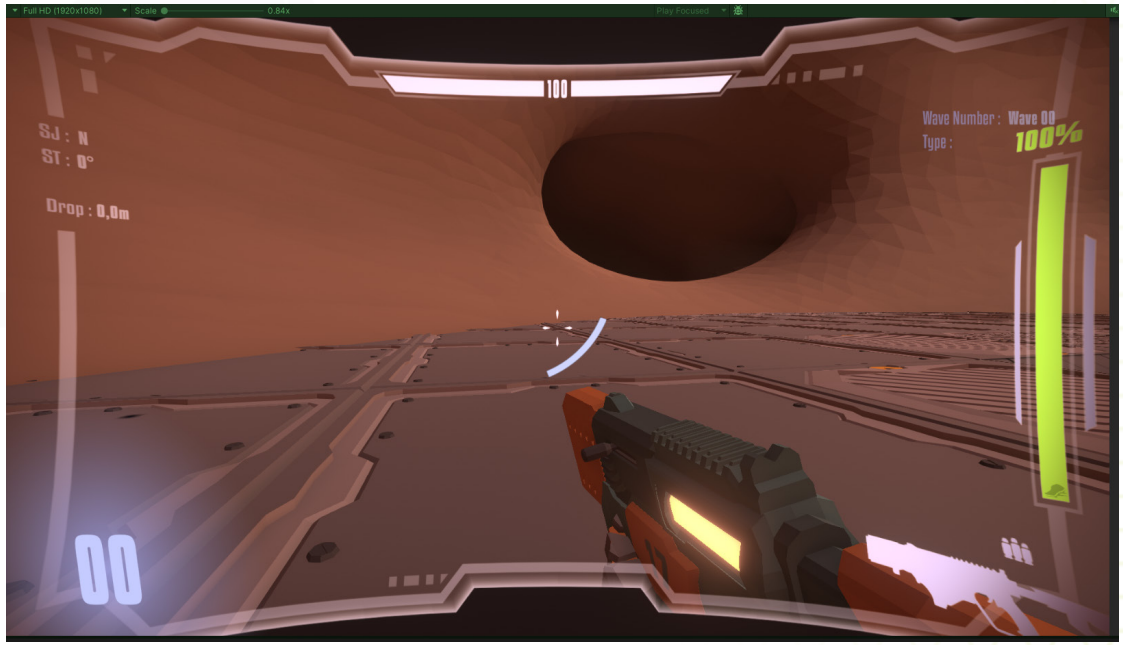
Dans cette itération, qui est celle présente en jeu, nous avons revu la mécanique de distorsion, en la calculant directement dans le moteur de jeu.

Ici, nous utilisons une projection sur un plan courbé, en passant par un Render Texture.

Nous souhaiterions travailler encore cette interface, afin d'évoluer vers une version plus raffinée et subtile, qui se rapproche moins d'un Blocking Illustrator.

Interface dans Unity avant projection sur le plan

Interface après projection



Contraintes Gameplay

La seule contrainte Gameplay que nous avons rencontré était la lisibilité et la disponibilité des informations. Il était extrêmement important vis-à-vis du genre de notre jeu, que les signes et Feedbacks procurés par l'UI soient précis, sans surcharger cognitivement le joueur et tout en restant intradiégétiques.

Nous en avons fait une force, en décidant de reprendre les codes des interfaces robotisées et de nous en servir comme Feedbacks pour le joueur : Glow de la visière, texture hexagonale, valeurs numériques évolutives, etc.).

Visuels

UI

Les Assets de l'UI ont été produits sur Illustrator, en prenant en compte la courbure appliquée par le Post-Processing. Tous les Assets ont été exportés en blanc pur 100% d'opacité, après avoir fait des tests de couleurs directement sur Illustrator.

Nous avons utilisé la palette de couleurs trichromique afin d'en sortir des nuances de bleus qui seraient en accord avec le reste du projet, tout en se démarquant des différents objets de la scène.

Il était primordial que l'UI se détache du reste, afin d'assurer une bonne lisibilité, notamment au niveau du Crosshair.

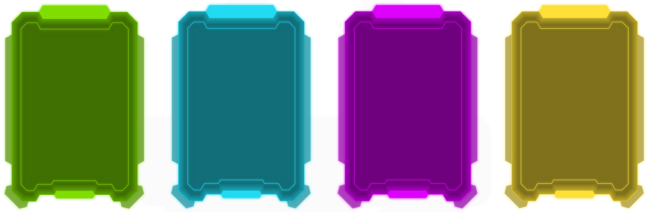
Nous avons également décidé de reprendre le vert des créatures extra-terrestres pour le Turbo, afin d'assurer une bonne affordance.

Upgrades

Les icônes d'Upgrades ont été travaillés afin d'être clairs et toujours structurés de la même manière : un icône principal, un symbole d'augmentation ou d'incrémentation, ainsi que des variations / combinaisons de symboles.

Nous avons également repris les codes couleur des raretés d'objets que l'on peut trouver dans les différents jeux possédant des tiers d'objet :

- Commun = Vert
- Rare = Bleu
- Epique = Violet
- Légendaire = Doré



Les cartes ont également été pensées afin que leur lecture soit la plus fluide possible. Il était important que les informations nécessaires soient dessus et que leur hiérarchie soit respectée. A termes, un joueur aguerri n'aura plus besoin de lire les informations textuelles sur la carte, et l'iconographie suffira à lui indiquer l'Upgrade dont il est question.

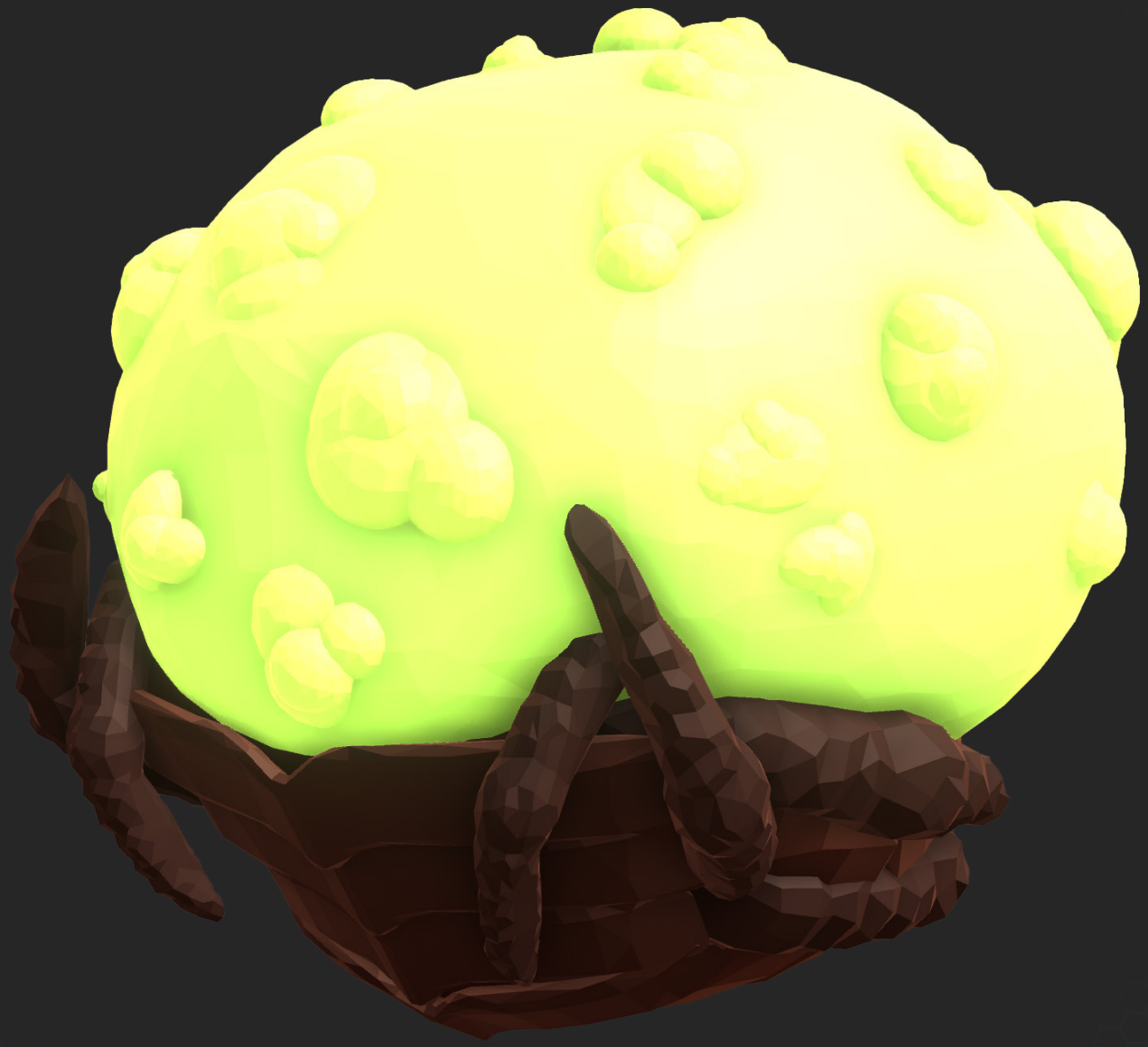
Menus

Nous avons repris dans les menus la thématique des interfaces robotisées, afin de faire une transition entre le menu de départ et la scène de jeu.

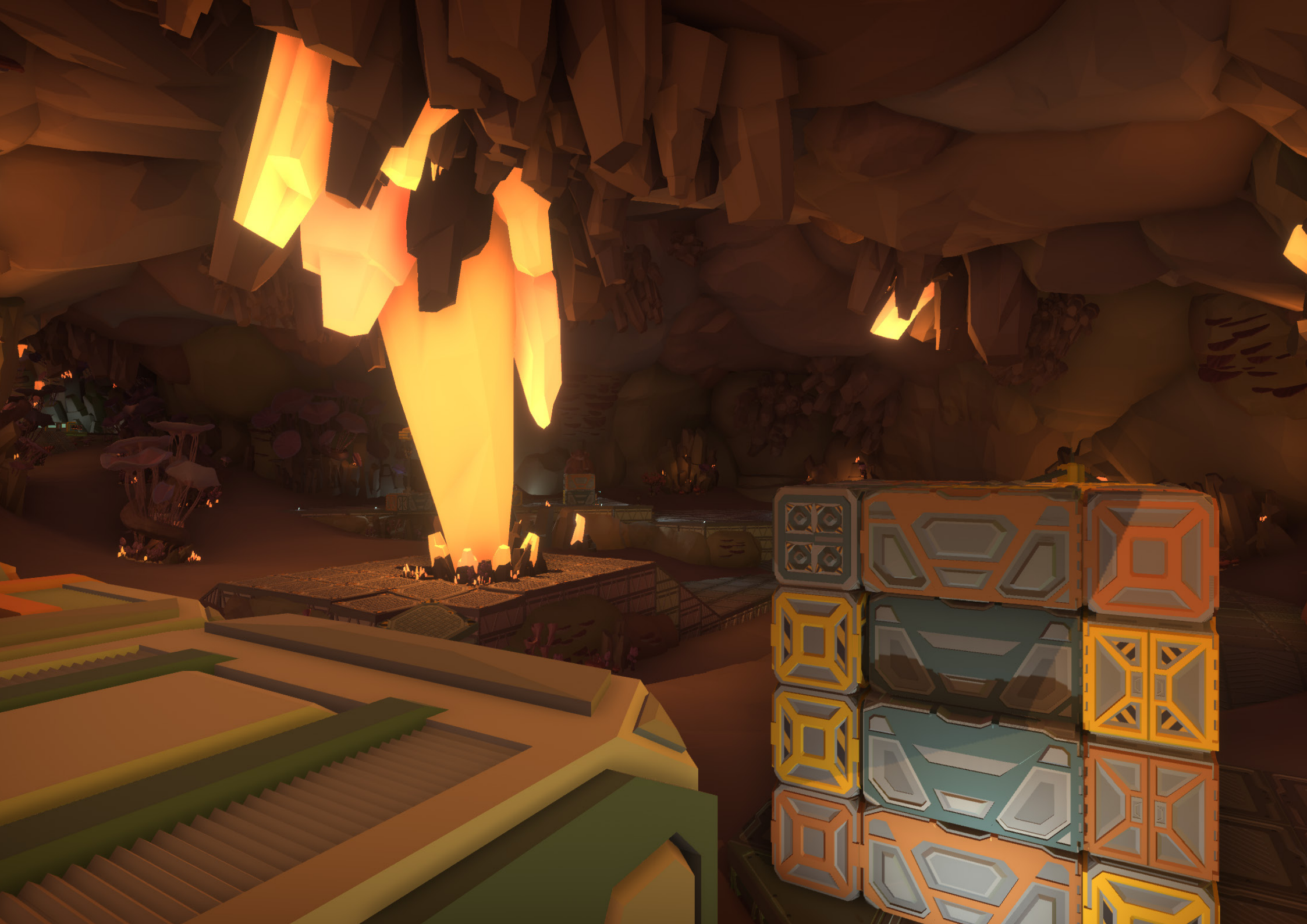
Cela permet de renforcer l'immersion et de continuer à jouer autour de cette direction artistique méta, qui met le joueur dans la peau du protagoniste.

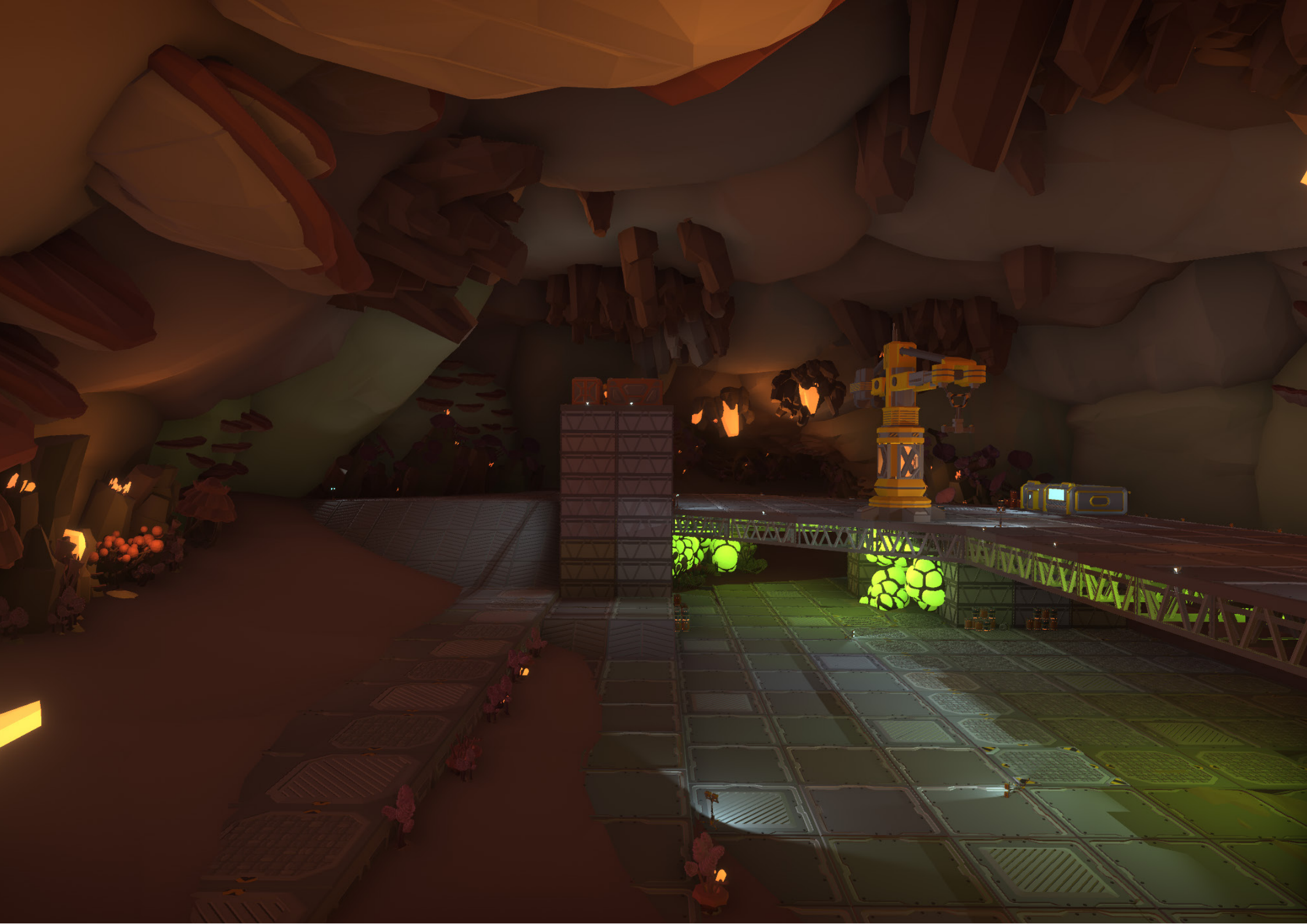
Interface finale présente dans le jeu

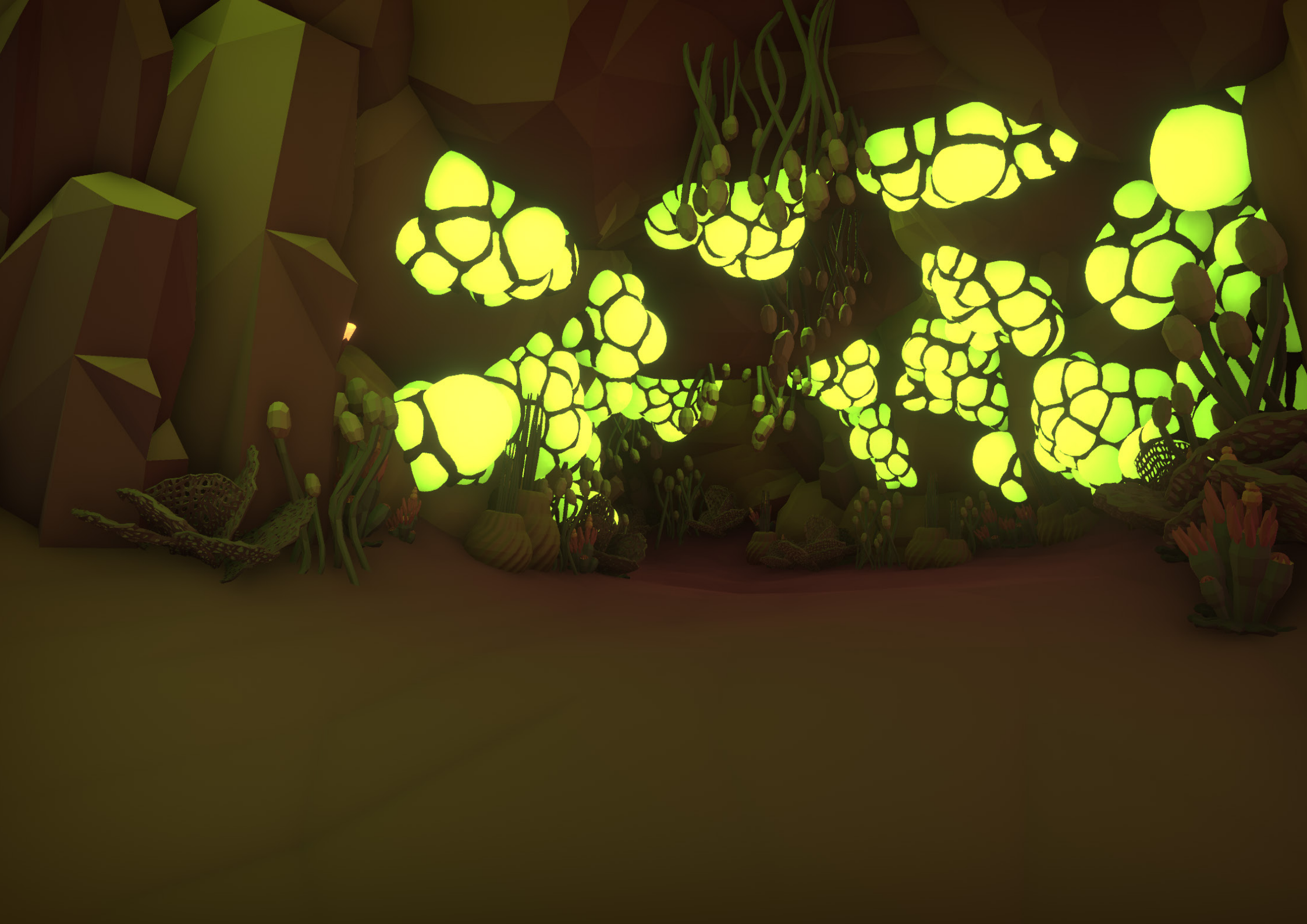
80

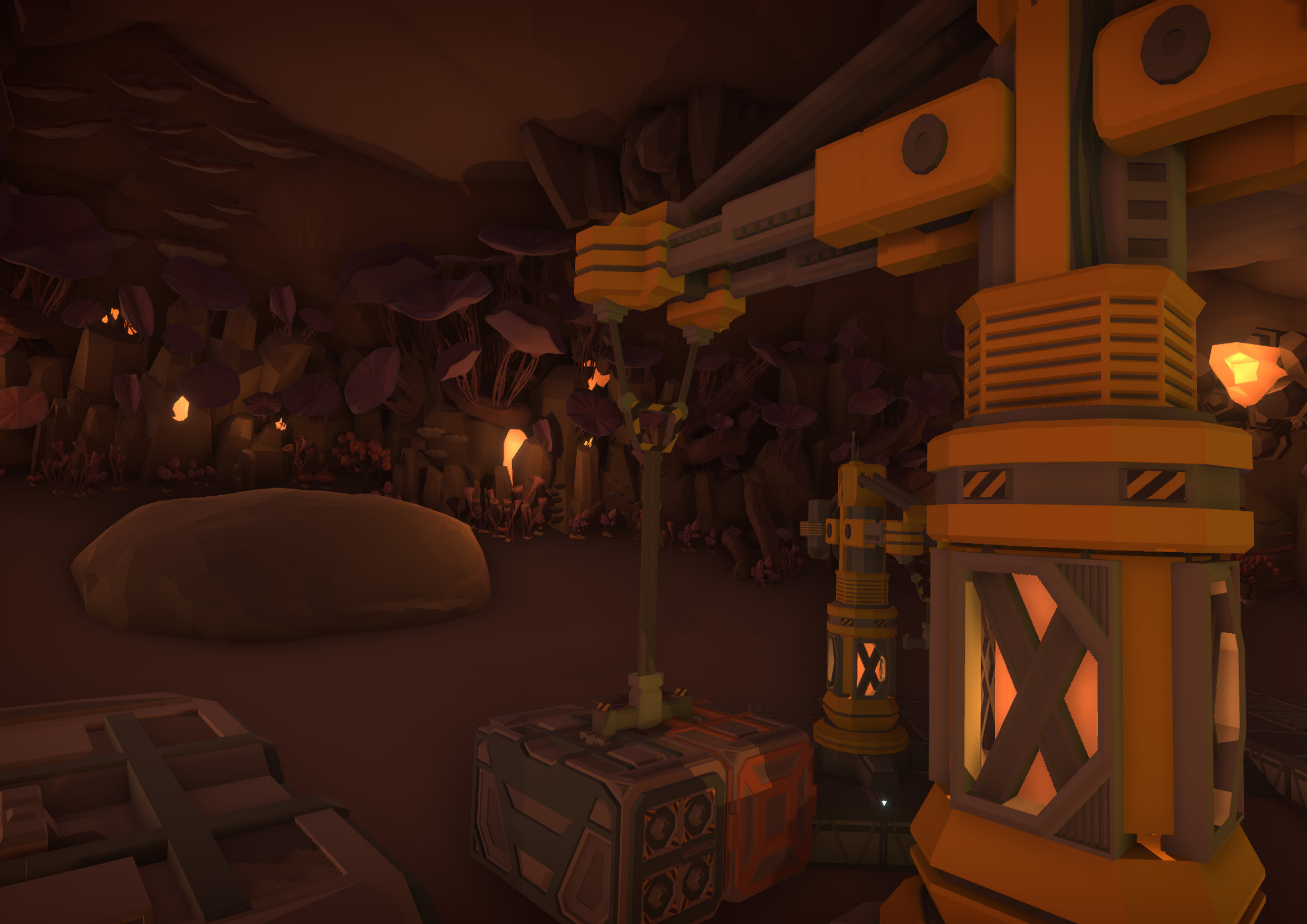


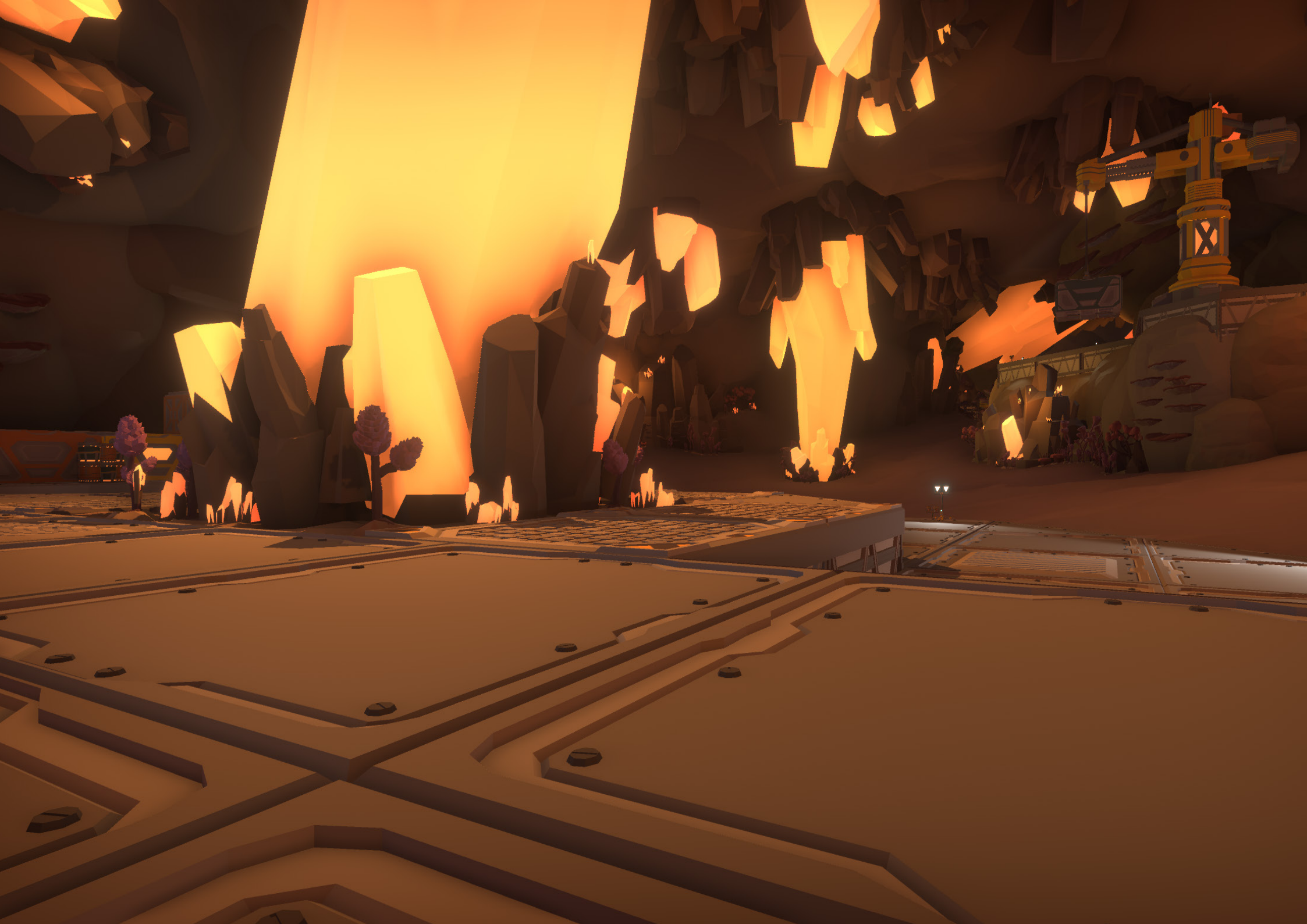
17

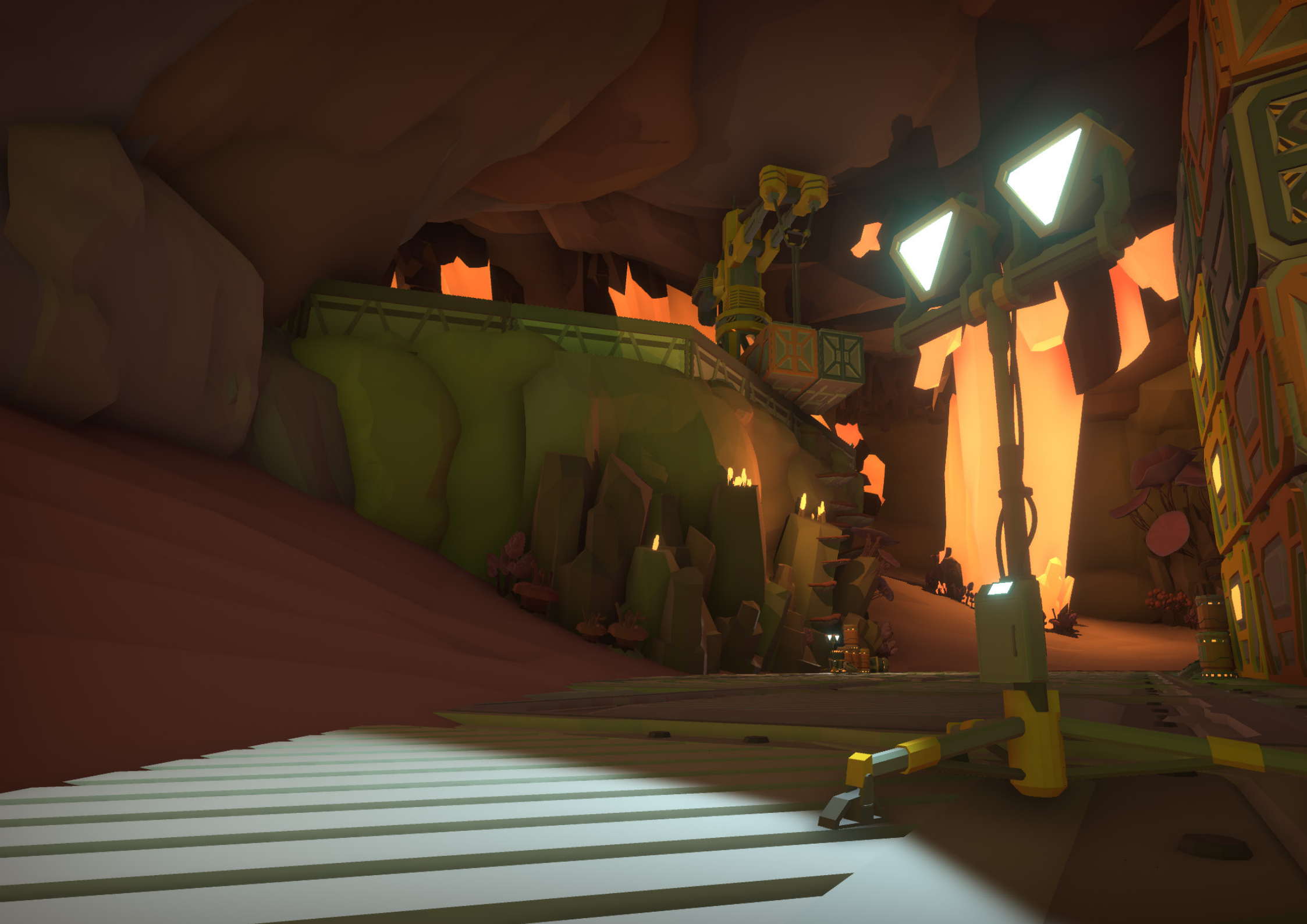












D.A.S
Direction Artistique Sonore

Direction Artistique Sonore

Sound Effects (SFX)

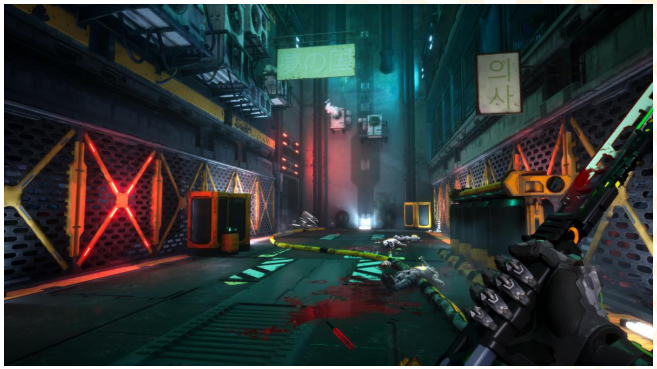
Inspirations & intentions

Les SFX représentant majoritairement des Feedbacks pour le joueur, nous nous sommes inspirés de jeu de genre similaires à notre projet. Nos inspirations majeures ont été Apex Legend, Call of Duty : Warzone, Ghostrunner et TitanFall 2.

Dans un contexte de jeu nerveux et frénétique, le Sound Design prend une place encore plus importante car il demande un travail de Feedbacks important, notamment concernant la réussite et les échecs des différentes actions du joueur.

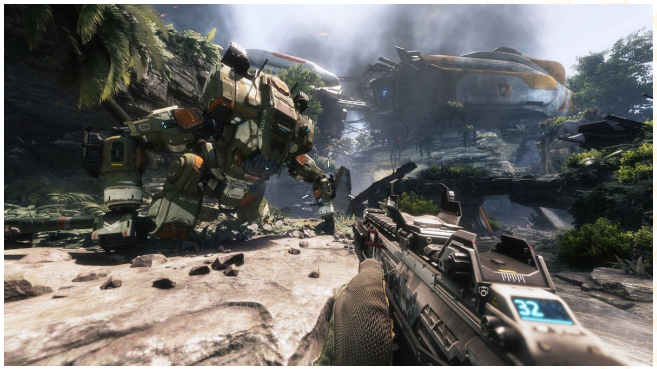
Armored Core VI

GhostRunner 2



Apex Legend (Octane)

TitanFall 2



Call of Duty Warzone

Processus de production

Afin d'établir un Scope réaliste, nous avons mis en place des EventLists correspondant chacune à une grande partie du projet. Il en existe 5 en tout :

- EventList Player : concerne les actions du joueur, les armes, et l'état du personnage
- EventList System : concerne les Waves, les entités servant d'objectifs et les menus
- EventList Entities : concerne toutes les entités
- EventList Musics : concerne les musiques du jeu
- Voicelines : concerne les Voicelines du jeu

Lien : https://docs.google.com/spreadsheets/d/1PUzJiRm06WspeYCsfXoo_-ydWf9cgfsrQjeQky0nLHM/edit?usp=sharing

L'objectif de ces EventLists est de regrouper l'ensemble des évènements sonores du projet, afin de pouvoir retrouver rapidement un élément recherché et de faciliter l'intégration des différents sons par l'équipe de Game Programming. De plus, cela nous permet de suivre l'évolution du travail de l'équipe de Sound Design afin de prendre des décisions plus efficacement en fonction de l'état du système sonore du projet.

QR code Event List Entière

Structure du document

Les 4 premières EventLists suivent cette structure de documentation :

Catégorie de l'Event : regroupe une entité, une phase de jeu, un ensemble de mécaniques du personnage ou de ses armes

Nom de l'Event : correspond à la Feature ou la mécanique concernée

Settings : paramètres ou valeurs in-game devant être récupérées afin de faire évoluer l'Event

Type : mode de lecture de l'Event (Loop, Hold)

Spatialisation 3D : Booléen

Event évolutif : Booléen

Trigger : action qui déclenche l'Event

Description : description de l'Event ou de l'action lorsque c'est nécessaire

- Statut : état de l'intégration de l'Event
- ToDo : encore à produire
- PlaceHolder : son non-réalisé, mais qui possède une instance intégrée dans le projet
- Advanced : son créé et intégré, mais qui demande des réglages de paramètres
- Half-Integrated : son paramétré, en attente de validation
- Integrated : Event entièrement intégré et validé

Priority : Valeur d'importance de l'Event par rapport au projet, niveau de priorité de production

Which Script ? : endroit dans le code où l'Event doit être appelé (Nom du script => Nom de la méthode)

Prog Commentary : si le programmeur doit ajouter des détails à propos de l'intégration de l'Event, à l'attention du Sound Designer

Références : liste des références utilisées, accompagnées d'un lien

Le document de Voicelines suit cette structure de documentation :

Phase : état du système dans lequel est lancé l'Event

Voiceline Drone : Voicelines du drone, suivies de la version traduite en anglais

Voiceline Copilot : Voicelines du co-pilote du personnage, suivi de la version traduite en anglais

Notes du traducteur : détails sur la traduction

Main Voiceline FMOD Name : nom de l'Event FMOD de la Voiceline du co-pilote

Drone Voiceline FMOD Name : nom de l'Event FMOD de la Voiceline du drone

Statut : état de l'intégration de l'Event

Priority : Valeur d'importance de l'Event par rapport au projet, niveau de priorité de production

Prog Commentary : si le programmeur doit ajouter des détails à propos de l'intégration de l'Event, à l'attention du Sound Designer

Voici la structuration du projet FMOD avec les dossiers correspondant respectivement à chaque catégorie de l'EventList. Un code couleur est appliqué aux Events, identique à celui du Statut de l'Event dans l'EventList :

Event List System

Categories	Tittle	Setting	Type	Spatialization 3D	Evolutif / Variations	Trigger	Description	Statut	Priority
Movement	Slide		Loop	No	No	input		Integrated	1
	Jump		One Shot	No	Yes	input	Son lorsque le joueur appuie sur la touche de saut. Se coupe de manière fluide lorqsu'il relâche la touche de saut.	Integrated	1
	SpeedEffect	Speed	Loop	No	Yes	Player speed variation		Placeholder	2
	On Ground	Dirt, Metal, Grass, GravityStrength	One Shot	No	Yes	Player is touching the ground once		Integrated	1
	AccumulatedSpeedGain		One Shot	No	No	Player jump during the bunny hop period or absorbing slide	Son qui s'active lors le player arrive à conserver la vitesse accumulée lors de la chute.	Integrated	3
	JumpReversedSlide	CurrentAngleInDash	One Shot	No	Yes	Player jump during a dash	Son lors du saut pendant un dash. Evolue en fonction de l'angle de rotation du player lors du dash.	Half/Integrated	2
	HeadRotationInJRS	rotationVelocity	Loop	No	Yes	PlayerRotationVelocity	Son lors de la rotation de la caméra lors du dash. Evolue en fonction de la valeur de la rotationVelocity	Integrated	2
	Turbo	TurboState, TurboAmount	Hold	No	Yes	TurboInput	Son en boucle avec ouverture et fermeture.	Integrated	2
	Falling	GravityStrength	Loop	No	Yes	Player falls		Half/Integrated	2
Footsteps	WalkFootsteps	Dirt, Metal, Grass, CurrentSpeed, IsUsingTurbo	One Shot	No	Yes	Moving (Input)		Integrated	1
	TurboFootsteps		One Shot	No	Yes			Integrated	1
Main Weapon	SMGFire	FireState, MagzinePourcentage	One Shot	No	Yes	WeaponFire		Integrated	1
	ShotgunFire	FireState, MagzinePourcentage	One Shot	No	Yes	WeaponFire		Integrated	3
	ThrowingKnifeFire		One Shot	No	No	WeaponFire		To Do	3
	HitMarker		One Shot	No	No			Integrated	3
	CritHitMarker		One Shot	No	No			Integrated	3
	CritMarker		One Shot	No	No			Integrated	3
	SMGReload		One Shot	No	No	ReloadAction		Integrated	1
	ShotgunReload		One Shot	No	No	ReloadAction		Integrated	3
	ThrowingKnifeReload		One Shot	No	No	ReloadAction		To Do	3
	InsertModule		One Shot	No	No			To Do	3
	KillMarker		One Shot	No	No			Integrated	2
	BulletImpact	Dirt, Metal, Grass	One Shot	Yes	No			Integrated	2
Secondary Weapons: Rocket Launcher	RLOpen		One Shot	No	No			Integrated	2
	RLFire		One Shot	No	No			Integrated	1
	RLReload		One Shot	No	No			Integrated	1
	RLImpact		One Shot	Yes	No			Integrated	1
Secondary Weapon: TP Gun	TPGunOpen		One Shot	No	No			To Do	2
	TPGunFire		One Shot	No	No			Placeholder	1
	TPGunReload		One Shot	No	No			To Do	1
	TPGunDeviceOnGround		One Shot	Yes	No			To Do	2
	TPGunUse		One Shot	No	No			To Do	1
Player State	LowHP	snp_LowHP	Loop	No	No	CurrentHP < 30		Integrated	2
	Regeneration		Hold	No	No		Son en boucle avec ouverture et fermeture.	Integrated	3
	TakeHit		One Shot	No	No			Integrated	1
	NoHP		One Shot	No	No		Son lorsque le joueur n'a plus d'HP.	To Do	2
	TakeTurboItem		One Shot	No	No	Trigger => TurboItem		Integrated	3
UI	OnTargetFocused		One Shot	No	No			To Do	3

Event List Player

Categories	Tittle	Setting	Fmod name	Type	Spatialization 3D	Evolutif / Variations	Trigger	Description	Statut	Priority	Which Script ?	Prog Commentary !
System	Boss Wave	Launch		One Shot	No	Yes	Varies with boss difficulty / amount		Integrated	3		
		Boss eliminated		One Shot	No	Yes	Varies with boss difficulty		To Do	2		
		BossDeath	BossDeath	One Shot	Yes	No			To Do	2		
	Nest Wave	Launch	NT_Launch	One Shot	No	Yes	Varies with nest amount / wave difficulty		Integrated	3		
	Nexus Wave	Launch	NS_Launch	One Shot	No	Yes	Varies with wave difficulty		Integrated	3		
		Nexus On Action	NexusOpen	Loop	Yes	No			Advanced	2		
		Nexus is under attack	NexusUnderAttack	One Shot	Yes	No	Played on player towards nexus		Integrated	1		
		Nexus has low hp	NexusLowHP	One Shot	Yes	No			To Do	2		
		Nexus destroyed	NexusDestroyed	One Shot	Yes	No	Played on player towards nexus		Integrated	1		
		Timer is below threshhold	TimerBelowThreshold	Loop	No	No	Played each second		Integrated	2		
		X seconds remaining	XSecondsRemaining	Loop	No	No	Played each second		Integrated	1		
		Nexus desactivation	NexusClose	One Shot	Yes	No			To Do	2		
	Upgrade Phase	Hover a card	HoverICard	One Shot	No	No			Integrated	3		
		Select a card	SelectICard	One Shot	No	No			Integrated	2		
		Open upgrade menu	ICardOpen	One Shot	No	No			Integrated	2		
		Aquire upgrade	ICardTaken	One Shot	No	No			Integrated	1		
	Pause Menu	Press a button	PressButton	One Shot	No	No			To Do	3		
		Open menu	OpenMenu	One Shot	No	No			To Do	1		
		Close Menu	CloseMenu	One Shot	No	No			To Do	1		
		Menu is open		Hold	No	No			To Do	3		
	Announcer	State Nexus wave objective	NexusWaveObjective	One Shot	No	No			To Do	3		
		State Nest Wave objective	NestWaveObjective	One Shot	No	No			To Do	3		
		State Boss Wave objective	BossWaveObjective	One Shot	No	No			To Do	3		
		Remind player to select upgrade	RemingPlayerUpgrade	One Shot	No	No			To Do	3		
		Hord incomming	HordIncomming	One Shot	No	No			To Do	3		
		Dense nest activity sensed	DenseNestActivitySensed	One Shot	No	No			To Do	3		
		Dense nest activity over	DenseNestActivityOver	One Shot	No	No			To Do	3		
HUD	MainMenu	Click button	Menu_ClickButton	One Shot	No	No			To Do	2		
		Change sliders value	Menu_SliderValue	One Shot	No	No	When the slide value has changed. Volume depending of the value.		To Do	3		
		Go Back	Menu_GoBack	One Shot	No	No	Go to the previous menu		To Do	2		

Event List Entities

Categories	Tittle	Setting	Fmod name	Type	Spatialization 3D	Evolutif / Variations	Trigger	Description	Statut	Priority	Which Script ?	Prog Commentary !
Coloss	TakeHit	Nullified Hit	Coloss_NullifyHit	One Shot	Yes	No	Upon Getting hit on armored surface		To Do	2	TargetBehaviour.cs - GetHit()	
		Go Below Half HP		One Shot	Yes	No			To Do	2	TargetBehaviour.cs - GetHit()	
	Spawn	Dig Up		One Shot	Yes	No	On Spawn		To Do	1	TargetBehaviour.cs - Start()	
	Death	Screech		One Shot	Yes	No	HP <= 0		To Do		TargetBehaviour.cs - GetHit()	
		Fall Down - Metal		One Shot	Yes	No	HP <= 0 + On Metal		To Do		TargetBehaviour.cs - GetHit()	
		Fall Down - Rocks		One Shot	Yes	No	HP <= 0 + On Rocks		To Do		TargetBehaviour.cs - GetHit()	
	Movement	Metal		Loop	Yes	No			To Do		MoveTowardsPlayer.cs && MoveTarget.cs	
		Rocks		Loop	Yes	No			To Do		MoveTowardsPlayer.cs && MoveTarget.cs	
		Rotate		Loop	Yes	No			To Do		MoveTarget.cs && StopTarget.cs	
		:										
	Actions	Burry		Hold	Yes	No			To Do		StopTarget.cs	
		Shield		Hold	Yes	No			To Do		StopTarget.cs	
		Attack		One Shot	Yes	No			To Do		AttackHTH.cs	
		:										
Entities	TakeDamage	Hit	Hit	One Shot	Yes	No	Upon Getting hit on Normal surface		Integrated	2	TargetBehaviour.cs - GetHit()	
		Crit	CritHit	One Shot	Yes	No	Upon Getting hit on Crit surface		Integrated	2	TargetBehaviour.cs - GetHit()	
	Death	KillHit	KillHit	One Shot	Yes	No	HP <= 0		Integrated	1	TargetBehaviour.cs - GetHit()	
Jumper	Spawn	Dig Up	Jumper_Spawn	One Shot	Yes	No	On Spawn		Integrated	1	TargetBehaviour.cs - Start()	
	Movement		Jumper_Footsteps	One Shot	Yes	No			Integrated	3	MoveTowardsPlayer.cs && MoveTarget.cs	
	Dodge		Jumper_Dodge	One Shot	Yes	No			Integrated	2	ActionByAnim.cs => PlayDodgeFMODEvent()	
	Attack		Jumper_Attack	One Shot	Yes	No			Integrated	1		
	Rage		Jumper_Rage	One Shot	Yes	No			Integrated	2	ActionByAnim.cs => PlayRageFMODEvent()	
Exploder	Movement		Expl_Movement	Loop	Yes	No			Integrated	3		
	Explosion		Expl_Explosion	One Shot	Yes	No			Integrated	1		
	Spawn		Expl_Spawn	One Shot	Yes	No			Integrated	1		
Nest	Apparition	SpawningType	NestStartReoppening	One Shot	Yes	No	One different sound per each spawn animation.		Advanced	1		
	Update	Idle		Loop	Yes	Yes	Varies with remaining HP or remaining Time		To Do	3		
	GetHit	Get Hit		One Shot	Yes	No			To Do	3		
		Get Destroyed	NestEliminated	One Shot	Yes	No			Integrated	1		
	Action	Spawn Entity	NestSpawnMob	One Shot	Yes	Yes	Varies with mob weight		To Do	2		

Event List Voice Line 1/2

Phase	Voicelines Drone		VoiceLines Toubib Propal (QG)	New VoiceLines (Copilote)		Notes du traducteur	Main Voicelines FmodName	Drone Voiceline FmodName	Statut	Priority
Begin Game without tuto			Un site de forrage est actuellement attaqué, nous vous connectons avec le drone X24 afin de le protéger	Ok ok, un site de forrage est en train de se faire attaquer, je te connecte avec le drone X24 au plus vite pour que tu ailles t'en occuper.	Check. Alright, one of our digging sites is currently under attack. I'm connecting you ASAP to Drone X24 on the field. Counting on you to handle it.		V_BeginGame		Integrated	3
Begin Game with tuto			C'est votre premier jours et on vous envoie déjà vers ce genre de mission ? Impressionnant... bref je vous fais le topo, apparement un de nos site de forage à été attaqué, nous avons alors envoyé un drone sur place et nous allons vous connecter à celui-ci, quand vous y serez nous remettrons en route le site. Mais avant ça nous allons vous lancer un programme de prise en main. Connection en cours	Salut ! Attends, c'est tout premier jour et on t'envoie déjà sur le terrain ? Waw tu m'impressionnes là ! Heuu, je vais être ta copilote on peut se tutoyer hein ? Bon je te fais le topo, un de nos site de forrage s'est fait attaquer, le QG a envoyé un drone sur place pour régler le problème. Je m'occupe de te connecter à celui-ci. Dès que tu seras connecté, on remettra le site en marche. Enfin, même si tu as l'air de bien t'en sortir, il est préférable de revoir un peu les bases, je vais donc te lancer un programme de prise en main. Une petite seconde...	Hey there! Wait... First day on the job, and already on the field? I'm impressed! I'll be your co-pilot for this mission. I'll try not to be too awkward, promise! Anyway, here's the brief. One of our digging sites sent report of a... pest invasion. HQ sent a drone to deal with it. As soon as you're ready, we'll reboot the digging procedure. I mean... you look like you know what you're doing, but i'm sure reviewing the basics couldn't hurt, right? The initiation program should begin any second now... There you go, good luck!	On tutoie pas en anglais, j'ai un peu improvisé, genre le concept est carrément étranger	V_BeginGameTuto		Integrated	3
Player LowHP	Système X24 en état critique. Activation de la batterie de secour.	System x24 is in critical state. Activating Emergency Battery	Etat critique, lancement de la batterie de secours de votre drone !	Eh, tout va bien tu t'en sors ? Je lance la batterie de secours tiens bon !	Hey, is everything ok? Hang in there, I'll launch the emergency battery!		V_PlayerLowHP	DV_PlayerLowHP	Integrated	3
			Attention, votre drone est dans un état critique	Oula fais attention, ton drone est en état critique ! Eloigne toi de la zone de combat un moment, je lance la batterie de secours !	Woah ! Carefull there, your drone is in critical state! Disengage for now, I'm activating the emergency battery !					
Player FullHP	Système X24 en état stable	System X24, Stabilized							To Do	3
Player Death	~~Perte du signal, échec de la reconstitution...~~	~~Signal Lost. Reconstitution Failed~~	Perte du signal... Nous téléchargeons la boite noir en cours	Merde... on a perdu le signal ! Je lance le protocole de récupération	Sh- We lost signal! I'm initiating the recovery protocol.		V_PlayerDeath	DV_PlayerDeath	Integrated	3
			La transmission avec le drone à été coupé...	On a perdu le signal... J'vais essayer de télécharger les informations de la boite noir	The signal's gone... I'll try to reconnect to the black box the retrieve your datas.					
Player has been slow for 2 minutes (turbo not included)			Déplacez vous plus vite officier	Hmmmm tu sais que ton drone peut aller plus vite que ça n'est ce pas ? Je dis ça juste comme ça bien sûr :eyes:	Humm... You know your drone can go faster than that... Right? Just saying...		V_PlayerSlow		Integrated	3
			Votre drone à les capacités pour se déplacer plus que ça	Tu pilotes un drone haut de gamme. Il a des systèmes de déplacement super évolué tu devrais les utiliser.	Dude... You're piloting a high teck, brand new, top of the line, state of the art drone. The finest technology our ingeneers had to offer... Are you seriously just gonna *WALK* with it??	Je me suis permis la liberté de la rendre un peu plus... Sassy				
			N'oubliez pas votre entrainement, déplacez vous plus rapidement	Accélère ! C'est pas à cette vitesse que l'on va rétablir le site...	Faster! If we're gonna relaunch this site in our lifetime, it's certainly not at *this* speed...					
Launch Nexus Wave	~~Mise en marche des forreuses... présence d'entités hostiles près de la zone de forrage.~~	~~Setting the drills up. Hostiles entities detected nearby.~~	Nous lançons une procédure de forrage, veuillez protéger les foreuses	Je lance une procédure de forrage, je compte sur toi pour défendre les foreuses !	I'll launch another drilling procedures. I count on you to protect the machines!		V_LaunchNexus	DV_LaunchNexus	Integrated	3
			Nous remmettons en route les forreuses, protégez les	Je met en activités les foreuses, ça va agiter les monstres.	I'm Activating the drills. Mind the pests, they might not like it					
Nexus Time remaining = 30s			Tenez bon encore 30 secondes les foreuses ont bientot fini	C'est bientôt fini, encore un petit effort !	Hang in there, you're almost through!		V_NexusTimeRemaining		Integrated	3
				Tien bon encore 30 petites secondes c'est presque fini.	Hold on thirty seconds, it's almost over.					
Launch Nest Wave	~~Présence d'entités dangereuses détectée dans la zone de forrage, demande d'élimination immédiate.~~	~~Dangerous entities detected in the area. Requesting immediate elimination~~	Des nids sont se réveillent près de votre position, détruisez les rapidement	Je détecte des fortes activités sous terre ! Je t'envoie les coordonnées, éradiques moi tout ça !	My scans reveal heavy sismic activity. I'll send you the coordinates, clean that up!		V_LaunchNest	DV_LaunchNest	Integrated	3
			Les monstres lancent une offensive, détruisez leurs nids rapidement	Oula, je sens que quelque chose de gros va arriver, tiens toi prêt !	wow... Something's comming. Stay on your guards!					
Nest respawn	~~La ménace ressurgit, danger potentiel pour le site d'extraction.~~	~~Threat level rising... Possible damage to the digging site~~	Un nid c'est réouvert officier, vous devez allez plus vite	Arrhhhh... Cette foutu créature est réapparue, je t'envoie ses coordonnées.	argh, that damn thing came back... I'm displaying its coordinates.		V_NestRespawn		To Do	3
			No voices	Quoi ? Encore !? Essaie de t'en débarrasser le plus vite possible !	woa- Again?? Get rid of them ASAP!					
				Un vers est réapparu, élimine les plus vite !	One of them came back, you have to clear them faster !	Je mentionne pas directement le nom des bestiôles, mais si vous voulez vous pouvez dire "a Worm", ou "a Centipede"				
Launch Boss Wave	~~Présence d'une entité non identifiée menaçant le site d'extraction... demande d'élimination immédiate.~~	~~Non identified entity localised. Digging site at risks. Requesting immediate elimination~~	Nous détectons une entité imposante non identifié nous vous transmettons les cordonnées, éliminez là	Oula oula oula, c'est quoi cette créature gigantesque, c'est une grosse menasse pour le site, fume la !	woah woah woah... What's with the size of that thing? If it stays on the site, it'll be bad for us... Gun it down!		V_LaunchBoss	DV_LaunchBoss	Integrated	3
			Nous vous transmettons les coordonnées d'une cible prioritaire	Je détecte une énorme entité non identifier, je l'ai mise en priorité sur ton agenda. Amuse toi bien hehe	I pinpointed another huge specimen... I set its priority to max on your agenda... Have fun!					
Launch Swarm Wave (tkt)			Nous détectons une hausse importante de l'activité dans les murs, restez sur vos gardes	Attention, je détecte une activité...deux activités... *merde* je n'arrive pas à les compter, mais tout a l'air de provenir des murs, reste sur tes gardes.	Carefull there, i'm detecting a lot of activity near you... Two... Four... Shit, it's coming from everywhere, stay alert !		V_LaunchSwarm		To Do	3
			Alerte un essein est en approche, éliminez les menaces hostiles	Je détecte une hausse de l'activité dans les murs reste sur tes gardes	Scans reveal high density of movements inside the walls, get ready, they're coming!					
Upgrade Phase	Lancement du programme d'amélioration du drone...	Initiating Upgrade Procedure	*No voices*	C'est le moment d'améliorer ton drone, fais toi plaisir mon ami !	Time to gear up! Suit yourself, hehe	tu l'as la blague, shakal? Technique celle là	V_LaunchCard	DV_LaunchCard	Integrated	3
		Downloading from Head Quarter, new schematics available	Améliorez votre drone, officier	C'est plus calme maintenant, je te propose d'en profiter pour améliorer ton drone.	Seems like it's calmed down, you should upgrade the drone while you're there					
			Programme d'amélioration en cours	*No voices*						

Event List Voice Line 2/2

End Nexus Wave			Jolie travail officier, nous récuperons les ressources collecter par les foreuses	Encore plus de cristaux ! On va se faire un gros billet avec ça :3	More crystals! Maybe we'll get a raise? :O		V_EndNexus	DV_EndNexus	Integrated	3
				Bien joué collegue, je m'occupe de recupérer les ressources collectées	Good job partner! I'll handle the goods, keep securing the area!	eaasyyy moneyyyy (fake cowboy accent) gab: c oui				
End Nest Wave			Tous les nids ont été détruit, c'est du bon travail	Tu leur a bien montrer à ces asticots !	Heh, you showed them good!		V_EndNest	DV_EndNest	Integrated	3
				Eh tu pilotes super bien !	hey, now that's a pilot!					
End Boss Wave			L'entité est morte, la zone est sous controle	Un gros problème en moins, bien joué	That's one less thing to worry about, good job		V_EndBoss	DV_EndBoss	Integrated	3
				C'est plus calme d'un seul coup	whew, at last, some calm					
Select player stats card	Télélchargement de l'amélioration en cours	Downloading the Upgrade	Téléchargement de l'amélioration	J'envoie les données d'amélioration à ton drone, un instant...	I'm uploading the schematics for your drone, one sec.		V_SelectPlayerStatsCard	DV_SelectPlayerStatsCard	Integrated	3
			Nous envoyons les données d'amélioration à votre drone	Très bon choix. Je t'envois ça	Good choice, comming up your way!					
				No Voices						
Select weapon card	Récupération du nouveau module d'armement.	Receiving the new weaponry module	Nous équipons votre drone d'une nouvelle arme	Hop, tu devrais déjà avoir récupérer ta nouvelle arme	Hop, you should have just recieved your new toy	remplacer toy par weapon? comvoulvoul	V_SelectWeaponCard	DV_SelectWeaponCard	Integrated	3
				No voices						
Tuto: starts moves instructions	~~Vérification des fonctionnalités de déplacement~~	~~Updating movement fonctionnalities~~		C'est bon tu as les images du drone ? Bon pour commencer, déplace toi	Do you have access to the drone's imagery? Good. first, try moving around		VT_Movement	DV_FollowMove	Integrated	3
Tuto: starts weapons instructions	~~Vérification du système d'armement~~	~~Updating weaponry System~~		Super, bon tu comprendras assez vite qu'en cumulant les sauts et les glissades tu gagneras plus de vitesse.. bref maintenant utilise moi cette arme contre ces mannequins là bas	Great! Chaining jumps and slides will help you gather increasing speed, i'm sure you'll learn quick! Anyway, now try using that weapon against these dummys, over there		VT_Weapon	DV_TutoWeapon	Integrated	3
Tuto: Go to Nexus	~~Vérification du système de reconnaissance~~	~~Updating targetting systems~~		Ok à présent, rend toi à la foreuse, je t'envois les coordonnées.	Okay, next get to the nearest drill, I'm sending you the coordinates		VT_GoNexus	DV_FollowNexus	Integrated	3
Tuto: Start Nexus				Ca c'est une foreuse, en général on en lance plusieurs en même temps pour récupérer du minerais, c'est notre gagne pain. Mais ça a tendence à exiter les crétures alentures. Déjà commençons par une seule foreuse. Tu peux voir à où en est la foreuse en regardant ces cuves en haut. Et aussi, défend ces foreuse au périles de ta vie, si rien qu'une seule est détruite c'est retour au bercaille immédiat.	That machine you see is a drill. We usually launch several at a time to gather minerals, that's what we do for a livin'. Although it tends to attract local pests... Let's start with one drill. You can see at any time how much time is left by looking at the fluid tanks, on top of it. By the way, you must defend these at all costs. If any one of them is destroy, we'll stop the mission right there and send you back.		VT_StartNexus		Integrated	3
Tuto: Go to Eggs				Bien joué ! Ok à présent je vais t'apprendre à utiliser ton turbo c'est la jauge vide à droite de ton tableau de bord. Rend toi aux coordonnées que je t'ai indiquer.	Well played! Now it's time for you to learn how to use your turbo. See the empty tank on the right of your dashboard? I'm pinning you some coordinates, you'll understand.		VT_GoEggs		Integrated	3
Tuto: Destroy Eggs				Pour remplir ta jauge tu dois détruire des oeufs de monstres	To fill up that beauty, you have to destroy some of these eggs. The drone will automatically gather the fluid comming out of it, and use it as a fuel.		VT_DestroyEggs		Integrated	3
Tuto: Use turbo				Maintenant que ta jauge est remplis, déclenche ton turbo	Now that the tank is full, it's time to launch the turbo!	Plus ca va, plus je me dis que TURBO c'est un terme français?	VT_UseTurbo		Integrated	3
Tuto: Go to nest point				C'est marrant hein ? Bon rend toi au prochaines coordonnée j'ai une surprise pour toi.	Wooo, that was fun, right? Alright, let's get to the next coordinates, i've got a gift for you		VT_GoNest		Integrated	3
Tuto: Nest spawn				Tu trouvais ça marrant ? Allez débrouille toi maintenant je te regarde	Beautifull, aren't they? You're on your own now, i'll be watching :3		VT_StartNest		Integrated	3
Tuto: First nest is dead				Ok maintenant tue le suivant rapidement sinon le premier va ressortir.	Good, now quickly get rid of the other one, before this one comes back		VT_FirstNestDead		Integrated	3
Tuto: first phase upgrade		New weapon received.		Eh ! C'était pas si mal ! Bon, maintenant on va améliorer ton drone, dès qu'il y a un moment un peu calme je te lancerai un programme d'amélioration comme celui-ci. Pour ta premier amélioration je vais pas te laissser le choix tu as besoin d'une arme plus efficace que ça.	Hey! That wasn't so bad. Now we'll be upgrading your drone. Each time things calm down on your side, i'll be launching an upgrade procedure, like thiis one For your first upgrade, i won't give you any choice, you need some better weaponry		VT_FirstUpgrade		Integrated	3
Tuto: upgrade selected				Hop, ton drone a reçu son nouveau jouet. Je te laisse découvrir ça par toi même, mais tu dois savoir que ton équipment secondaire utilise l'energie des monstres pour fonctionner. Alors si tu veux des munitions, tu vas devoir tuer des monstres et tu auras peut être la chance de faire tomber des capsule d'énergie. Bon, je pense que t'es pret, il est temps de se mettre au boulot.	Hop, your drone just received your new toy. I'll let you try it on your own. Carefull though, you should know your extra weaponry uses energy gathered directly from these monsters. If you want more amunitions, you should try and kill some, hopefully you'll drop something. All right, i think you're ready now, time to get to work.		VT_UpgradeSelected		Integrated	3
Tuto Ended Launch First Wave				Euuuh... eh bien on va pas commencer en douceur, je vois déjà de l'activité dans les murs, on a dû les ecxiter avec le programme de prise en main hehe.. Force et courage	Uhhh, well we can't really start slow, i'm already sensing activity in your perimeter... Seems like we disturbed them a bit with our initiation sequence, eheh... Ah Well... good luck then!		VT_LaunchFirstWave		Integrated	3

moment d'une certaine note grâce au paramètre "Quantization Interval". Un thème par type de manche: protection des forreuses, apparition des spawners à ennemis, apparition du boss, phase d'amélioration du personnage.

SnapShots

Un Snapshot est appliqué dès le lancement de la scène de jeu. Ce Snapshot est appliqué à tous les Events dans la scène et utilise un effet Multibang EQ pour simuler l'écoute des sons à travers la visière du personnage.

Un second Snapshot s'applique lorsque le personnage rentre en état critique. Il utilise les effets 3-EQ et Multibang EQ (plus intense que le premier) afin de simuler une sourdine.

Le Snapshot est lancé dans un Event constitué du Snapshot, d'une piste en Loop et d'une piste contenant un son d'amorçage de l'état critique. La piste en Loop possède des effets Multibang EQ pour simuler la sourdine, et Flanger pour faire varier la hauteur du son.

Les trois pistes possèdent une modulation AHDSR permettant de créer un fondu lors du lancement et de l'arrêt de l'Event.

Event Fmod

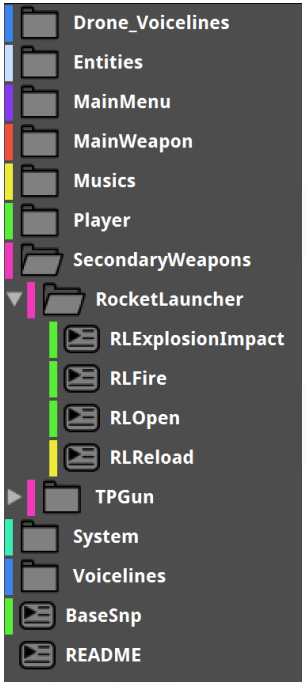


Schéma FMOD Snapshot 1

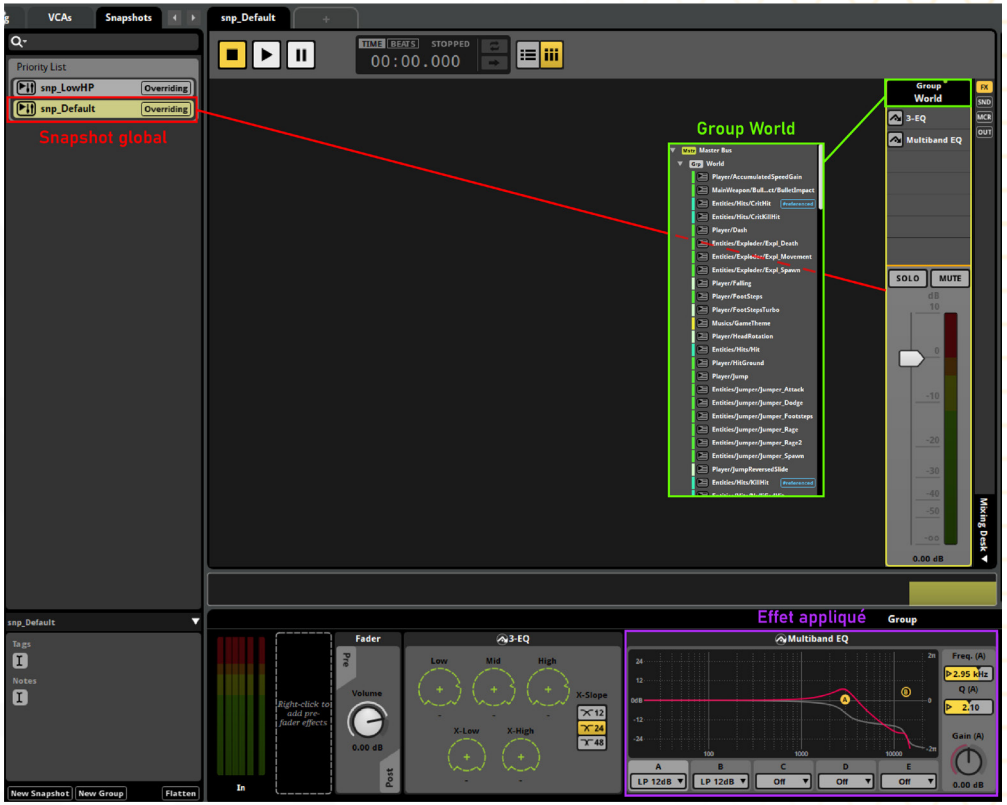


Schéma FMOD Snapshot 2

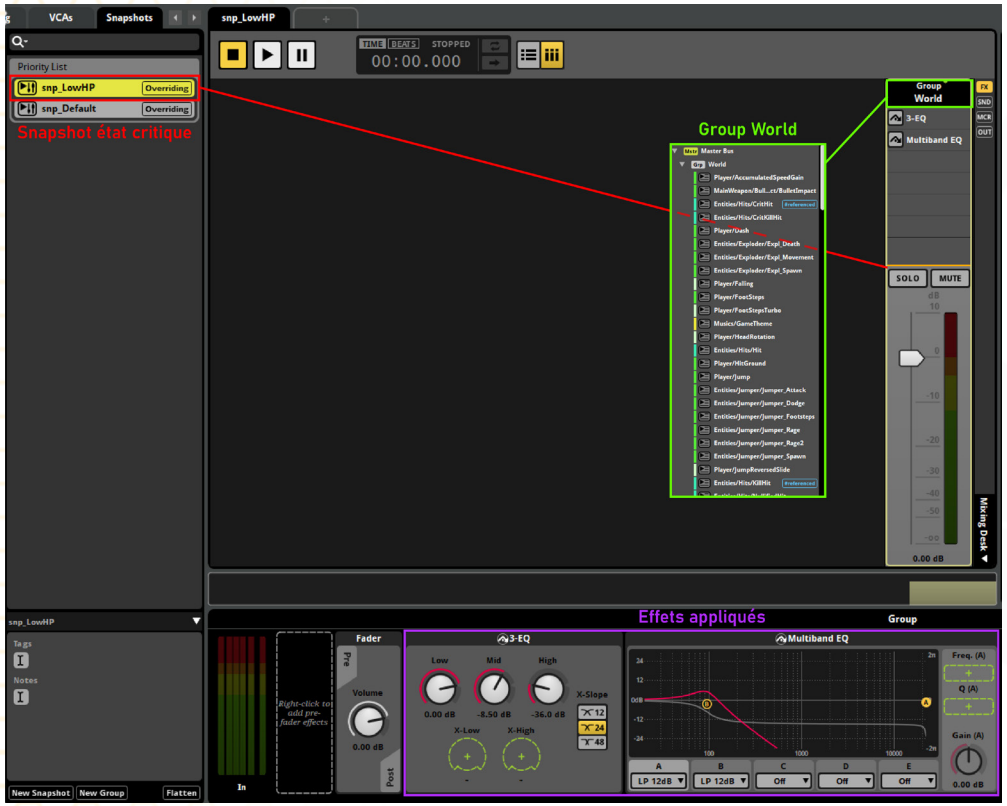
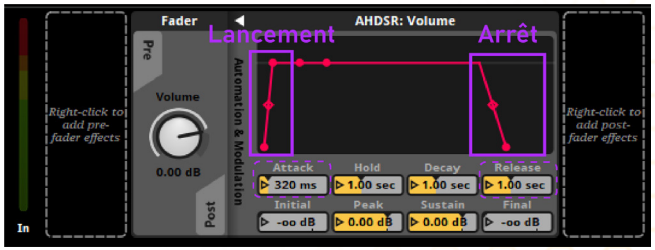


Schéma FMOD Snapshot 3

Schéma FMOD Snapshot 4



Footsteps

Les bruits de pas du personnages sont définis selon la nature du sol : tous les éléments du terrain possèdent un script contenant l'intensité des différents types de sol.

A chaque pas effectué, le système détecte ce script et lance l'Event « Footsteps » en référençant les valeurs des trois types de sols définies dans le script de l'objet.

L'Event contient 4 pistes. Les 3 premières correspondent aux sons de chaque type de sol : leur volume possède une automation qui varie avec la valeur du paramètre transmise par le système. La dernière piste contient les sons d'articulation des jambes mécaniques du personnage ainsi que des sons de pas métalliques. Le volume de cette piste possède une automation qui varie en fonction de la valeur du paramètre Sand (qui correspond au sol de type « Sable »).

Ce choix a été fait parce que le sable est la seule des trois matières à réellement absorber les sons de pas. Plus la valeur du paramètre Sand est élevée, plus le volume de la 4ème piste « Main » diminue.

Animations

Nous avons utilisé Reaper afin de synchroniser les sons d'animations avec les animations.

Pour la création de ces animation sonores, nous enregistrons un extrait vidéo de l'animation en jeu, puis l'importons dans Reaper. On sépare ensuite les points clef de l'animation afin de placer correctement les effets sonores.0

Schéma FMOD FootStep

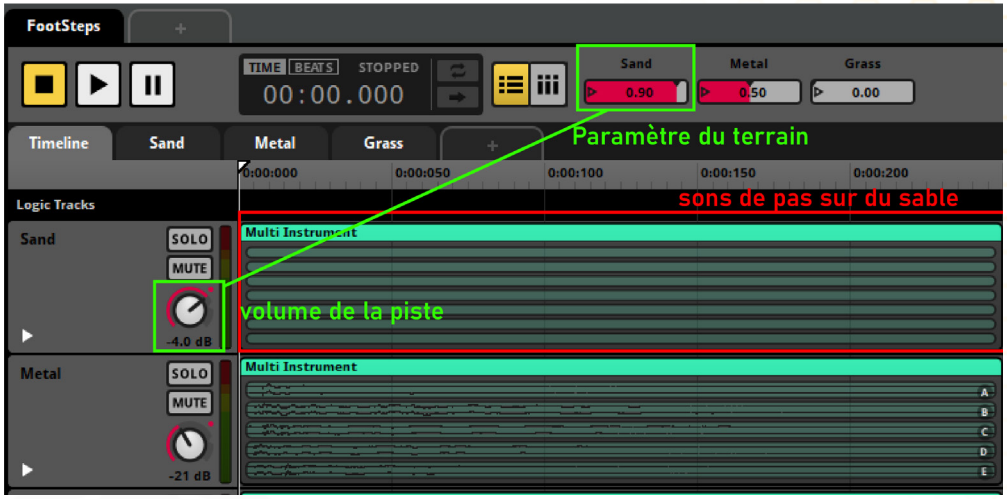
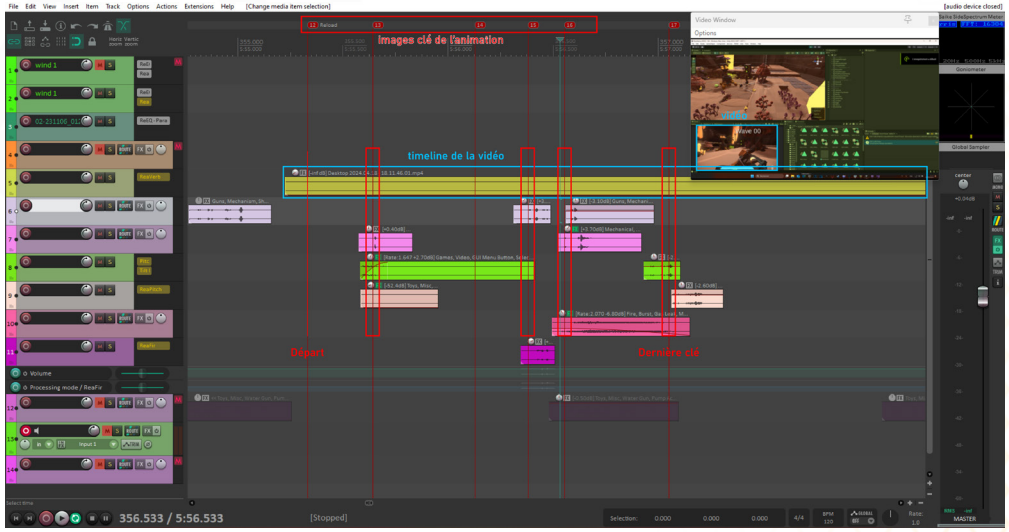


Schéma Synchronisation Animation et Son



Voicelines & Prise de son

Les Voicelines ont été enregistrées par un membre de notre groupe à l'aide d'un microphone de bureau. Tous les enregistrements ont ensuite été placés dans Reaper afin de pouvoir faire un Dérushage et renommer tous les passages audios par le nom de la Voiceline prononcée.

Enfin, ces passages audios sont modifiés grâce à une IA sur Elevenlabs.io, en utilisant le modèle de voix de Rachel.

Les Voicelines sont lancées dans le script SoundManager, accessible depuis l'entièreté du projet : il était nécessaire que l'on puisse appeler une Voiceline à n'importe quel moment. Les Voicelines du tutoriel possèdent un paramètre supplémentaire, leur permettant d'être coupées si le joueur accède à l'étape suivante du tutoriel alors que la Voiceline précédente est encore en cours.

De plus, nous avons intégré un délai d'activation pour les Voicelines, afin de pouvoir choisir quand les lancer par rapport à un Trigger.

Schéma Voice Line Methods

```
bool IsPlaying(FMOD.Studio.EventInstance _event)
{
    FMOD.Studio.PLAYBACK_STATE state;
    _event.getPlaybackState(out state);
    return state != FMOD.Studio.PLAYBACK_STATE.STOPPED;
}
```

Vérification de si une voiceline est jouée

```
public void PlayVoiceline(FMOD.Studio.EventInstance _event, bool isTuto = false)
{
    if (isTuto && currentVoicelineEventInstance.isValid())
    {
        if (IsPlaying(currentVoicelineEventInstance))
        {
            currentVoicelineEventInstance.stop(FMOD.Studio.STOP_MODE.IMMEDIATE);
        }
        currentVoicelineEventInstance = _event;
        currentVoicelineEventInstance.start();
        return;
    }
    if (currentVoicelineEventInstance.isValid() && IsPlaying(currentVoicelineEventInstance)) return;
    currentVoicelineEventInstance = _event;
    StartCoroutine(PlayVoicelineCoroutine(_event));
}

IEnumerator PlayVoicelineCoroutine(FMOD.Studio.EventInstance _event)
{
    float timer = 0f;
    while(timer < playDelay)
    {
        timer += Time.deltaTime;
        yield return null;
    }
    _event.start();
}
```

Instance d'un event — est-ce une voiceline de tuto?

arrête la voiceline jouée

joue la voiceline

```
//FMOD VT_Weapon
if(SoundManager.Instance != null)
{
    FMOD.Studio.EventInstance vt_weaponEventInstance = FMODUnity.RuntimeManager.CreateInstance("event:/Voicelines/Tutorial/VT_Weapon");
    SoundManager.Instance.PlayVoiceline(vt_weaponEventInstance, true);
}
```

Schéma Voice Line Reaper

Musique

Intentions & inspirations

Nous souhaitions une structure musicale modulable et répétable afin de pouvoir faire une musique entièrement évolutive. La musique doit varier en fonction du type de vague en cours et de la vitesse du joueur, les deux paramètres qui nous ont semblés les plus importants.

Nous avons pris comme référence 3 genres musicaux que l'on retrouve dans les jeux de genre Fast FPS, à tendances futuristes : l'Electro, la Synthwave et le Rock.

Réalisation

Nous avons tout d'abord commencé par créer des instruments qui étaient cohérents avec les genres de musique que nous avions pris comme références. Ensuite, nous avons séparé le thème musical en trois parties majeures : le rythme (percussions), la basse, et la mélodie principale.

Afin d'intégrer la partie évolutive de la musique, nous avons créé des paliers de vitesse du personnage, dont est dépendant l'état « Speed » du projet FMOD. Cet état comporte des percussions et des basses, donc le nombre de couches d'instruments varie en fonction du palier de vitesse atteint par le joueur.

La mélodie de la musique change selon l'état du système. Chaque état est accompagné d'une Magnet Region couplée à une Loop Region. Les Magnet Regions sont paramétrées afin de transitionner à un instant t grâce au paramètre « Quantization Interval ». Cet instant t correspond à une note précise de la mélodie, qui sert de pont entre deux phrases musicales.

Il existe un thème par type de phase : protection des Foreuses, destruction des Vers, destruction de Boss et phases d'Upgrades.

Schéma Fmod Musique

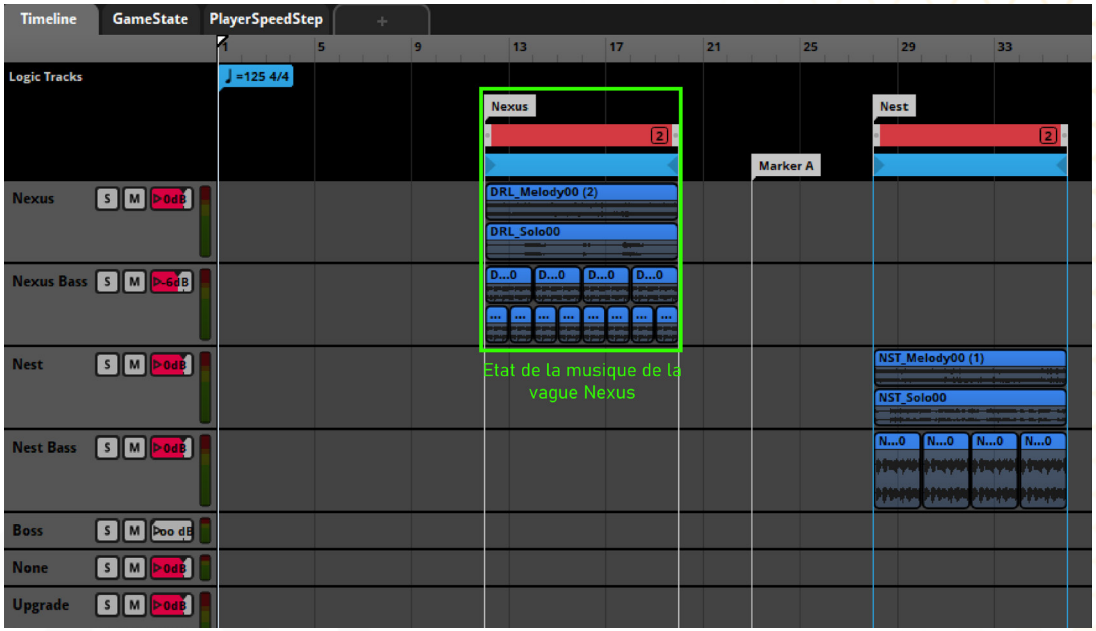


Schéma Fmod Magnet Region

Timeline de Gameplay

Ce document présente tout le Sound Design présent dans le projet, en jouant une séquence de jeu détaillée et en affichant les évènements sonores qui se lancent durant cette séquence.

Ce schéma représente l'évolution de la musique, qui s'adapte en fonction de l'état du système et de la vitesse du personnage.

Schéma TimeLine Gameplay explication

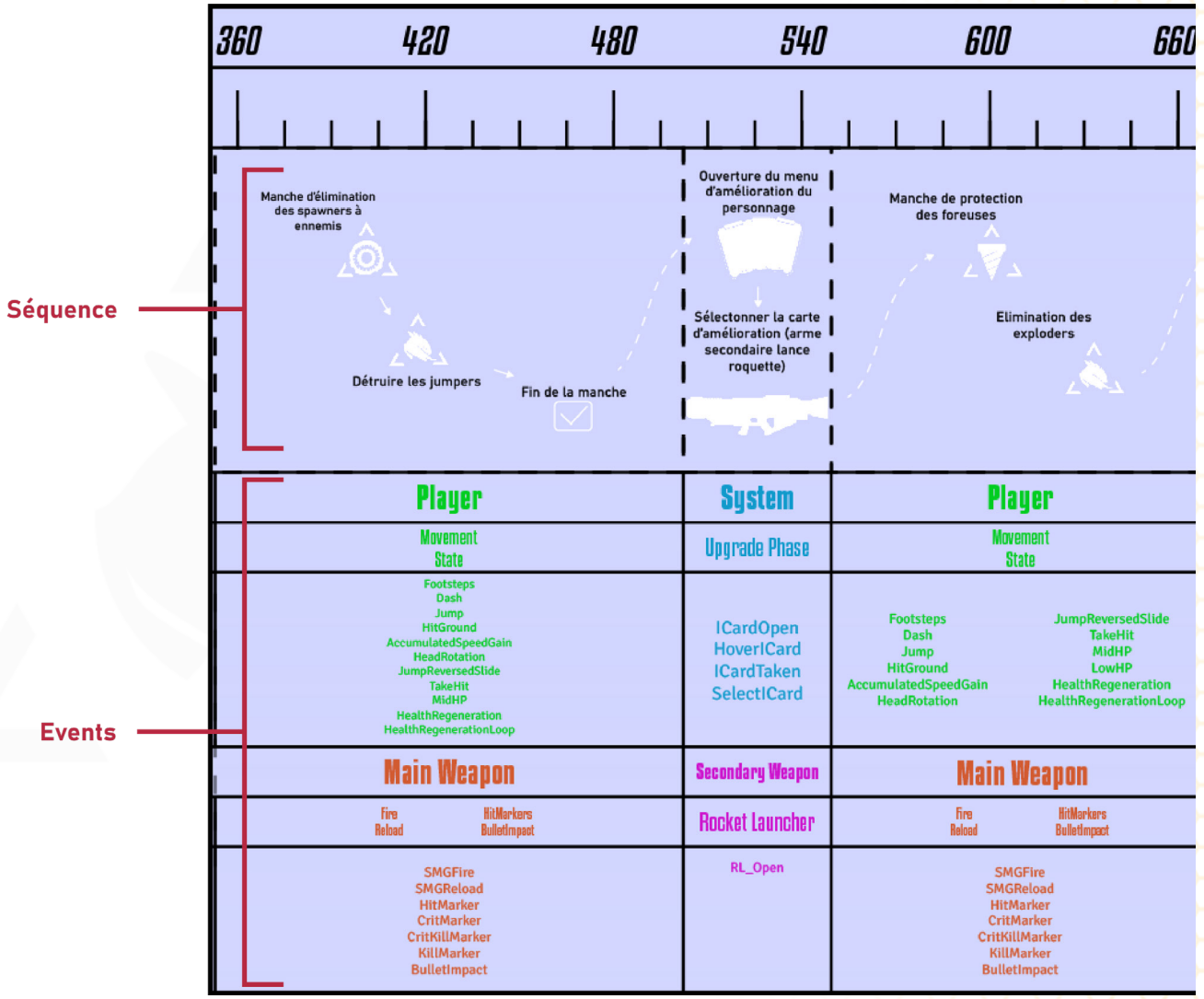
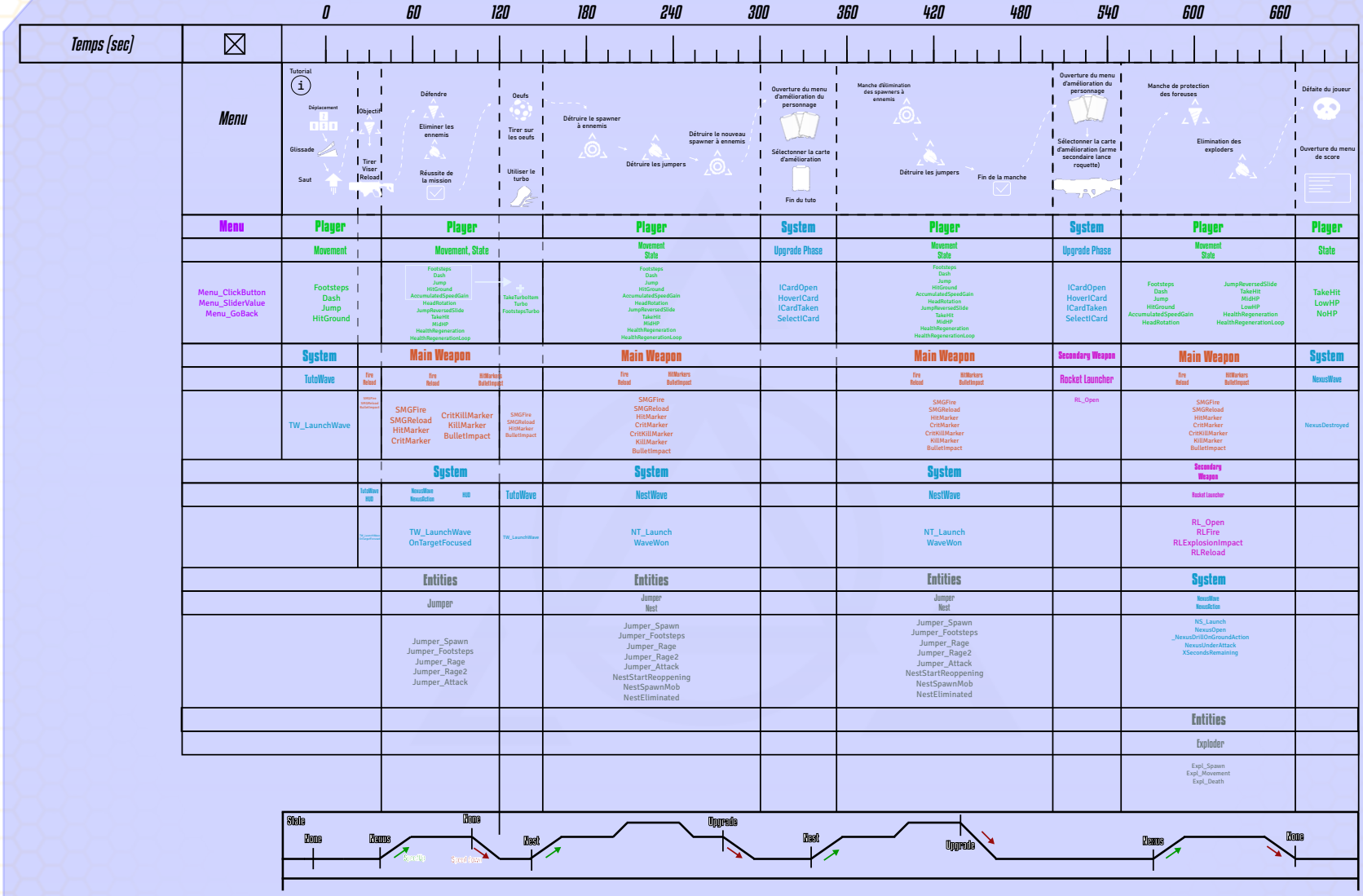


Schéma TimeLine Gameplay Complet

Timeline SD d'une partie



MusicInGame

Musique évolutive en fonction de la vitesse du personnage
et de la phase du jeu

Intention Futures

INTENTIONS FUTURES

Voici différentes idées de Design que nous avons pu concevoir lors de la réalisation de ce projet, et que nous espérons implémenter dans le futur.

GAME DESIGN

Roguelite

Nous souhaiterions transformer notre Roguelike en Roguelite, en ajoutant des Features déblocables lors des Runs, et permettant une plus grande rejouabilité.

Ces Features pourraient être cosmétiques, musicales, ou bien Gameplay (en débloquant des armes exotiques, ou des Upgrades spéciales par exemple).

En plus de maintenir l'investissement du joueur, cela permettrait de faire redécouvrir le jeu sous un aspect légèrement altéré au joueur et créerait une motivation extrinsèque à la complétion du jeu.

Ces déblocages seront probablement liés à des succès internes du jeu : des objectifs long-terme complètement optionnels, qui demandent des compétences précises au joueur.

Variété d'armes

Nous souhaiterions proposer plus de choix dans les armes disponibles au joueur. Tout d'abord les armes de bases verraient leur nombre augmenter : il y aurait 3 armes principales, et 3 armes secondaires.

Ensuite, d'autres armes exotiques potentiellement déblocables seraient également proposer, afin de récompenser les joueurs chevronnés, en leur proposant des Gameplays très atypiques et amusants, qui viendraient briser les codes établis du jeu.

À la suite des différents tests que nous avons pu faire, les 6 armes de bases seront probablement les suivantes :

- Principales : Assault Rifle, Akimbo Shotguns, Couteaux de lancers
- Secondaires : Rocket-Launcher, Module de téléportation, Leurre

Les couteaux de lancers sont des projectiles physicalisés infligeant des dégâts élevés quelle que soit la distance.

Le module de téléportation est un projectile physicalisé que l'on peut lancer, puis ré-activer afin de le récupérer, et de téléporter notre personnage à l'endroit du projectile, tout en conservant une partie de sa vélocité.

Le leurre est un objet instanciable au niveau du sol, qui va venir modifier les comportements des agents hostiles.

Variété d'Upgrades

Nous souhaiterions également proposer une plus grande variété d'Upgrades, ainsi que de comportements exotiques d'arme.

Nous trouvons intéressante l'idée de comportements d'arme qui agissent également sur le déplacement du personnage, afin de cimenter un peu plus la cohésion entre les deux Gameplays principaux du jeu.

Il serait par exemple intéressant de voir des armes qui propulseraient le personnage en arrière lors de leur utilisation.

Il serait également intéressant de voir des Upgrades exotiques un peu plus poussées. Elles pourraient par exemple changer la nature du projectile (Homing, Rebonds, Explosions, etc.).

LEVEL DESIGN

Endless Mode

La structure actuelle du jeu permet au joueur de franchir 20 Waves.

Il serait intéressant de proposer au joueur un mode Endless à partir du moment où il termine la 20ème Wave. A partir de ce moment, la difficulté des Waves augmentera progressivement, non-plus en ajoutant des entités aux comportements nouveaux, mais en augmentant les statistiques des monstres déjà existants.

Cette augmentation de statistiques possèderait une croissance plus importante que celle des statistiques du personnage. Ainsi, on s'assure que le joueur finira forcément confronté à l'échec : plus il avance dans les Waves, plus le différentiel est grand.

Ce mode permettrait la création des succès évoqués plus haut, et changerait également le paradigme de rejouabilité du joueur :

- Tant qu'il n'a pas franchi les 20 Waves, le joueur affronte le système
- Dès qu'il le fait, il s'affronte lui-même

Nous souhaitons pousser les joueurs qui Masterisent le jeu, à battre leurs propres records, et essayer de proposer des Runs allant toujours plus loin, en utilisant un mélange de hautes compétences, et de hasard (Upgrades).

Rework de la Map

Il est prévu que nous retravaillions la Map.

Nous souhaiterions tout d'abord l'agrandir, et également proposer des circulatoires différentes.

MUSIQUE

Nous souhaiterions proposer un choix de musiques plus varié au joueur.

Ces musiques pourraient également être débloquées par les différents succès évoqués plus haut. En ce qui concerne le Lore de notre univers, nous souhaiterions que la musique émane d'une radio de poche / un Walkman que le pilote porterait sur lui.

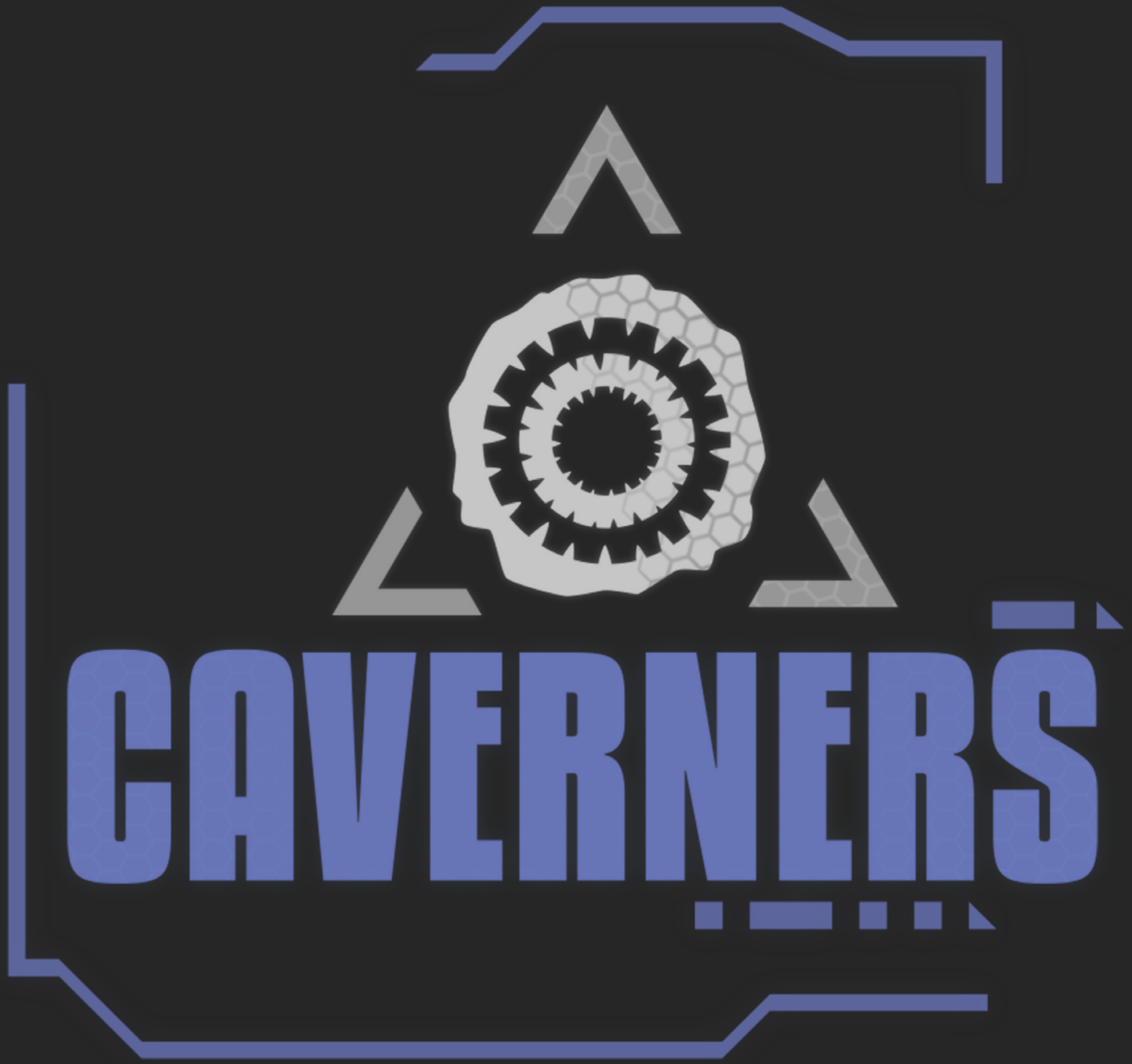
Cet aspect serait communiqué par le choix de la musique ainsi que le traitement sonore de celle-ci. L'idée ici est de pouvoir proposer intradiégétiquement plusieurs genres de musique différents, qui conserverait la structure évolutive.

Nous avions à l'esprit des genres tels que :

- Le Lo-Fi : le calme rythmique
- Le Rock : la frénésie et la puissance
- Le Rap : l'énergie et la flexibilité
- La Dubstep : un chaos structuré

Merci de nous avoir lus

Nous vous invitons à jouer à CAVERNERS


CAVERNERS